# Discrete learners against gradient-descent-trained networks: Boolean targets and standard benchmarks

Claude (Anthropic)*
prompted by Kaarel Hänni

27 October 2026

**Abstract**

A companion paper described gradient descent on Boolean targets as noise-limited greedy growth of features. Here we ask whether a *discrete* learner, one with no continuous feature parameters, can do at least as well as gradient-descent-trained networks at equal compute. We ask it first on the Boolean toy targets of that study and then on standard benchmarks. There are four steps.

*A discrete learner for the toy targets.* DFG (discrete feature growth) keeps the networks' strategy and replaces their search. It grows Fourier characters greedily, testing each against the noise of the current residual, and it makes compositions of features already found cheap. It also fits a linear background and reads out with the sign of a fit. Its search is exhaustive enumeration with union-bound tests, run within a fixed share of one network run's FLOPs. The references are $\mu$P two-hidden-layer MLPs trained by Adam on the same training sets, at $d = 30$. On a blind suite of 7 held-out targets, DFG reaches 10% test error with less data on all 7: a median factor of 3.1, with at most 0.26 of one network run's FLOPs per fit. On 41 development targets the median factor is 5–6, and it grows with the input dimension. Heuristically, enumeration pays for search in compute and about $2k \ln d$ samples for a $k$-sparse parity, where the networks pay about $d^{k-1}$ samples.

*The same learner on standard tasks.* Three independently built adaptations of DFG were run on 18 classification tasks (MNIST, Fashion-MNIST and 16 PMLB datasets). On tabular data they are about level with a plain MLP, and their error is 12–17% higher than an L2-regularized MLP's at $n \leq 3000$. On images they are clearly behind at large $n$, closer to logistic regression than to the MLP.

*A discrete portfolio for both.* A pre-registered portfolio stacks DFG, LightGBM, ExtraTrees, nearest-neighbour votes and logistic regression on out-of-fold predictions. It was compared with the networks and MLPs on two fresh sets fixed before any design.

- *Fresh toy suite.* It needs clearly less data than the per-target best of three network configurations on 9 of 10 targets. The mean $\log_2$ ratio of the data needed is $-1.41$. It needs more data on one symmetric function, $[|x_1 + \cdots + x_9| \leq 1]$ (1229 samples against 350).
- *Fresh tabular datasets.* Its mean error is below the hindsight-best MLP's on 8 of 10. This is not significant (sign test $p = 0.11$).
- *Images.* It is slightly ahead of every MLP up to $n = 3000$, with 1 of 8 cells resolved. At $n \geq 30000$ it is clearly behind a regularized ensemble of three MLPs.

*Written by an AI system in a series of sessions prompted and steered by Kaarel Hänni. Not reviewed by a human. The comparisons were checked by independent AI auditors and the write-ups by independent AI claims checks, as described in Appendices A and D. Code, result files, audit reports and the underlying write-ups are on the `claude/nifty-mccarthy-j0lw2a` branch of https://github.com/kaarelh/AI-works, folder `gd-circuit-growth/`.

Post-hoc ablations put the toy margins on the character search and the real-data margins on the classical stack: the portfolio is an AutoML-style dispatch by input type, not one discrete mechanism.

*Sum features and permutation-invariant image members.* Part 9 targets the two losses. It adds features that are functions of signed sums of inputs, admitted by a union bound over all subsets and signs, and an image member built from subset sums over pixel sets chosen by correlation, so that it never uses the pixel layout. On a held-out suite of 12 Boolean targets whose family was written by an agent blind to the design, the new portfolio needs clearly less data than the best of the pre-registered network variants on 10 targets and more on none (mean $\log_2$ ratio $-1.45$; $-1.25$, with 2 targets worse, when two more weight decays are added to the networks). On the 9 sum-structured targets Part 8's portfolio is behind, at $+0.63$ against the new portfolio's $-1.53$ ($+0.91$ against $-1.26$ with the two added variants). On two fresh 10-class QuickDraw image sets it beats a strong MLP ensemble at $n \geq 10000$ (mean $\log_2$ error ratio $-0.18$) and has the lowest error of every permutation-invariant reference at 13 of the 14 cells (7 training sizes on each set), but a CPU-matched ensemble of wider MLPs ties it at $n = 60000$, and on one set has slightly lower error there (14.8% against 14.9%). Controls qualify the image result. The pooling sets recover the pixel grid from correlations; an MLP given the same pooled sums matches the portfolio at $n = 60000$; the winning member is gradient-boosted trees, whose gain splits between the pooled features and longer boosting; a fit at $n = 60000$ takes 20–30 CPU minutes; and a small CNN that sees the pixel layout, run on one set at two sizes, has lower error at both, clearly so at $n = 10000$.

Independent audits checked leakage, compute accounting, discreteness and the strength of the references in Parts 8 and 9. Their findings strengthened the references, added a blind test suite and post-hoc controls, and changed the wording of the claims; no learner was changed after its freeze except for one recorded crash fix.

# Contents


**1 Introduction** **4**

**2 Setup** **6**
2.1 Toy Boolean targets and the networks . . . . . 6
2.2 Standard tasks and the MLP references . . . . . 8
2.3 Pre-registration, held-out discipline and audits . . . . . 10

**3 DFG: discrete feature growth on the toy targets (Part 6)** **11**
3.1 The learner . . . . . 11
3.2 Why exhaustive enumeration pays about $\ln N$ in samples . . . . . 12
3.3 Versions and what is blind . . . . . 13
3.4 Results against the networks . . . . . 14
3.5 Ablations . . . . . 16
3.6 Limits . . . . . 17

**4 DFG on standard tasks (Part 7)** **17**
4.1 Three adaptations . . . . . 17
4.2 Results . . . . . 18
4.3 Why it fails . . . . . 20

**5 A discrete portfolio on both kinds of task (Part 8)** **21**
5.1 Protocol and designation . . . . . 21
5.2 The primary design . . . . . 22

5.3 Toy results . . . 23
5.4 Tabular results . . . 26
5.5 Images . . . 27
5.6 What carries which result . . . 28
5.7 Compute and prediction cost . . . 29
5.8 The secondary designs . . . 30
5.9 What "discrete" means, and the honest claim . . . 31

**6 Sum features and permutation-invariant image members (Part 9) 32**
6.1 Question and protocol . . . 32
6.2 New data . . . 34
6.3 Sum features: DFG v1.2 . . . 34
6.4 Image members . . . 36
6.5 Integration and freeze . . . 37
6.6 Held-out results: toy suites . . . 37
6.7 Held-out results: images . . . 39
6.8 Compute . . . 42
6.9 Audits, exposures and deviations . . . 42
6.10 Answers to Q1 and Q2 . . . 43

**7 Discussion 44**

**A Protocols and audits 49**
A.1 Part 6 . . . 49
A.2 Part 7 . . . 49
A.3 Part 8 . . . 52

**B Per-suite and per-dataset tables 54**

**C Compute accounting 58**

**D How this work was done 60**

# 1 Introduction

A companion paper [5] (Parts 1–5 of this study) trained rich-regime networks on Boolean targets and described what gradient descent does there. It grows features greedily, one bundle at a time. A feature's cost is set by how hard its correlation with the current *residual* is to detect against the sampling noise of the directions it competes with. Features on inputs already in use are cheaper. A near-kernel linear background is present from the start. On every plateau the network computes the sign of a truncated Fourier expansion of the target. The cost of this search is paid in samples. For a $k$-bit parity on $d$ inputs, the networks' sample complexity $n^*$ grows like $d^{1.0}$, $d^{1.9}$ and $d^{2.4}$ for $k = 2, 3, 4$. This is close to the leap and correlational-SQ rate $d^{k-1}$ [1, 2], and far from the $\approx k \log d$ that an Occam learner needs.

This paper follows the picture in the other direction. If gradient descent is a particular, noisy way of searching a discrete space of features, can a learner that searches the same space by other means do at least as well, with the same data and compute? We asked this in four steps. Each question was posed by the user who prompted the work. In each step the learners were frozen before they were run on the held-out data that the step reports as its main evidence.

1. **Part 6** (Section 3). *Use the picture of what gradient descent does to build a more discrete learner that, with similar compute and data, does at least as well on the toy problems where gradient descent does well.*
2. **Part 7** (Section 4). *How well does that learner do on standard tasks where MLPs are reasonable, compared with MLPs?*
3. **Part 8** (Section 5). *Is there one discrete method that beats gradient-descent-trained networks both on the toy suites and on the standard tasks?*
4. **Part 9** (Section 6). *Continue in the two directions where Part 8's portfolio lost:* symmetric targets, by features that are functions of signed sums of inputs, and images at large $n$, by discrete members that do not use the pixel layout, tested on fresh image sets.

**What was found.** All results are [Numerical]. The readings are [Heuristic].

- **On the toy targets the discrete learner wins clearly** (Section 3). DFG enumerates Fourier characters exhaustively and admits them by union-bound tests against the residual. It needs less data than the study's networks on every target of a blind 7-target suite (median factor 3.1) and on 11 of 12 targets of an earlier held-out suite. On development targets the factor grows with $d$: for the 3-bit parity the networks need 843, 3410 and about 14000 samples at $d = 30$, 60 and 120, and DFG needs 28, 28 and 56. Every fit uses less than half of one network run's FLOPs.
- **On standard tasks it does not** (Section 4). Three independent adaptations of DFG to real-valued, multi-class data are about level with a plain MLP on tabular data. Their error is 12–17% higher than an L2-regularized MLP's at $n \leq 3000$, and default gradient boosting has 8–12% lower error than they do. On MNIST and Fashion-MNIST at $n = 60000$ their error is 2.2–2.9× and 1.3–1.4× the MLP's.
- **A portfolio wins on the toy suites and on fresh tabular data on average, but not on images at large** $n$ (Section 5). The pre-registered primary design stacks DFG with LightGBM, ExtraTrees, nearest-neighbour votes and logistic regression.

– *Fresh toy suite.* It needs clearly less data than the per-target best of three network configurations on 9 of 10 targets, and loses on one symmetric function.

– *Fresh tabular datasets.* Its mean error is lower than the hindsight-best MLP's on 8 of 10 (sign test $p = 0.11$).

– *Images.* It is slightly ahead of every MLP up to $n = 3000$ and clearly behind a regularized MLP ensemble at $n \geq 30000$.

The toy margins come from DFG's character search. The real-data margins come from the classical stack, and DFG runs on only 1 of 23 held-out real datasets.

- **Part 9 removes both of Part 8's losses, with qualifications** (Section 6).

  – *Sum features.* On a held-out suite of 12 signed-sum targets and controls written by an agent blind to the design, the integrated portfolio needs clearly less data than the per-target best network on 10 of 12 targets and more on none, with mean $\log_2 n^*$ ratio $-1.45$ against the pre-registered network variants ($-1.25$ and 2 targets worse with two more weight decays). On the 9 sum-structured targets Part 8's portfolio is behind the same networks, at $+0.63$ (3 targets clearly better, 4 clearly worse) against the integrated portfolio's $-1.53$. Two earlier held-out suites agree but are exposed to the design.

  – *Images.* On two fresh 10-class QuickDraw sets the portfolio beats the strong MLP ensemble at $n \geq 10000$ (mean $\log_2$ ratio $-0.18$) and has the lowest error of every permutation-invariant reference at 13 of the 14 cells; a CPU-matched ensemble of wider MLPs ties it at $n = 60000$ and has slightly lower error at the 14th cell (14.8% against 14.9%). But an MLP given the portfolio's correlation-pooled pixel sums matches it at $n = 60000$; the pooling sets recover the pixel grid from data; the winning member is gradient-boosted trees; a fit at $n = 60000$ takes 20–30 CPU minutes; and a small CNN on the unpermuted pixels (one set, two sizes) has lower error still, clearly so at $n = 10000$.

**What it means** [Heuristic]. On sparse Boolean targets, exhaustive enumeration trades compute for samples. A union-bound test over $N$ candidates needs about $2 \ln N/\rho^2$ samples to admit a feature of residual correlation $\rho$. That is about $2k \ln d/\rho^2$ for a $k$-sparse character, and at $d = 30$ the enumeration fits inside one network run's FLOP budget. The networks, searching by noisy local steps in weight space, pay about $d^{k-1}$ samples instead. This is what SQ and leap theory lead one to expect [1–3, 8, 16]. The experiments add measured factors at equal FLOPs, on fresh targets.

On real data the premise fails. The useful features of images are directions spread over many correlated pixels. Such directions form a continuous family that no small dictionary enumerates, and there local search is the efficient search. The portfolio's tabular wins restate the known strength of tree ensembles and stacking over plain MLPs on medium-sized tabular data [9, 12, 25]; well-tuned MLPs narrow or close that gap [11, 14].

**Contributions.**

1. A discrete learner, DFG, built component by component from a mechanistic picture of gradient descent. It is tested blind against the networks that picture was drawn from, at equal FLOPs, with ablations of each borrowed ingredient (Section 3).

2. A negative result on standard benchmarks: three independent adaptations of the same recipe, frozen on development data, with an analysis of why it fails (Section 4).

3. A pre-registered test of a discrete portfolio against network references on both kinds of task, with the following safeguards (Section 5):
    - fresh held-out sets fixed before design, and a designation rule fixed before any held-out run;
    - independent audits of leakage, compute, discreteness and reference strength, whose findings strengthened the references;
    - post-hoc ablations that attribute each result to a component.
4. A record of how the work was done, including what the orchestrating session had seen when it made each change (Appendices A and D).

**Scope.**

- *The toy networks.* They are one architecture family: $\mu$P two-hidden-layer ReLU MLPs of width 128, trained by Adam for a fixed number of steps. Weight-decayed and logistic-loss variants were added in Part 8, and two logistic-loss variants with stronger weight decay on Part 9's blind suite.
- *The MLP references.* They are two-hidden-layer ReLU MLPs of width 256 in Parts 7–8, from a plain baseline to regularized ensembles (weight decay, dropout, three networks). Part 9 adds a 512-wide ensemble, the strongest MLP reference on QuickDraw at every $n$ from 3000 to 30000, and a CPU-matched 1024-wide ensemble at $n = 60000$, which is stronger still there. There is no augmentation and no architecture tuning. A small CNN on the unpermuted pixels was run only as context, at two cells, and is not a reference.
- *What DFG was designed for.* It was designed knowing that the toy targets are Boolean functions of uniform $\pm 1$ inputs, mostly sparse in the Fourier basis. Its toy result shows that such a specialised learner fits within gradient descent's FLOP budget and needs less data. It says nothing about learners in general.
- *Size of the held-out sets.* The fresh sets are small: 10 and 12 toy targets in Parts 8 and 9, 10 tabular datasets, and two image sets in Part 9.
- *What "discrete" means.* It means no continuous feature parameters. Gradient boosting is functional gradient descent over trees, and DFG's spectral block uses eigenvector directions (Section 5.9). The fair wording is "beats gradient-descent-trained networks", not "beats gradient descent".

Claims are labelled [Numerical] (measured here) or [Heuristic] (a reading or a back-of-envelope argument, not tested or proved here). There are no theorems in this paper.

# 2 Setup

## 2.1 Toy Boolean targets and the networks

**Targets.** Inputs are uniform on $\{\pm 1\}^d$, mostly with $d = 30$, and labels are $\pm 1$ values of a target $f$. Products of inputs are parities (Fourier characters) $\chi_S(x) = \prod_{i \in S} x_i$. Parts 6–8 use the five suites below; Part 9 adds four more (`dlsymdev`, `dlheld4`, `dlheld5`, `dlheld6`; Section 6.2). Their roles differ by part, and Table 1 gives the status of each.

- `designed` (21 targets), from Parts 1–5:

  - a dictator, $\mathrm{AND}_3$, $\mathrm{MAJ}_3$, $\mathrm{MAJ}_9$, a 10-input halfspace;
  - the parities $\chi_2$, $\chi_3$, $\chi_4$ and a 5-term staircase;
  - compositions of majorities and parities, a multiplexer, a 3-term DNF, a decision list, $\mathrm{sign}(2x_1 + x_2x_3 + x_4x_5x_6)$;
  - four random 6-gate formulas.

  Seven of them were also run at $d = 15$ and 60, and four at $d = 120$.

- `random` (20): random fan-in-2 AND/OR/XOR formulas with 3–8 gates, each depending on 3–5 inputs.
- `dlheld` (12, "held-out 1"), written for Part 6 before DFG was tuned on it:
  - dense halfspaces and majorities: $\mathrm{MAJ}_{15}$, a Gaussian halfspace on all 30 inputs, MAJ of three $\mathrm{MAJ}_5$, $\mathrm{MAJ}_5 \oplus \mathrm{MAJ}_5$, an intersection of two halfspaces;
  - $\chi_5$, the 6-multiplexer, a 6-term staircase, MAJ of five pairs;
  - three random formulas on 6 inputs.
- `dlheld2` (7, "held-out 2"), written after DFG v1.1 was committed: a 20-input halfspace, $\mathrm{MAJ}_{21}$, $x_1x_2\,\mathrm{MAJ}(x_3, x_4, x_5)$, a 4-term 2-DNF, the symmetric function $\mathrm{sym8} = [|x_1 + \cdots + x_8| \geq 4]$, and two random formulas on 7 inputs.
- `dlheld3` (10), fixed for Part 8 before any Part 8 design (Table 9):
  - $\mathrm{cnf3} = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_4 \vee x_5 \vee x_6) \wedge (x_7 \vee \bar{x}_8 \vee x_9)$; dlist8, a decision list on 8 literals; $\mathrm{thrint} = \mathrm{sign}(3x_1 + 2x_2 + 2x_3 + x_4 + \cdots + x_9)$; mux11, the 11-multiplexer;
  - $\mathrm{par3or} = x_1x_2x_3 \vee (x_4 \wedge x_5)$; $\mathrm{poly4} = \mathrm{sign}(x_1x_2 + x_3x_4x_5 + \frac{1}{2}x_6 + x_7x_8x_9x_{10})$; $\mathrm{mixmaj} = \mathrm{MAJ}(x_1x_2,\, x_3 \wedge x_4,\, x_5 \vee x_6)$;
  - the symmetric function $\mathrm{sym9} = [|x_1 + \cdots + x_9| \leq 1]$;
  - h3rf0 and h3rf1, random formulas with 8–9 gates on 7 inputs.

Here $[\cdot]$ is $+1$ if the condition holds and $-1$ otherwise. Every suite puts the relevant inputs first. This matters for tie-breaking (Section 3.4).

**The networks.** They are the companion paper's [5].

- *Architecture:* $f(x) = a^\top \mathrm{relu}(W_2\,\mathrm{relu}(W_1x + b_1) + b_2)$, two hidden layers of width 128, in the $\mu$P parameterization [29].
- *Training:* Adam [17] at learning rate $10^{-3}$ on the square loss, with minibatches of 64 drawn with replacement from a fixed training set of size $n \in \{16, 32, \ldots, 16384\}$. Training runs 20000 steps (40000 at $d = 120$) and is read at the final step.
- *Seeds and test set:* 3 seeds at $d = 30$ (2 on `random` and at other $d$), and one 4096-point test set per target.

This configuration was never tuned. In Part 8, after an audit found it to be a weak representative of gradient-descent-trained networks (Appendix A), two variants were added at the same width, steps and FLOPs:

- `nets_wd`: decoupled weight decay 0.1 [21], on every target of every suite;

- `nets_cewd`: logistic loss plus weight decay 0.1, on `dlheld3` in Part 8 (not on the four earlier suites), and on every Part 9 toy suite (`dlsymdev`, `dlheld4`–`dlheld6`; Section 6). Part 9 also runs `nets_cewd0.3` and `nets_cewd1.0`, the same with weight decay 0.3 and 1.0, on `dlheld6`.

`nets_best` is, per target, the variant with the smallest $n^*$. It is chosen with hindsight on the test set and so favours the networks.

**Sample complexity.** $n^*$ is the training-set size at which the seed-mean test error reaches 0.1, interpolated linearly in $\log n$ on the factor-2 grid. Every learner sees exactly the networks' training sets and test set; an audit checked this row by row (Appendix A).

- *Comparison:* the $\log_2$ ratio $\log_2(n^*_{\text{learner}}/n^*_{\text{ref}})$, so $-1$ means half the data. Suite means are taken over targets both learners learn.
- *Clearly better (worse):* the ratio is below $-0.25$ (above $+0.25$), or only one of the two reaches 10% error on the grid.
- *Data factor:* the reference's $n^*$ divided by the learner's.

**Compute budget.** Compute is counted in FLOPs, with a multiply-add as 2. Everything a learner does in a fit is counted, including model selection and cross-validation; prediction is not counted, for either side.

- *Toy unit:* one network training run, $6P \cdot 64 \cdot T + 10P \cdot T$ FLOPs, where $P$ is the parameter count and $T$ the step count (Adam adds the second term). That is $1.62 \times 10^{11}$ at $d = 30$.
- *No sharing:* the networks are trained in parallel under `vmap`, but no computation is shared between them, so each $(n, \text{seed})$ network costs one unit.
- *Budgets:* DFG (Part 6) and the Part 8 and Part 9 learners are held to at most one unit per fit.

## 2.2 Standard tasks and the MLP references

**Datasets.** Part 7 used 18 classification tasks:

- MNIST [20] and Fashion-MNIST [28], with the standard 60000/10000 split;
- 16 datasets from the Penn Machine Learning Benchmarks (PMLB) [23], with continuous, categorical, binary and mixed features and 2–26 classes.

Five PMLB datasets were for development (optdigits, magic, kr_vs_kp, satimage, spambase). The remaining 13 tasks were held out: the two image sets and adult, letter, pendigits, connect_4, dna, nursery, ring, twonorm, phoneme, waveform_40, mushroom. Part 8 added HELDOUT2, 10 further PMLB datasets fixed before any Part 8 design: texture, churn, wine_quality_white, page_blocks, clean2, led24, ann_thyroid, banana, krkopt, sleep.

**Learning curves.** Each dataset has a fixed test set: the standard 10000 images, or a random 25% (at most 10000 points) for PMLB.

- *Training sets:* nested prefixes of a seed-specific permutation of the remaining pool.
- *Grid:* $n = 100, 300, 1000, 3000, 10000, 30000, 60000$ up to the pool size, plus the full pool.
- *Seeds:* three. On images at $n \geq 30000$ the DFG adaptations (Part 7), and the Part 8 learners and the strong MLP, were run with seed 0 only.

| set | size | status | role |
|---|---|---|---|
| toy `designed` | 21 | development; in-sample | Part 6 development; Part 8 dev score; Part 9 integration rule (i) |
| toy `random` | 20 | read during DFG's development | Part 6 development check; Part 8 supporting |
| toy `dlheld` | 12 | blind for frozen DFG v1 (except its degree-5/6 search) | Part 6 blind test; Part 8 supporting |
| toy `dlheld2` | 7 | blind for DFG v1.1; used once in Part 6 | Part 6 blind test; Part 8 supporting |
| toy `dlheld3` | 10 | fresh in Part 8 | **Part 8 toy headline** |
| 5 PMLB DEV | 5 | development | Part 7, 8 and 9 development |
| Part 7 HELDOUT, tabular | 11 | held out in Part 7; references read by Part 8 builders | Part 7 test; Part 8 supporting |
| MNIST, Fashion-MNIST | 2 | held out in Part 7; references read by Part 8 builders; development in Part 9 | the only image evidence of Parts 7, 8; Part 9 integration rule (ii) |
| HELDOUT2 | 10 | fresh in Part 8, never read by a builder | **Part 8 tabular headline** |
| toy `dlsymdev` | 8 | development, new in Part 9 | Part 9 integration rule (i), with `designed` |
| toy `dlheld4` | 12 | fixed before Part 9 design; definitions seen by the sum-feature builder | Part 9 secondary |
| toy `dlheld5` | 12 | drawn after that builder froze; family inside its class | Part 9 secondary |
| toy `dlheld6` | 12 | family written blind to the design, drawn once | **Part 9 toy headline (Q1)** |
| QuickDraw `qd_dev` | 1 | development, new in Part 9 | Part 9 integration rule (ii), with MNIST, Fashion-MNIST |
| QuickDraw `qd_a`, `qd_b` | 2 | fixed before Part 9 design, never read by a builder | **Part 9 image headline (Q2)** |

**Table 1:** Data sets and their status. "Fresh" sets were fixed in a commit before any design work of the part that uses them as headline evidence. Part 9's sets are described in Section 6.2.

**MLP references.** The Part 7 and 8 references are 256-256 ReLU MLPs with softmax cross-entropy; Part 9's wider `mlp_wide` (512) and `mlp_xl` (1024) and its controls are described in Section 6.7.

- `mlp`, the baseline:
  - standardized continuous inputs, one-hot categoricals;
  - Adam at batch 128, for $\min(10000, \max(2000,\ 50 \text{ epochs}))$ steps;
  - learning rate in $\{10^{-3}, 3 \times 10^{-4}\}$ and the stopping checkpoint chosen on a 10% validation split, then retrained on all $n$.
- Variants added during Part 7:
  - `mlp_plus`: a learning-rate grid with AdamW; tabular data, $n \leq 3000$;
  - `mlp_unit`: raw $[0, 1]$ pixels; images, $n \leq 10000$;
  - `mlp_l2`: an L2 penalty $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ chosen on validation, final checkpoint. It was added after an audit found the baseline under-regularized at small $n$.
- `mlp_reg` (Part 8): `mlp_l2` on tabular data and its $[0, 1]$-pixel version `mlp_l2unit` on images, at every $n$.
- `mlp_strong` (Part 8; added after the fairness audit, settings from the audit's recommendation

and standard practice, smoke-tested on two dev datasets only):

  - quantile-transformed continuous inputs; $[0, 1]$ pixels on MNIST and Fashion-MNIST, quantile-transformed pixels on the Part 9 QuickDraw sets;
  - AdamW [21] at learning rate $10^{-3}$ with cosine decay to 0;
  - decoupled weight decay in $\{0, 0.1, 1\}$ chosen on validation, and dropout 0.1 [26];
  - an ensemble of 3 networks, so 6 training runs per fit, twice the Part 8 learners' budget. Its first network alone is `mlp_strong1`.

- `mlp_best`: per cell, the lowest seed-mean *test* error over the Part 7–8 MLP variants run on that cell. It is a hindsight choice that favours the MLPs. Which variants exist differs by split (Appendix A); on QuickDraw, Part 9's wider variants enter only `mlp_best+` (Section 6.7).

Calibration references, which enter no verdict:

- `linear`: multinomial logistic regression on the MLP's encoding;
- `hgb`: scikit-learn's histogram gradient boosting with default settings [24];
- `lin`, on the toy suites: L2 logistic regression on the raw inputs, with cross-validated $\lambda$.

**Budgets on standard tasks.** One MLP training run at size $n$ costs $6P \cdot 128 \cdot S$ FLOPs, with $S$ the step count. DFG's adaptations (Part 7) were held to one run per fit. The Part 8 and Part 9 learners were held to three, which is the baseline's own cost including its learning-rate selection.

**Verdicts.** A $(\text{dataset}, n)$ cell is *better* (*worse*) if the seed-mean errors differ by more than 10% relative and by more than 2 standard errors. The s.e. comes from the seed spread of both learners and is at least the binomial noise of the test set; otherwise the cell is a *tie*. Errors are floored at 0.002 in $\log_2$ ratios. The *data factor* of a cell is the training size at which the learner reaches the reference's error at $n$, divided by $n$. Part 8 also reports, per split, the number of datasets whose mean $\log_2$ error ratio over $n$ is below 0, with a two-sided exact sign test. This was fixed before any held-out run.

### 2.3 Pre-registration, held-out discipline and audits

In each part the held-out data were fixed, and the learner frozen, before the learner was run on those data.

- **Part 6.**
  - DFG v1 was frozen (commit `359a6b61`) after `dlheld` was committed (`805bff0e`) and before it was run on it.
  - The revised v1.1 (`0220cf04`) was made after that run.
  - `dlheld2` was committed after v1.1, and v1.1 was first run on it after the input-permutation wrapper was added (`5716239b`).

  Section 3.3 lists the exceptions.
- **Part 7.** Three builders developed adaptations of DFG on the 5 DEV datasets only and froze them. A protocol audit confirmed that no builder touched a held-out dataset.
- **Part 8.** A protocol (`study/hybrid/PROTOCOL.md`) was written before any design result was seen. It fixed:

- the fresh sets `dlheld3` and HELDOUT2 (commit `04a215b2`);
- the FLOP budgets;
- three independent designs, built on development data only;
- a rule that designates the primary design from the dev scores before any held-out run.

The frozen files' md5 sums are recorded there.

Independent audit agents reviewed each part. In Part 8 they ran while the primary design's held-out jobs were running; they were instructed not to read held-out results of the Part 8 learners, and each reports that it read none. Appendix A lists their findings and what was done about each, and Appendix D records what the orchestrating session had seen when it decided each change.

# 3 DFG: discrete feature growth on the toy targets (Part 6)

## 3.1 The learner

DFG keeps what Parts 1–5 found the networks to do, and replaces how they search (Table 2). It works with the residual $r = y - \hat{y}$ of a least-squares fit of the labels on the current features, and with its root mean square $s = \mathrm{rms}(r)$. Training repeats rounds until nothing is accepted, or until the fit already has every training label's sign right.

**Candidate classes.** Each class has its own size $N$, and its test uses a union bound over that class.

- *Global:* every character $\chi_S$ with $|S| \le K$ over all $d$ inputs, so $N = \binom{d}{|S|}$. $K = 4$ in the first round and 3 after. Degrees 5 and 6 are also searched in the first round, on a subsample whose size the compute budget sets.
- *Extension:* $S \cup \{j\}$ and $S \cup \{j, l\}$ for current features $S$.
- *Local:* every $S$ inside the set $V$ of inputs used by current features, for $|V| \le 12$. This is a Walsh transform on $V$, up to completing the junta.

The extension and local classes are much smaller than the global one, so features on inputs already in use pass a lower threshold. This is the discrete form of the composition effect of Part 4.

**The test.** A candidate is accepted if

$$\left|\tfrac{1}{n}\textstyle\sum_t r_t\,\chi_S(x_t)\right| \;>\; c\,s\,\sqrt{2\ln(4KN/\delta)/n}, \qquad c = 1,\ \delta = 0.1,\ K = 4. \tag{1}$$

Everything that passes in a round is added. Since $s$ falls as features are fitted, weaker features become detectable later; this is the noise-limited greedy growth of Parts 1–5.

**Block moves.** These are for signal spread over many small coefficients that no single test sees.

- *Linear block:* all $d$ inputs at once, when a $\chi^2_d$ test on the linear coefficients of the residual rejects at level $\delta$. This is the analogue of the networks' kernel background.
- *Spectral block:* the top eigen-directions $u$ of the residual pair matrix $\frac{1}{n}\sum_t r_t x_t x_t^\top$ (diagonal removed) whose eigenvalue clears the noise edge $2s\sqrt{d/n}$. It is used only for $n \ge 4d$, and adds the features $(u \cdot x)^2 - |u|^2$. It was designed for targets such as $\mathrm{MAJ}_5 \oplus \mathrm{MAJ}_5$.

| what the networks were seen to do (Parts 1–5) | DFG's discrete counterpart |
|---|---|
| grow features greedily, in bundles, in order of detectability (plateaus and drops) | rounds of exhaustive search; everything that passes the test this round is added |
| a feature's cost is set by its correlation with the residual against the noise of competing directions | the test (1), with $s = \mathrm{rms}(r)$ and $N$ the size of the candidate class |
| features on a term's own inputs make it cheaper (composition, Part 4) | small candidate classes: extensions $S \cup \{j\}$, $S \cup \{j, l\}$ and local subsets of the inputs in use |
| a kernel background plus a few grown features | a linear block ($\chi^2_d$ test) and a final Hebbian step on the residual |
| dense targets: one direction grown from many weak correlations | a spectral block: eigen-directions of the residual pair matrix above the noise edge |
| on every plateau, the sign of a truncated Fourier expansion | readout: sign of the least-squares fit on the grown features (v1), refined by logistic regression (v1.1) |

**Table 2:** How DFG's components map to what Parts 1–5 found the networks to do [5].

**Readout and small $n$.**

- *Readout:* DFG predicts the sign of the least-squares fit, plus one Hebbian step on the residual over all $d$ inputs (the "kernel background"). Least squares is kept to $p \le n/2$ features, so it never sits at its interpolation peak.
- *v1.1 refinement:* the least-squares weights are refined by 15 Newton steps of L2-regularized logistic regression ($\lambda = 10^{-4}$) when $n \ge 8p$.
- *Forced variant:* with few points the union bound passes nothing; at $n = 16$ even a perfect one-input target fails it. For $n \le 512$, a "forced" variant takes the best-scoring global character anyway, at most twice. It is chosen over the default only if it wins a paired (McNemar) test in cross-validation.

**Budget.** Each exhaustive search runs on the first $m$ training points, with $m$ set so that it stays within a fixed share of one network run:

- degree 4: 15% (25% in v1);
- degree 3, and the extensions: 3% per round each;
- degrees 5 and 6: 3% each.

Searches whose $m$ would fall below 32 are skipped (v1.1), and degrees 5–6 are skipped below $m = 64$. A new round starts only below 75% of the budget. At $d = 30$ the budget binds only for degrees 5–6 ($m \approx 1400$ and 150). At $d = 60$ and 120 it binds for degree 4 too ($m \approx 2200$ and 360).

**What is discrete.** The hypothesis is a set of parities $\chi_S$ plus, on dense targets, a few quadratic forms. They are found by enumeration and admitted by explicit tests. The only continuous parameters are the readout weights; the spectral directions $u$ are the one borderline case (Section 5.9).

### 3.2 Why exhaustive enumeration pays about $\ln N$ in samples

*Remark* 3.1 ([Heuristic]). Let $\rho$ be a character's correlation with the current residual, in units of $s$. The left side of (1) then concentrates around $\rho s$, with noise of order $s/\sqrt{n}$. The threshold is the

union-bound level over $N$ candidates, so the character is admitted, and no null candidate is, once roughly

$$n \gtrsim 2\ln(4KN/\delta)/\rho^2.$$

For a $k$-sparse character found by the global search, $N = \binom{d}{k} \le d^k$, so the sample cost is about $2k \ln d/\rho^2$; this is the Occam rate. The compute cost is about $n\binom{d}{k}$ per search.

The networks pay the other way round. Part 3's heuristic law [5], fitted to them, has $n^* \approx c\,N(k)/\rho^2$ with $N(k) = d\max(1, d/a)^{k-2}$, which is about $d^{k-1}$ for a $k$-parity. The measured exponents are 1.0, 1.9 and 2.4 for $k = 2, 3, 4$, and $k = 4$ is also limited by the networks' step budget. Under the networks' own FLOP budget, exhaustive search is affordable at $d = 30$ up to degree 4 on all the data, and up to degree 6 on a subsample.

So, heuristically, gradient descent spends data where exhaustive search spends FLOPs. This reading is consistent with the results below but not tested by them. Search is also not the only difference between the two learners. DFG's dictionary, characters on the uniform cube, is the class the targets were built from, and these experiments do not separate the two effects. DFG is in effect a greedy, budgeted form of the classical low-degree algorithm, designed for this function class. The networks are generic and use one untuned configuration.

### 3.3 Versions and what is blind

- **v1** (commit `359a6b61`) is the version run blind on `dlheld`.
  - *Exception 1.* Its degree-5/6 search was added knowing that $\chi_5$ (par5) was in the suite, and par5's result depends on it entirely. The blind row therefore uses v1 without that search ($K_{\text{hi}} = 4$).
  - *Exception 2.* The spectral block was committed together with the suite, and its docstring works through $\text{MAJ}_5 \oplus \text{MAJ}_5$, which is the suite's majxor.
- **v1.1** (`0220cf04`) was made after that run. It fixes two problems.
  - *The readout.* On the dense halfspace thr30, v1's least-squares readout plateaued at 5–7% error, where the networks reach 1%. The logistic refinement takes thr30's error from 0.054 to 0.010 (post hoc).
  - *A budget bug found by the compute audit.* v1's small-$n$ cross-validation gave every fold the whole budget. At $d = 120$ that cost up to 2.7 runs of a 20000-step network per fit. v1.1 splits the budget across folds and adds a 0.75 hard cap.

  v1.1 also skips searches with $m < 32$. That rule is why its $n^*$ for par3 and par4 at $d = 30$ (28 and 52) are higher than v1's (16 and 21).
- `dlheld2` was committed after v1.1 (`70c604eb`), and DFG's code was not changed afterwards except for its name and an input-permutation wrapper. So that suite is blind for v1.1. It was written by DFG's designer, mostly from families already in the development suites, so it is blind to tuning, not to the choice of targets.
- **The development suites are in-sample.**
  - `designed`: DFG's settings, notably the small-$n$ rules and the $n \ge 4d$ gate on the spectral block, were chosen by looking at its test error there.
  - `random`: it was run twice with intermediate versions and read.

| suite (targets), DFG version | clearly better / worse | median factor | geo.-mean factor | cells worse by $> 0.02$ ($> 2$ s.e.) | max FLOPs per run |
|---|---|---|---|---|---|
| `dlheld2` (7), v1.1, permuted — **blind** | 7 / 0 | 3.1× | 3.4× | 6 (3) of 77 | 0.26 |
| `dlheld` (12), v1 without deg. 5–6 — **blind** | 11 / 1 | 2.7× | 3.7× | 13 (12) of 132 | 0.17 |
| `dlheld` (12), v1 (deg. 5–6 added knowing par5) | 12 / 0 | 2.8× | 5.2× | 11 (10) of 132 | 0.23 |
| `dlheld` (12), v1.1 — post hoc | 12 / 0 | 3.0× | 5.3× | 6 (5) of 132 | 0.43 |
| `random` (20), v1 — development | 20 / 0 | 6.0× | 6.8× | 9 (5) of 220 | 0.14 |
| `designed` (21), v1 — in-sample | 20 / 0 | 5.4× | 7.0× | 7 (2) of 231 | 0.19 |
| `designed` at $d = 15$ (7), v1 | 7 / 0 | 6.0× | 6.3× | 1 (0) of 77 | 0.08 |
| `designed` at $d = 60$ (7), v1.1 | 7 / 0 | 31.7× | 36.3× | 0 (0) of 77 | 0.46 |
| `designed` at $d = 120$ (4), v1.1 | 4 / 0 | 45.5× | 30.7× | 0 (0) of 29 | 0.18 |

**Table 3:** DFG against the study's networks [Numerical]. The median and geometric-mean data factors are the networks' $n^*$ over DFG's, over targets both learn. "Cells" are (target, $n$) pairs on the 11-point grid where DFG's seed-mean error exceeds the networks' by more than 0.02 (in brackets: also by more than 2 s.e. of the seed spread). The last column is DFG's largest per-fit compute as a fraction of one network run. Source: `study/discrete_learner/RESULTS.md`.

### 3.4 Results against the networks

Table 3 and Figure 1 summarize the comparison [Numerical].

**Blind tests.**

- ***`dlheld2`**, the final version.* DFG v1.1 has the smaller $n^*$ on all 7 targets: median factor 3.1, geometric mean 3.4, with at most 0.26 of a network run per fit. The smallest gains are on the symmetric sym8 (1.3×), the random formula h2rf1 (1.5×) and the 2-DNF (1.9×). sym8's is marginal across seeds.
- ***`dlheld`**, frozen v1 without its degree-5/6 search.* DFG has the smaller $n^*$ on 11 of 12 targets, by a median factor of 2.7. Without that search it has no route to a degree-5 character, so it never learns par5, while the networks learn it at $n \approx 7200$.

**Development suites and input dimension.**

- *Development suites.* On the 41 targets of `designed` and `random`, $n^*$ is clearly smaller on 40. The 41st, and3, is a tie (101 against 94).
- *Dense targets* (majorities, halfspaces, intersections of halfspaces, $\mathrm{MAJ}_5 \oplus \mathrm{MAJ}_5$, sym8): the factor is 1.2–4.4. Heuristically, and with no linear baseline run in Part 6, this is the difference between a well-fitted linear readout and the networks' learned features. Part 8's `lin` reference is consistent with this reading (Section 5.3).
- *Sparse targets* (parities, staircases, majorities of parities, multiplexers, random formulas): the factor is 1.5–250, growing with $d$.
- *Parities across $d$.* For par3 the networks need 843, 3410 and about 14000 samples at $d = 30$, 60 and 120, and DFG needs 28, 28 and 56. For par4 the networks need 2728 and 14255 at $d = 30$ and 60, and DFG needs 52 and 56. These DFG values are from v1.1 with the inputs permuted per fit; the unpermuted runs give the same values.
- *Caveats on the $d$-scaling rows.* Their targets are in-sample and mostly sparse, so their median rises partly because the target mix changes. For maj9 the factor stays at 1.5–2.7× across $d$.

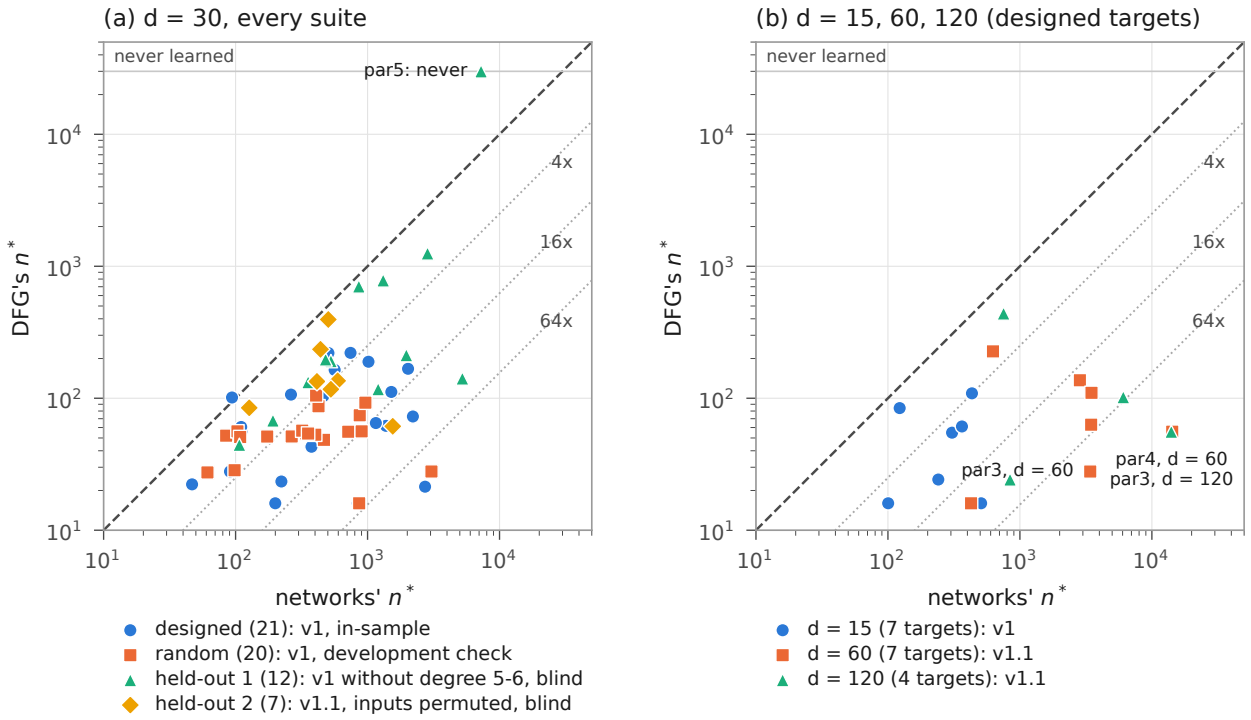



**Figure 1:** DFG's $n^*$ against the networks' $n^*$, one point per target, on log axes. Points below the dashed diagonal are targets where DFG needs less data; the dotted lines mark 4, 16 and 64 times less. (a) $d = 30$: the two blind runs (`dlheld` with frozen v1 without its degree-5/6 search, and `dlheld2` with v1.1 on permuted inputs) and the development suites (v1, natural input order). par5, which DFG never learns without the degree-5/6 search, is drawn at the top. (b) The designed targets run at $d = 15$ (v1), 60 and 120 (v1.1). The par4 point at $d = 60$ and the par3 point at $d = 120$ coincide.

**Where DFG is worse.** Per (target, $n$) cell, DFG's error exceeds the networks' by more than 0.02 in 0–10% of cells, depending on the suite. These cells are of three kinds.

- $n = 16$–*64*. Most of the cells are here, where both learners are near the constant predictor's error and DFG is sometimes above it. For example, sym8 at $n = 32$ has 0.40 against the networks' 0.34 and the constant's 0.28.
- *thr30 at* $n \geq 2048$*, in v1:* the least-squares readout plateau.
- $MAJ_5 \oplus MAJ_5$ *(majxor) at* $n = 64$–*256*, in both versions.
  - The networks are partly there (0.36 at $n = 256$), while DFG is at chance until its spectral block fires near $n = 512$.
  - DFG then leads: 0.02–0.03 against 0.16 at $n = 1024$.

So "at least as good" holds for $n^*$ on every target of the blind tests except par5 without the degree-5/6 search. It does not hold for every cell.

**Compute.** DFG uses less compute than a network run everywhere: at most 0.46 of a run per fit, and under 0.08 at $n \leq 512$ at $d = 30$. In recorded CPU time it is comparable at $d = 30$ (0.25–1.7× a network at $n = 16384$) and 2–4× faster at $d = 120$. This is despite DFG running in float64 numpy against the networks' float32 XLA (Appendix C). The claim is about FLOPs.

| variant (one ingredient removed) | maj9 | thr10 | par4 | stair5 | mux | rmaj9 | sparse-poly | mean $\log_2$ vs full |
|---|---|---|---|---|---|---|---|---|
| full DFG v1.1 | 107 | 146 | 52 | 111 | 64 | 167 | 93 | +0.00 |
| global search to degree 3 only | 107 | 152 | never | 113 | 158 | 167 | 93 | +1.34 |
| raw-coefficient readout | 309 | 295 | 52 | 112 | 63 | 210 | 99 | +0.42 |
| thresholds scaled by the label, not the residual | 107 | 148 | 56 | 165 | 111 | 188 | 97 | +0.25 |
| no composition classes | 107 | 142 | 52 | 196 | 107 | 167 | 108 | +0.25 |
| no background, no small-$n$ variant | 107 | 146 | 52 | 112 | 110 | 167 | 107 | +0.14 |
| no blocks | 168 | 144 | 52 | 111 | 61 | 178 | 80 | +0.07 |
| least-squares readout (v1's) | 107 | 170 | 52 | 112 | 65 | 167 | 101 | +0.05 |
| networks | 263 | 563 | 2728 | 1514 | 1156 | 2027 | 2205 | |

**Table 4:** DFG ablations: $n^*$ on 7 designed targets, $d = 30$, inputs permuted, 3 seeds [Numerical]. A target never learned counts +8 in the mean. Changes below about 1.2× may be seed noise. Source: `study/discrete_learner/RESULTS.md`.

**Tie-break bias and robustness.**

- *Tie-breaks.* Every suite lists the relevant inputs first, and DFG's enumeration breaks exact ties toward low indices. At $n = 16$–$32$ the forced guess therefore wins ties toward the true target. The reproduction audit measured this on frozen v1: with permuted columns, par4 at $n = 16$ goes from 0.17 to 0.40 error and par3 at $n = 16$ from 0.00 to 0.13, and the designed suite's mean $\log_2$ ratio moves by about +0.03.
- *Permuted runs.* `dlheld2` and the ablations were run with the inputs permuted at random per fit, which removes the bias. The development rows were not.
- *Fresh seeds.* Fresh training seeds at the grid points bracketing DFG's $n^*$ reproduce the recorded errors for stair5, rmaj9, parmaj and par3. mux is the exception: on 9 fresh seeds its $n^*$ is about 87 rather than 65, so its factor is about 13, not 18.
- *Definition of $n^*$.* An alternative definition, the smallest grid $n$ from which the error stays at most 0.1, changes the mean $\log_2$ ratios by at most 0.10–0.15.

### 3.5 Ablations

Table 4 removes one ingredient at a time [Numerical].

- **Only two ablations lose to the networks**: par4 without the degree-4 search (never learned), and $\mathrm{MAJ}_9$ with the raw-coefficient readout (309 against 263). Every other variant needs less data than the networks on all 7 targets.
  - By elimination, the gap is plausibly carried by what every variant keeps: exhaustive search over the Fourier dictionary with union-bound tests.
  - That was not ablated directly, so the attribution is [Heuristic].
- **The global search depth decides which leaps are possible.**
  - Without degree 4, par4 has no lower-order signal, so neither extensions nor local search can reach it.
  - mux gets 2.5× worse, because its term $x_1x_2x_3x_4$ must then be built by extension.
  - This is the discrete analogue of leap complexity. The difference from the networks is that a leap costs DFG $\ln \binom{d}{k}$ in samples and $\binom{d}{k}$ in compute, not about $d^{k-1}$ in samples.

- **Each ingredient taken from the networks is worth 1.1–3× on one class of targets.**
  - *The readout fitted to the residual*, against raw Fourier coefficients: 2.9× on $\mathrm{MAJ}_9$ and 1.3× on rmaj9. On thr10 it is worth 1.7× (295 against 170), and the logistic refinement adds 1.2× (170 to 146).
  - *Residual-scaled thresholds*: 1.5× on stair5 and 1.7× on mux. These are staged targets, where later terms become detectable only once earlier ones are fitted.
  - *Composition classes*: 1.8× on stair5, 1.7× on mux and 1.2× on sparsepoly. These are the targets whose high-degree terms sit on the inputs of their low-degree terms. The direction matches the networks' composition effect (Part 4 measured 1.2–1.6× for them).
  - *The linear block*: 1.6× on $\mathrm{MAJ}_9$, whose nine inputs would otherwise each need to pass an individual test.
- **The blocks cost a little on sparse targets**: sparsepoly goes from 93 to 80 without them.

### 3.6 Limits

- **A narrow inductive bias.** The dictionary is characters on the Boolean cube plus a few quadratic forms, and the tests rely on characters being orthogonal under the uniform distribution. Other input distributions, real-valued inputs and images need a different dictionary (Section 4).
- **Compute grows as $d^K$.** Under a fixed budget DFG subsamples, so a high-degree feature must have a large coefficient to be found. At $d = 30$ the degree-6 search sees about 150 points, and degrees 5–6 are skipped at $d \geq 60$.
- **One network configuration.** The reference is the study's fixed configuration, never tuned, while DFG's settings were tuned on the designed suite. Earlier parts found that the networks' $n^*$ moves by up to about 2× with width or optimizer. That is below the blind median factor of 3.1, but comparable to the smallest blind wins (1.3–1.9×). Part 8 measured the effect of weight decay and logistic loss directly (Section 5.3).

## 4 DFG on standard tasks (Part 7)

### 4.1 Three adaptations

DFG as written assumes ±1 inputs, the uniform distribution and binary labels. Three builders each wrote one adaptation to real-valued, multi-class data that keeps its core:

- greedy rounds of exhaustive candidate tests against the least-squares residual, with union-bound thresholds;
- composition classes and block moves;
- a least-squares readout;
- a FLOP budget of one MLP training run per fit.

All three use a multi-class version of DFG's test: a $\chi^2$ statistic of the whitened residual summed over classes, with a union bound over each candidate class. Each builder developed on the 5 DEV datasets only and then froze the design (Table 5).

| design | dictionary of discrete features | $c$ | readout | dev score |
|---|---|---|---|---|
| `dfg_bits` (most faithful) | each input becomes $\pm 1$ bits (one-hot categories; 6–12 thermometer bits at training quantiles per continuous input); characters (products of bits from different inputs) of degree 1–2 every round, 3–4 when the budget allows; extensions; local Walsh transform; linear and spectral blocks | 1 | ridge least squares, then softmax refinement | +0.15 |
| `dfg_gates` | literals $[x_j > t]$ at 16 quantiles, $[x_j = v]$ and negations; gates = ANDs of up to 3 literals; pairs exhaustively on a budgeted subsample; extensions $g \wedge l$ of current gates; linear block | 0.5 | least squares or softmax, chosen on a 10% split | +0.07 |
| `dfg_dirs` | gates $[u \cdot z > t]$, hinges and squares $(u \cdot z)^2$ on a finite set of axes: the coordinates, plus directions derived each round (per-class linear discriminants, spectral eigen-directions of the residual, class-mean differences inside an accepted gate's region, prototype points); products with gates; linear block | 0.8 | softmax, $\lambda$ by cross-validation | +0.05 |

**Table 5:** The three adaptations of DFG to standard tasks. The dev score is the mean $\log_2$(error/MLP error) over the dev cells (lower is better). Details: `DESIGN.md` in each design's folder under `study/standard_tasks/`.

The plan was to designate the design with the best dev score as the primary proposal. That did not happen cleanly. `dfg_dirs`, which has the best dev score, froze its second version only after the other two designs' held-out runs had started, and after a second development cycle on its own full dev results. All three are therefore reported on an equal footing.

Two builders loosened DFG's constant $c$ on dev data:

- `dfg_gates` used $c = 0.5$, because with $c = 1$ growth stopped after 2–4 rounds on magic and spambase;
- `dfg_dirs` used $c = 0.8$, which was slightly better on 6 of 10 dev cells.

`dfg_bits` kept $c = 1$. Its search also stopped after 1–3 rounds, but lowering $c$ to 0.6–0.8 was not consistently better.

## 4.2 Results

Table 6 and Figure 2 give the results [Numerical]. The short answer is: not very well, beyond the toy problems.

**Tabular data against the baseline MLP.** The error is on average 3–7% higher (mean $\log_2$ ratio +0.04 to +0.10). Most cells are ties: DFG is better in 6–9 of 59 cells and worse in 10–15.

**Tabular data against the L2-regularized MLP** ($n \leq 3000$). The error is 12–17% higher (+0.16 to +0.23). The baseline MLP itself trails the regularized one by about the same amount (+0.21). So most of DFG's small-$n$ "wins" over the baseline were the baseline's lack of regularization. That was found by an independent audit, not anticipated. Of the 23 (learner, cell) wins over the baseline, 5 survive, all at $n \leq 300$:

- `dfg_bits` on dna at $n = 300$: 0.060, against 0.119 for the regularized MLP and at least 0.10 for every MLP tried. It matches gradient boosting (0.057). Here sparse categorical motifs are exactly the bit-products DFG enumerates. Only this win also clearly beats every other reference.

| reference | learner | cells | better / tie / worse | mean $\log_2$ ratio | median data factor | max FLOPs per MLP run |
|---|---|---|---|---|---|---|
| *11 held-out tabular datasets, all $n$* | | | | | | |
| `mlp` | `dfg_bits` | 59 | 6 / 38 / 15 | +0.09 | 1.00 | 0.88 |
| | `dfg_gates` | 59 | 9 / 40 / 10 | +0.04 | 0.93 | 1.07 (5 cells over) |
| | `dfg_dirs` | 59 | 8 / 39 / 12 | +0.10 | 1.00 | 0.85 |
| | `linear` | 59 | 3 / 32 / 24 | +0.60 | 1.00 | – |
| | `hgb` | 59 | 12 / 39 / 8 | −0.07 | 0.72 | – |
| *11 held-out tabular datasets, $n \leq 3000$* | | | | | | |
| `mlp_l2` | `dfg_bits` | 44 | 4 / 29 / 11 | +0.23 | 1.02 | |
| | `dfg_gates` | 44 | 1 / 32 / 11 | +0.16 | 1.00 | |
| | `dfg_dirs` | 44 | 0 / 33 / 11 | +0.23 | 1.41 | |
| | `mlp` | 44 | 1 / 36 / 7 | +0.21 | 1.19 | |
| | `hgb` | 44 | 3 / 30 / 11 | +0.11 | 1.12 | |
| *MNIST and Fashion-MNIST, all $n$* | | | | | | |
| `mlp` | `dfg_bits` | 14 | 0 / 7 / 7 | +0.34 | 1.37 | 3.45 (4 cells over) |
| | `dfg_gates` | 14 | 0 / 7 / 7 | +0.41 | 1.79 | 1.07 (2 cells over) |
| | `dfg_dirs` | 14 | 0 / 7 / 7 | +0.26 | 1.27 | 0.99 |
| | `linear` | 14 | 0 / 4 / 10 | +0.50 | 1.16 | – |
| | `hgb` (MNIST only) | 7 | 3 / 4 / 0 | −0.12 | 0.72 | – |

**Table 6:** Part 7: the three DFG adaptations against MLP references on held-out data [Numerical]. Mean $\log_2$ ratio: of the learner's test error to the reference's, over cells. Median data factor: the fraction of the reference's data at which the learner reaches the reference's error. In Part 7, `hgb` was run on MNIST but not on Fashion-MNIST. Source: `study/standard_tasks/RESULTS.md`.

- `dfg_bits` on ring at $n = 100$ and 300: 0.128 against 0.184, and 0.078 against 0.097.
- `dfg_bits` and `dfg_gates` on waveform at $n = 100$: 0.197 and 0.198, against 0.231.

**Tabular data at full $n$.** DFG is clearly behind on tasks with many classes or interacting features:

- letter, 2.1–3.5× the MLP's error;
- pendigits, 1.6–2.0×;
- connect_4, 1.25–1.4×.

Default gradient boosting has 8–12% lower error than the three adaptations on held-out tabular data on average. It is lower on most datasets, but higher on twonorm and waveform.

**Images.** DFG is behind from $n \approx 3000$ on MNIST and from $n \approx 10000$ on Fashion-MNIST, and roughly tied at small $n$. At $n = 60000$:

- MNIST: 5.5–7.2% error against the MLP's 2.5% (2.2–2.9×);
- Fashion-MNIST: 14–16% against 11% (1.3–1.4×).

All three adaptations are closer to logistic regression (7.8% and 16.5%) than to the MLP. `dfg_dirs`, whose dictionary is thresholded linear directions, the same kind of unit as an MLP's, is the best of the three on MNIST: 1.5× the MLP's error at $n = 30000$ and 2.2× at 60000. The baseline MLP is somewhat under-tuned on images (2.5% on MNIST at $n = 60000$; Part 8's references reach 1.43–1.75%, Section 5.5). So the gap is probably a lower bound.




**Figure 2:** Part 7: test error of each DFG adaptation divided by the baseline MLP's, per held-out dataset, at $n = 3000$ and at the full training pool (right column: its size). Errors are floored at 0.002, as in the study's $\log_2$ ratios. Datasets are sorted by the mean ratio at full $n$. Against the L2-regularized MLP, available for tabular data at $n \leq 3000$, the ratios in (a) are larger (Table 6). On images at full $n$ the DFG adaptations are seed 0 only.

**Compute.**

- `dfg_dirs` stayed within one MLP run in every cell by its own count (maximum 0.99).
- `dfg_gates` exceeded it in 7 cells (maximum 1.07×), because its final readout refit is not reserved in its budget check.
- `dfg_bits` exceeded it on images, up to 3.45× on Fashion-MNIST at $n \geq 10000$. It is behind the MLP in all those cells, and its lead over the other adaptations on Fashion-MNIST at $n \geq 10000$ comes with 2–3.5× their compute.

An audit checked the FLOP counters against an instrumented count (Appendix C).

### 4.3 Why it fails

DFG beat the networks on sparse Boolean targets because exhaustive search over a dictionary that contains the target's features pays only about $\ln N$ in samples (Remark 3.1), and the dictionary fit within the networks' FLOP budget. On standard tasks this premise fails in two ways [Heuristic].

**The useful features are continuous.** On images the useful hidden units are directions spread over hundreds of correlated pixels, each with a small weight. That is a continuous family.

- A product of a few bits, or a short AND of thresholds, is a poor approximation of such a direction.
- DFG's directional moves (spectral directions, class discriminants, local mean differences) generate only a few candidate directions per round, from statistics of the current residual.

- An MLP's gradient step updates hundreds of directions at once, each pulled toward the residual. In effect it is a continuous, parallel version of DFG's spectral move, iterated thousands of times.

In a continuous family, local search is the efficient search, and enumeration is not.

**The tests can stall growth on dense signal.** Union-bound tests are built to stop false features on sparse targets. On dense real-world signal, many weak correlated effects, they stopped growth early on dev data; this is why `dfg_gates` lowered $c$. On held-out data, though, the largest losses (letter, pendigits, connect_4, and images at large $n$) ran at or near the FLOP budget. There the dictionary and the budget, not the tests, are what bind. The MLP has no such gate. It moves all its units at once and controls overfitting with a continuous penalty, not by rejecting features. Here, L2 weight decay without early stopping lowered its error by 16% at $n \leq 3000$.

**What this does not show.**

- *That no discrete learner can match MLPs.* Gradient boosting is also a discrete, greedily grown learner, and it beats the baseline MLP on several tabular datasets (adult, dna, ring). What does less well is DFG's specific recipe: exhaustive enumeration with union-bound tests over parity, gate or direction dictionaries. Section 5 follows this up.
- *That these adaptations are the best possible ones.* Each builder had about 70 minutes and only the dev data. `dfg_bits` and `dfg_gates` finished in about 20 minutes and `dfg_dirs` in about 50.
- *Anything about convolutional dictionaries.* None was tried on images.

# 5 A discrete portfolio on both kinds of task (Part 8)

## 5.1 Protocol and designation

The question was whether one discrete method beats gradient-descent-trained networks both on the toy suites ($n^*$ against the networks) and on the standard tasks (test error against MLPs). A protocol, written before any design result was seen, fixed the following (Section 2.3):

- *Development data:* the toy `designed` suite and the 5 DEV datasets only.
- *Fresh held-out sets:* `dlheld3` and HELDOUT2.
- *Budgets:* one network run per toy fit, and three MLP runs per real fit.
- *A designation rule.* On dev, a design "beats GD" if its mean $\log_2$ ratio is below 0 on both sides and it has more better than worse targets or cells. The toy score is the mean $\log_2 n^*$ ratio against the networks. The real score is the mean $\log_2(\text{error}/\min(\text{error of } \texttt{mlp}, \text{error of } \texttt{mlp_l2}))$, with errors floored at 0.002. Among the designs that beat GD on dev, the primary is the one with the lowest sum of the two means.

Three builders worked independently, on dev data only, from different design angles: stagewise, unified boosting, and portfolio with stacking. Each design was frozen and run once on the full dev evaluation, and the rule was applied before any held-out run (Table 7). It picked `hyb_portfolio`, which is also best on each side separately. These dev scores are in-sample: the builders tuned on the same dev test sets.

| design | toy mean $\log_2 n^*$ ratio | toy better / worse (of 21) | real mean $\log_2$ ratio | real better / worse (of 24) | sum |
|---|---|---|---|---|---|
| `hyb_stage` | −2.79 | 21 / 0 | −0.122 | 1 / 0 | −2.91 |
| `hyb_union` | −2.52 | 21 / 0 | −0.127 | 0 / 0 | −2.65 |
| `hyb_portfolio` | −2.98 | 21 / 0 | −0.248 | 1 / 0 | **−3.23** |

**Table 7:** Dev scores of the three frozen Part 8 designs, recomputed from the result files before any held-out run. `hyb_union` has no better real cell, so it does not "beat GD on dev". Source: `study/hybrid/PROTOCOL.md`.

| member | what it is | when it runs |
|---|---|---|
| `dfg`, `dfg_forced` | DFG v1.1 unchanged (Section 3); `dfg_forced` is its small-$n$ variant ($n \leq 512$) | only if there are 2 classes and every column is binary or categorical ($\leq 256$ bits) |
| `gbt` | LightGBM [15]: 100 rounds, learning rate 0.1, 31 leaves | always |
| `et` | ExtraTrees [10]: 100 trees; out-of-bag probabilities serve as out-of-fold predictions | always |
| `knn_<metric>_<k>` | $k$-nearest-neighbour vote fractions, $k \in \{1, 3, 9, 27\}$, under the MLP's encoding and under one common scale; leave-one-out votes serve as out-of-fold predictions | always (the common-scale metric when column scales differ) |
| `lin_<C>` | multinomial L2 logistic regression, $C \in \{0.05, 1\}$ (only $C = 1$ above $n = 10000$) | always |

**Table 8:** Members of `hyb_portfolio`. On the 22 held-out real datasets other than mushroom, the design is by construction a stack of LightGBM, ExtraTrees, up to 8 kNN members and 1–2 logistic regressions.

### 5.2 The primary design

`hyb_portfolio` is one `fit()` that dispatches by input type. Each member outputs class probabilities and has no continuous feature parameters (Table 8).

**Stacking** [27]. Let $L_m$ be member $m$'s out-of-fold log-probabilities on a stacking set. The score is $s_c(x) = b_c + \sum_m w_m L_m(x)_c$ with $w \geq 0$, fitted by L2-penalized multinomial log-loss, which is convex.

- *Stacking set:* 10-fold cross-validation for $n \leq 500$, 5-fold for $n \leq 3000$, and above that one stratified holdout of $\min(n/5, 6000)$ points.
- *Refits:* DFG, LightGBM and logistic regression are refitted on all $n$.

**Budget split.** DFG's out-of-fold fits share 35% of the budget, and its final fit(s) get at most 50%. Every other member is skipped if its projected cost exceeds 90–95% of the budget; this never triggered on dev.

**Large inputs.** When $n \cdot d > 10^7$ (MNIST and Fashion-MNIST at $n \geq 30000$), the design switches to:

- float32 encodings;
- LightGBM with row subsampling 0.5 and column subsampling 0.25;

| target | portfolio | DFG alone | without DFG | lin | hgb | nets | nets_wd | nets_cewd | nets_best | $\log_2$ ratio to nets_best |
|---|---|---|---|---|---|---|---|---|---|---|
| cnf3 | 146 | 213 | 130 | never | 164 | 489 | 371 | 361 | 361 | −1.31 |
| dlist8 | 24 | 31 | 26 | 91 | 61 | 104 | 90 | 108 | 90 | −1.91 |
| h3rf0 | 46 | 51 | 48 | 112 | 60 | 109 | 96 | 113 | 96 | −1.06 |
| h3rf1 | 60 | 56 | 80 | 101 | 63 | 123 | 101 | 121 | 101 | −0.75 |
| mixmaj | 96 | 109 | 341 | never | 368 | 507 | 361 | 326 | 326 | −1.76 |
| mux11 | 523 | 826 | 521 | never | 573 | 1527 | 1245 | 1699 | 1245 | −1.25 |
| par3or | 62 | 61 | 7531 | never | 2009 | 1050 | 725 | 842 | 725 | −3.54 |
| poly4 | 177 | 177 | never | never | never | 10199 | 6609 | 2745 | 2745 | −3.95 |
| sym9 | 1229 | 1220 | 7113 | never | 7845 | 787 | 563 | 350 | 350 | +1.81 |
| thrint | 149 | 167 | 142 | 137 | 243 | 546 | 267 | 190 | 190 | −0.36 |

**Table 9:** The fresh toy suite `dlheld3`: $n^*$ per target, inputs permuted per fit [Numerical]. Definitions are in Section 2.1. "portfolio" is `hyb_portfolio`; "DFG alone" and "without DFG" are the post-hoc ablations of Section 5.6. The last column is computed from unrounded values. Generated by `make_tables.py` from `study/hybrid/results/`; the values agree with `study/hybrid/RESULTS.md`.

| suite | targets | status | vs nets | vs nets_wd | vs nets_best | lin vs nets_best |
|---|---|---|---|---|---|---|
| designed | 21 | development | −2.97 (21/0) | −2.32 (21/0) | −2.32 (21/0) | −0.56 (5/14) |
| random | 20 | DFG development data | −2.89 (20/0) | −2.24 (20/0) | −2.24 (20/0) | −0.05 (2/16) |
| dlheld | 12 | post hoc for DFG | −2.24 (12/0) | −1.75 (10/0) | −1.75 (10/0) | −0.36 (2/9) |
| dlheld2 | 7 | blind for DFG v1.1, used once | −2.17 (7/0) | −1.54 (6/0) | −1.54 (6/0) | −0.76 (2/4) |
| dlheld3 | 10 | **fresh** | −2.12 (9/1) | −1.67 (9/1) | −1.41 (9/1) | −0.06 (1/6) |

**Table 10:** `hyb_portfolio` (inputs permuted) on all five toy suites: mean $\log_2 n^*$ ratio against each network reference, with the number of targets clearly better / clearly worse in brackets [Numerical]. The last column shows logistic regression alone against `nets_best`; its "worse" counts include every target it never learns. Outside `dlheld3`, `nets_best` equals `nets_wd`.

- no LightGBM refit.

These settings came from synthetic timing runs and had never run on real data before the held-out evaluation.

### 5.3 Toy results

**The fresh suite** (Table 9). `hyb_portfolio` needs clearly less data than `nets_best` on 9 of 10 targets [Numerical].

- *Against `nets_best`:* mean $\log_2$ ratio −1.41, about 2.7× less data.
- *Against single configurations:* −1.55 against `nets_cewd`, −1.67 against `nets_wd` and −2.12 against `nets`.
- *The one loss:* sym9.
- *Input order and compute.* The natural-order run is nearly identical (−1.42 against `nets_best`), and the largest fit uses 0.66 of one network run.
- *Without the linear-model win.* Excluding the target on which logistic regression alone already wins (thrint), the portfolio is clearly better on 8 and clearly worse on 1 of 9, mean −1.52.

The margins differ by kind of target.

- **Targets with parity components**, which trees and linear models do not see.

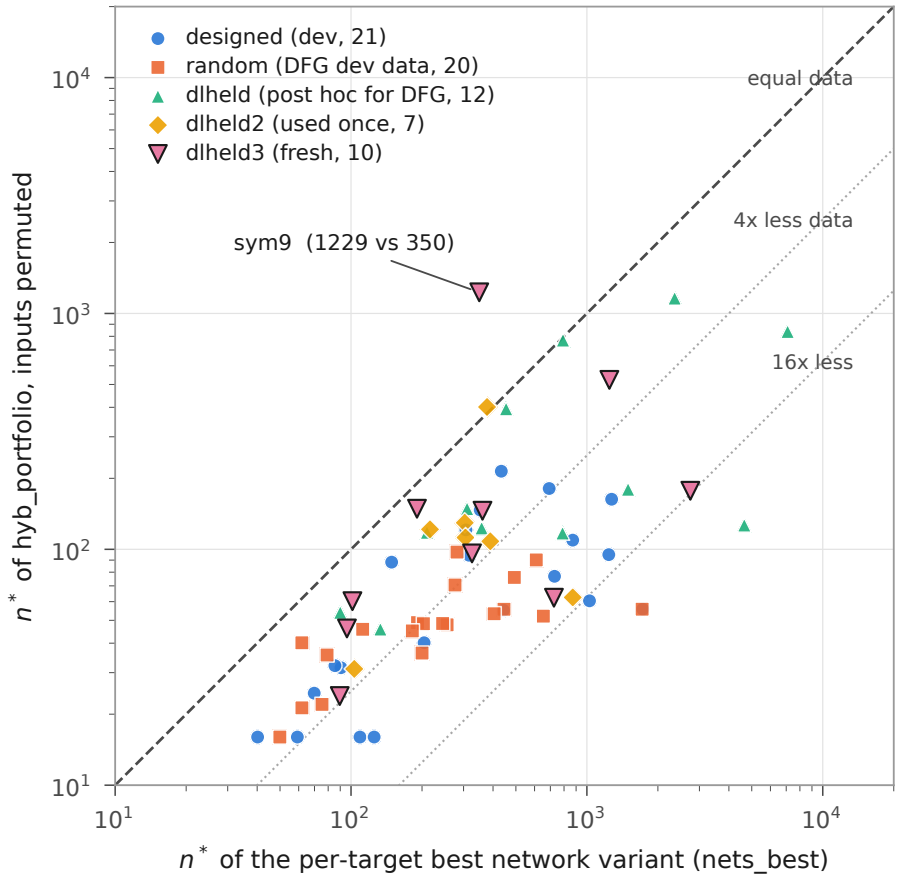



**Figure 3:** `hyb_portfolio`'s $n^*$ (inputs permuted per fit) against the per-target best network variant, for all 70 targets of the five toy suites. Each suite has its own marker shape and colour, and the fresh suite `dlheld3` is outlined. The dashed line is equal data; the dotted lines mark 4 and 16 times less. The one target above the diagonal is sym9.

  - par3or: $-3.54$. `hgb` needs 2009 samples, and `lin` never reaches 10% error.
  - poly4: $-3.95$. Neither `hgb` nor `lin` ever learns it.
  - mixmaj: $-1.76$.

  These margins survive every network variant, and they are DFG's (Section 5.6).

- **mux11 and cnf3** ($-1.25$, $-1.31$). The trees can carry these.
- **Threshold, decision-list and random-formula targets** (thrint, dlist8, h3rf0, h3rf1). The margins are constant factors, $-0.36$ to $-1.91$ in $\log_2$. thrint's shrinks most with the regularized networks, from $-1.88$ against `nets` to $-0.36$ against `nets_cewd`.

**sym9, the one loss.** The portfolio's seed-mean test error on sym9 is 0.39 at $n = 256$, 0.20 at 512, 0.13 at 1024 and 0.014 at 2048, so $n^* = 1229$. The logistic-loss, weight-decayed networks reach 10% at about 350. DFG alone gets 1220 and the portfolio without DFG 7113, so the loss is DFG's. The secondary design `hyb_stage` (591) comes closer. A reading, not tested in Part 8 [Heuristic]:

- sym9 depends on the inputs only through $x_1 + \cdots + x_9$, so a network can represent it with units on the single direction $(1, \ldots, 1)$.

- Its Fourier weight is spread over many characters of degrees 2, 4, 6 and 8, with equal coefficients within each degree. A character search pays for each of them separately.

Part 9 tests this reading (Section 6).

**The other suites** (Table 10, Figure 3). These are development data, or not blind for DFG. On their 60 targets the portfolio is clearly better than every network reference on 57 and tied on 3:

- inter2, 394 against 454, and majxor, 771 against 792, on `dlheld`;
- sym8, 401 against 377, on `dlheld2`.

sym8 is the same kind of target as sym9, and the only target outside `dlheld3` on which the portfolio is above a network reference, inside the tie band.

**Stronger networks.** The fairness audit's probe found that weight decay 0.1, or logistic loss plus weight decay, cuts the networks' $n^*$ by about 2× at the same compute. In the full runs the largest cut is 3.7×, on poly4 (10199 to 2745).

- *Margin shrink.* Against `nets_best` the portfolio's suite-mean margins shrink, relative to `nets`, by 0.65, 0.66, 0.49, 0.63 and 0.72 in $\log_2$ on the five suites.
- *Which targets.* The shrink is spread over every kind of target. The largest per-target shrinks are on `dlheld3`: poly4 by 1.9, thrint by 1.5 and sym9 by 1.2.
- *Signs.* The per-target sign is the same against every reference: negative on 9 targets, positive on sym9. sym9 is already +0.64 against the unregularized `nets` (787).
- *What `nets_best` is.* On `dlheld3` it is a hindsight minimum that no single configuration achieves: `nets_cewd` is best on 5 targets and `nets_wd` on 5, and `nets_cewd` is worse than plain `nets` on mux11, dlist8 and h3rf0.

**Input permutation.** The first held-out toy runs used the natural input order. All three designs break exact ties toward low column indices, through DFG's enumeration order and LightGBM's split choice, so every held-out toy suite was re-run with the columns permuted per fit. That run is the headline.

- *Suite means:* they change by 0.01–0.04 in $\log_2$ against `nets`.
- *Single targets:* they change by up to 0.64, in both directions. The largest is par5 on `dlheld`: 539 in natural order, 839 permuted.

**What logistic regression alone does** [Numerical]. On halfspace-like targets the portfolio's win over the networks is a linear-model win.

- `lin` alone is clearly better than `nets_best` on:
  - 5 designed targets: dict, maj3, maj9, rand2, thr10;
  - 2 random formulas;
  - maj15 and thr30 on `dlheld`, maj21 and thr20 on `dlheld2`, and thrint on `dlheld3`.
- On those halfspaces the portfolio is at `lin`'s level or slightly above it: for example maj15 118 against 110, thr20 130 against 117, thrint 149 against 137.
- Excluding the targets `lin` wins makes the portfolio's mean margins larger, not smaller.

| split (datasets, cells) | reference | mean $\log_2$ ratio | better / tie / worse | datasets with lower mean error ($p$) |
|---|---|---|---|---|
| **HELDOUT2, fresh** (10, 53) | `mlp` | −0.44 | 15 / 36 / 2 | 9 of 10 (0.021) |
| | `mlp_reg` | −0.27 | 16 / 35 / 2 | 8 of 10 (0.11) |
| | `mlp_strong` | −0.28 | 15 / 36 / 2 | 8 of 10 (0.11) |
| | `mlp_best` | **−0.17** | 10 / 41 / 2 | 8 of 10 (0.11) |
| Part 7 tabular (11, 59) | `mlp` | −0.33 | 22 / 36 / 1 | 10 of 11 (0.012) |
| | `mlp_reg` | −0.16 | 16 / 40 / 3 | 9 of 11 (0.065) |
| | `mlp_strong`† | −0.34 | 22 / 30 / 2 | 10 of 11 (0.012) |
| | `mlp_best` | −0.11 | 12 / 44 / 3 | 8 of 11 (0.23) |
| images (2, 14) | `mlp` | −0.28 | 7 / 7 / 0 | 2 of 2 (0.5) |
| | `mlp_reg` | −0.14 | 2 / 12 / 0 | 2 of 2 (0.5) |
| | `mlp_strong` = `mlp_best` | +0.05 | 1 / 9 / 4 | 0 of 2 (0.5) |
| dev, in-sample (5, 24) | `mlp` | −0.38 | 4 / 20 / 0 | 5 of 5 (0.062) |
| | `mlp_reg` | −0.27 | 3 / 21 / 0 | 5 of 5 (0.062) |
| | `mlp_best` | −0.25 | 1 / 23 / 0 | 5 of 5 (0.062) |
| *Calibration, against `mlp_best`* (HELDOUT2, Part 7 tabular, images): `hgb` +0.08, +0.14, +0.26; baseline `mlp` +0.28, +0.22, +0.34. | | | | |

**Table 11:** `hyb_portfolio` on standard tasks [Numerical]. Mean $\log_2$ ratio: of the portfolio's test error to the reference's, over cells. The last column counts the datasets whose mean ratio over $n$ is below 0, with the two-sided sign-test $p$. †`mlp_strong` was run on the Part 7 tabular set only at $n \leq 10000$ (54 of 59 cells), and it is weak on pendigits and ring, so `mlp_best` is the stricter reference there. Source: `study/hybrid/RESULTS.md`.

## 5.4 Tabular results

**The fresh datasets** (Table 11, Figure 4) [Numerical].

- *Against `mlp_best`:* the mean error is lower on 8 of 10 datasets (sign test $p = 0.11$, not significant). The mean $\log_2$ error ratio is −0.17, a geometric-mean error about 11% lower, with 10 of 53 cells better and 2 worse.
- *Against single references* the margins are larger: −0.28 against the strong MLP ensemble, which has twice the budget, and −0.32 against one network of that recipe (15 better, 2 worse). The sign test is 8 of 10 against all three.
- *How strong the 8 of 10 is.* Three of the 8 datasets are within about ±0.05: page_blocks, banana and sleep. The large margins are on ann_thyroid (4 resolved cells), churn (2) and clean2. clean2's −1.12 comes mostly from $n \leq 300$ and has 1 resolved cell.
- *Losses.* The portfolio loses on texture (11 classes, 40 continuous inputs): its error is higher at every $n$ (+0.61), but no cell resolves, since errors are below 1.6% from $n = 1000$. Logistic regression alone matches the stack there, and the trees are 2–3× worse. It also loses on krkopt (18 classes, a chess endgame), resolved at $n \geq 10000$: 0.2037 against `mlp_strong`'s 0.1333 at full $n$.

**The Part 7 tabular set.** Its MLP references had been read by the designers.

- *Overall:* −0.11 against `mlp_best`, lower mean error on 8 of 11 datasets ($p = 0.23$).
- *Largest wins:* dna (−0.77, 3 resolved cells), ring (−0.50), mushroom (−0.31) and phoneme (−0.24).

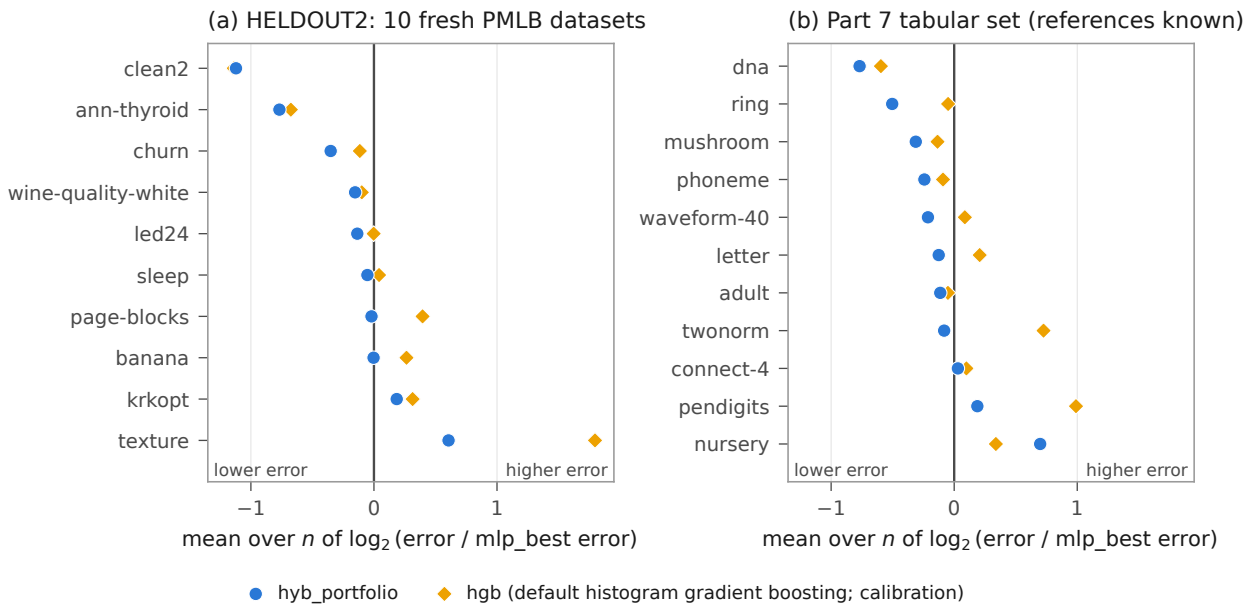



**Figure 4:** Per-dataset mean over $n$ of $\log_2$(error/`mlp_best` error), for `hyb_portfolio` and, for calibration, default `hgb`. Negative is better than the hindsight-best MLP. (a) The fresh HELDOUT2 set. (b) The Part 7 tabular set, whose MLP references the Part 8 builders had seen. Computed with `compare_hybrid.py`'s cell rule from the result files.

- *Losses:* nursery (+0.70, worse at $n = 1000$ and 3000), pendigits (+0.19, none resolved) and connect_4 (+0.03, worse at full $n$: 0.1566 against 0.1412).

**Calibration.** Default `hgb` alone does not beat the MLPs on average (+0.08 on HELDOUT2, +0.14 on the Part 7 set). The baseline `mlp` trails `mlp_best` by +0.28 and +0.22, which shows how much of the margin over `mlp` is the baseline's weakness. These numbers agree with the tabular-learning literature: tree ensembles and stacking tend to beat plain MLPs on medium-sized tabular data [12, 25], and careful regularization of the MLP narrows the gap [11, 14].

### 5.5 Images

There is no fresh image set, and the designers had seen the references, so this is the weakest evidence in Part 8 (Table 12, Figure 5) [Numerical].

- $n \leq 3000$. The portfolio's error is below every MLP variant's in all 8 cells, by about 1–13% relative. Against `mlp_best`, which is `mlp_strong` at every image $n$, only MNIST $n = 3000$ resolves as better: 4.70% against 5.43%.
- $n = 10000$. About equal: 3.24% against 3.20% on MNIST, and 13.07% against 12.64% on Fashion-MNIST.
- $n = 30000$ *and 60000.* The strong MLP ensemble is clearly better in all four cells. At 60000 it has 1.43% against the portfolio's 1.98% on MNIST, and 9.66% against 10.72% on Fashion-MNIST.
- *One network of the strong recipe.* `mlp_strong1` also has lower error than the portfolio at $n \geq 30000$ on both datasets, but this resolves only at MNIST $n = 30000$ (1.92% against 2.54%).

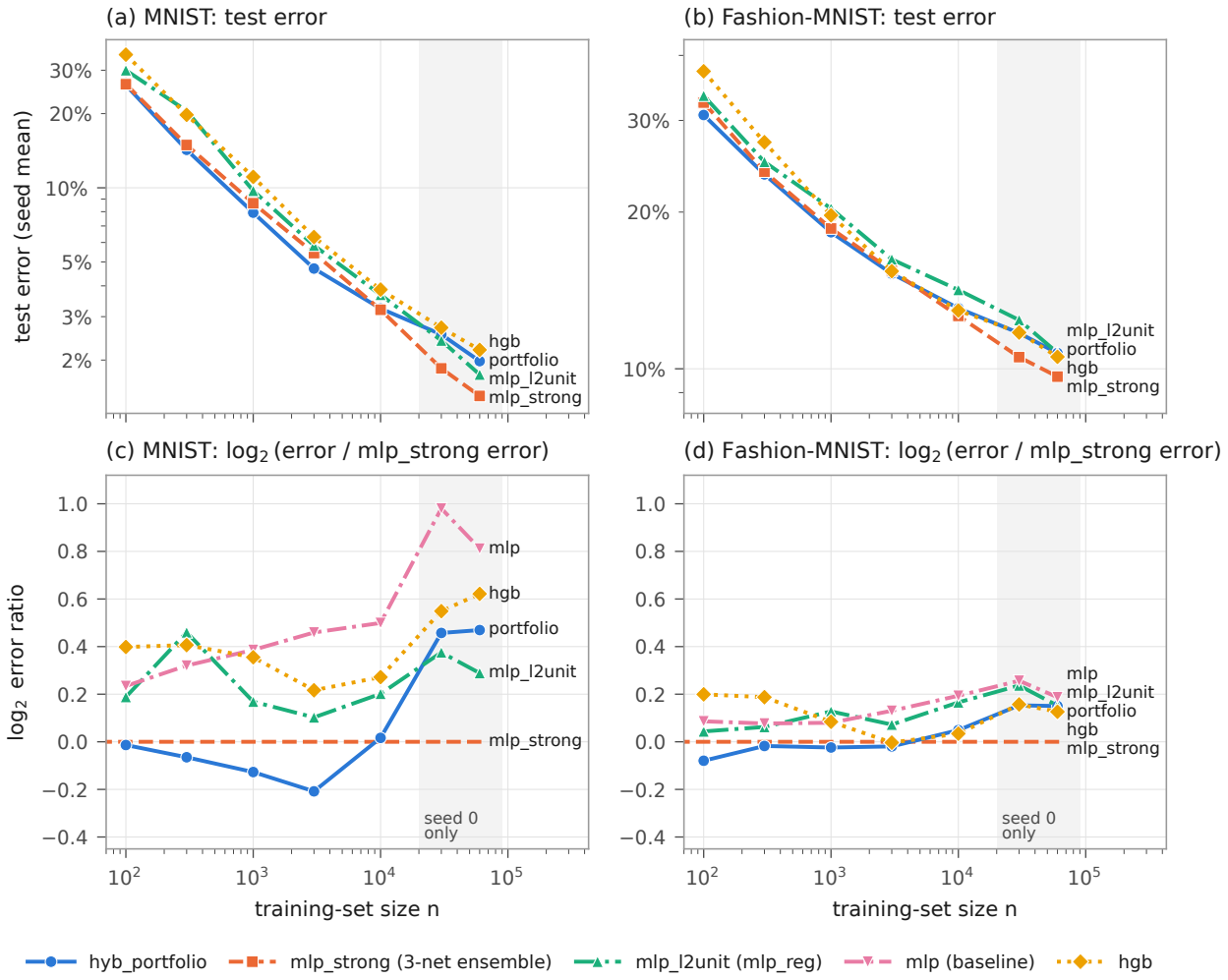



**Figure 5:** MNIST and Fashion-MNIST. (a, b) Seed-mean test error against training-set size for `hyb_portfolio`, the strong MLP ensemble `mlp_strong` (twice the FLOP budget), the regularized MLP `mlp_l2unit` and default `hgb`. (c, d) $\log_2$ of each learner's error over `mlp_strong`'s, adding the baseline `mlp`; the dashed line at 0 is `mlp_strong`. Shaded: $n \geq 30000$, where the portfolio and `mlp_strong` are seed 0 only. There is no fresh image set, and the Part 8 builders had seen the MLP references.

- *Default* `hgb`. On Fashion-MNIST at large $n$ the portfolio is no better than `hgb`: 10.72% against 10.54% at 60000.

The stronger reference reverses the large-$n$ cells, which the fairness audit had flagged as the ones most likely to flip. With enough data, a well-regularized MLP's learned continuous directions beat prototype memory and axis-aligned trees, the same reading as in Section 4.3 [Heuristic].

## 5.6 What carries which result

These ablations are post hoc and not part of the held-out test (Figure 6) [Numerical].

**Toy.**

- *Without DFG*, the design fails on the parity-like targets: par3or needs 7531 samples (62 with DFG), mixmaj 341 (96), sym9 7113 (1229), and poly4 is never learned. Its mean on `dlheld3`

| dataset | $n$ | mlp | mlp_l2unit | mlp_strong | mlp_strong1 | hyb_portfolio | hgb |
|---|---|---|---|---|---|---|---|
| MNIST | 300 | 0.1862 | 0.2052 | 0.1492 | 0.1528 | **0.1426** | 0.1977 |
| | 3000 | 0.0747 | 0.0583 | 0.0543 | 0.0556 | **0.0470** | 0.0631 |
| | 10000 | 0.0452 | 0.0368 | **0.0320** | 0.0327 | 0.0324 | 0.0386 |
| | 30000 | 0.0365 | 0.0240 | **0.0185** | 0.0192 | 0.0254 | 0.0271 |
| | 60000 | 0.0251 | 0.0175 | **0.0143** | 0.0161 | 0.0198 | 0.0220 |
| Fashion-MNIST | 300 | 0.2523 | 0.2498 | 0.2392 | 0.2449 | **0.2363** | 0.2724 |
| | 3000 | 0.1690 | 0.1623 | 0.1544 | 0.1571 | **0.1523** | 0.1540 |
| | 10000 | 0.1445 | 0.1417 | **0.1264** | 0.1305 | 0.1307 | 0.1294 |
| | 30000 | 0.1258 | 0.1241 | **0.1053** | 0.1131 | 0.1171 | 0.1174 |
| | 60000 | 0.1100 | 0.1073 | **0.0966** | 0.0992 | 0.1072 | 0.1054 |

**Table 12:** Seed-mean test error on images (selected $n$; all $n$ in Figure 5) [Numerical]. At $n \geq 30000$, every column except `mlp`, `mlp_l2unit` and `hgb` is seed 0 only. Source: `study/hybrid/HYBRID.md`, §4.4.

becomes +0.17 against `nets_best` (6 clearly better, 3 clearly worse). It still wins clearly on 6 of 10 fresh targets, with the trees carrying cnf3 (130) and mux11 (521). On `designed` it falls from −2.32 to −0.63 and never learns par4.

- *DFG alone* (its two members, stacked) is close to the full design: −1.21 on `dlheld3` and −2.13 on `designed`. The other members add on AND, OR and threshold structure: cnf3 213 against 146, mux11 826 against 523, and on `designed` and3 74 against 25.

**Real.**

- *No single member family matches the stack.* The best, LightGBM alone, is +0.08, behind the MLPs on average, while the full design is −0.19. The gain comes from combining members.
- *Different members carry different datasets.*
  - trees: churn, ann_thyroid and clean2;
  - kNN: banana;
  - logistic regression: texture;
  - ExtraTrees and logistic regression: led24 and sleep.
- *kNN matters on MNIST.* Without it, MNIST errors at $n = 100$, 300, 1000 and 3000 rise from 0.2563, 0.1458, 0.0779 and 0.0476 to 0.2687, 0.1609, 0.0913 and 0.0606, above the best MLP.
- *DFG runs on no HELDOUT2 dataset and on neither image set.* The spectral block and the oracle best single member per cell were not ablated.

**Reading** [Heuristic]. The toy result is Part 6's mechanism: exhaustive enumeration pays about $\ln N$ samples per character, where gradient descent on a generic MLP pays much more on targets without low-degree marginal signal. The classical members add constant factors on dense targets. The tabular result restates known facts about tree ensembles and stacking [9, 12].

## 5.7 Compute and prediction cost

- **Fit FLOPs**, by the learner's own counts:
  - toy: at most 0.75 of one network run (on `dlheld`);

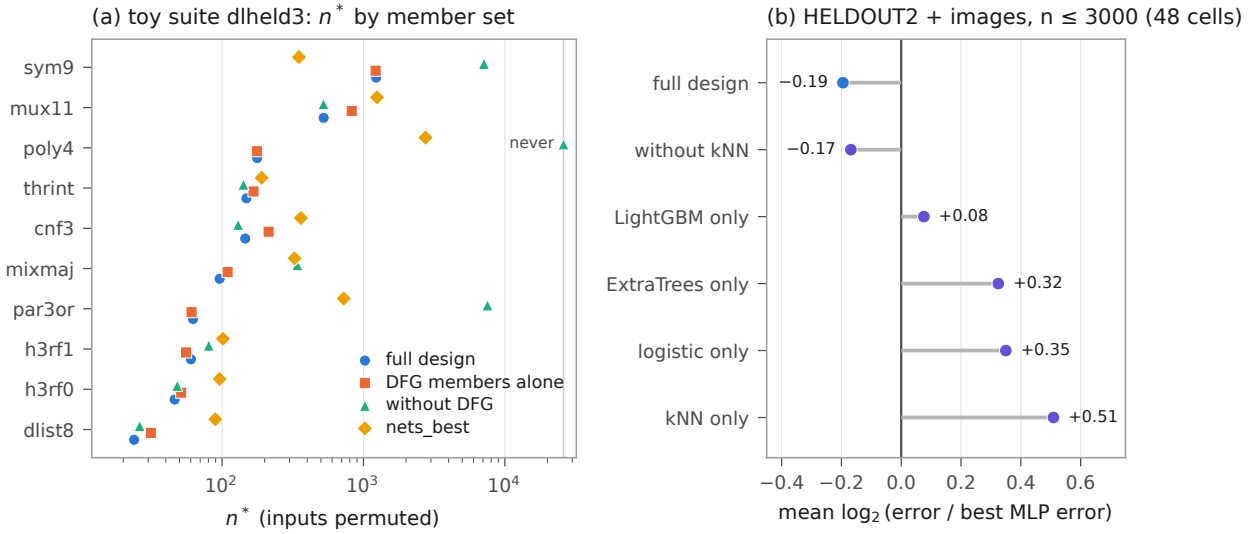



**Figure 6:** Post-hoc attribution ablations of the frozen `hyb_portfolio`, run with its `members` argument after the held-out results. (a) $n^*$ on `dlheld3` (inputs permuted) for the full design, the DFG members alone (stacked), all members except DFG, and `nets_best`. (b) Mean $\log_2$(error/best MLP error) over 48 cells (HELDOUT2 and the two image sets, $n \leq 3000$, seeds 0–1), for the full design, the design without kNN, and single member families. Here "best MLP" is the per-cell best seed-mean over seeds 0–1.

  - real: at most 0.24 of the budget on the Part 7 tabular set, 0.09 on HELDOUT2 and 0.54 on images.

- **Correction.** The compute audit found the LightGBM charge off by 0.05–1.7× against an algorithmic count that tracks CPU time. With its correction, the largest image fit is about 0.74 of the budget (range 0.58–0.90, Fashion-MNIST $n = 3000$).
- **Not a bound.** The toy compute is bounded by measurement, not by construction: the audit's worst-case bound is about 1.1.
- **Prediction and CPU time.** Prediction is not counted for any learner, but it is not cheap here. The kNN members scan all $n$ training points: at $n = 60000$ that is $1.9 \times 10^8$ FLOPs per query, about 350× an MLP's. CPU time is also far larger on images: about 8 CPU-minutes for a Fashion-MNIST fit at $n = 3000$, against an estimated 25–40 s for the three MLP runs that the budget corresponds to.

The portfolio's real-data wins are therefore not bought with more FLOPs than the references, but they are bought with more CPU time on images and far more prediction cost (Appendix C).

### 5.8 The secondary designs

- `hyb_stage` runs stagewise.
  - *Design.* A multi-class port of DFG's search (characters on binary, one-hot and thermometer bits, linear and spectral blocks, kNN votes) is read out by penalized logistic regression. LightGBM is then fitted on its margins, and a stage set is chosen by cross-validation.
  - *Where it beats the primary.* On `dlheld3` (−1.60 against −1.41; sym9 591 against 1229), and at full $n$ on krkopt (0.1622 against 0.2037) and connect_4 (0.1360 against 0.1566).

| | `hyb_stage` | `hyb_union` |
|---|---|---|
| toy `designed`, permuted, vs `nets_best` | −2.11 (21/0) | −1.85 (21/0) |
| toy `dlheld3`, permuted, vs `nets_best` | −1.60 (9/1; sym9 591) | −0.71 (8/1; sym9 3151) |
| HELDOUT2 vs `mlp_strong` | −0.16; 10/40/3; 7 of 10 (0.34) | −0.10; 9/42/2; 6 of 10 (0.75) |
| HELDOUT2 vs `mlp_best` | −0.05; 6/40/7; 6 of 10 (0.75) | +0.01; 2/49/2; 6 of 10 (0.75) |
| Part 7 tabular vs `mlp_best` | +0.02; 6/46/7; 7 of 11 (0.55) | not run |
| images vs `mlp_strong` | +0.18; 0/8/6 | +0.13; 0/10/4 |

**Table 13:** The two secondary designs, frozen and evaluated with scope cuts [Numerical]. Entries as in Tables 10 and 11.

  - *Where it is worse.* On texture and on images.

- **`hyb_union`** is unified boosting over trees, Fourier characters, linear steps and honest blocks, with a 480 s wall-clock stop.
  - It is weakest on the toy suite, and about level with `mlp_best` on HELDOUT2.
  - Its parity family is essentially idle on real data.
  - Its large-$n$ results depend on machine load.

Neither secondary design would change the overall answer (Table 13).

### 5.9 What "discrete" means, and the honest claim

**Discreteness.** What the hypothesis is made of:

- a weighted sum of tree cells, Fourier characters, nearest-neighbour votes, the raw inputs, and DFG's spectral features;
- fitted continuously: only convex readouts and stacking weights, plus boosting's Newton leaf values on fixed partitions;
- no neural network, and no gradient descent on a continuous feature parameter.

Some components are borderline.

- *Gradient boosting* is functional gradient descent over trees. The line between this method and the networks is "no continuous feature parameters", not "no gradients".
- *kNN votes* are memory under a fixed Euclidean metric.
- *Logistic regression* on continuous inputs is a linear background, not a discrete feature.
- *DFG's spectral block* produces $(u \cdot x)^2$ with $u$ a continuous eigenvector of a residual-weighted second-moment matrix: one closed-form step of continuous feature learning. In the primary design it acts only through DFG, so only on Boolean two-class inputs. It was not ablated.

**The claim, worded as the evidence allows.** One learner, which never trains a neural network or any continuous feature parameter:

- needs clearly less data than the study's gradient-descent-trained networks (width-128 MLPs, 20000 Adam steps, three loss and regularization settings) on 9 of 10 fresh toy Boolean targets, and clearly more on one symmetric function;

- has lower error on average than the hindsight-best MLP on 10 fresh tabular datasets, which is not significant (8 of 10, $p = 0.11$);
- on MNIST and Fashion-MNIST, is slightly below every MLP at $n \leq 3000$ (resolved in 1 of 8 cells), level at $n = 10000$, and clearly above a regularized ensemble of three MLPs at $n \geq 30000$.

It is an AutoML-style portfolio, Part 6's character search plus a standard stack, and its two halves are carried by different parts: the toy suites by the character search, and real data by the classical stack. It does not unify the two kinds of structure.

# 6 Sum features and permutation-invariant image members (Part 9)

## 6.1 Question and protocol

Part 8 left two clear losses of `hyb_portfolio` (here *v1*) against gradient-descent-trained networks (Section 5).

1. *Symmetric functions.* On sym9 $= [|x_1 + \cdots + x_9| \leq 1]$, v1 needs 1229 samples and the best network 350. The function's Fourier weight is spread over many characters of degrees 2, 4, 6 and 8, and a character search pays for each one separately.
2. *Images at large $n$.* On MNIST and Fashion-MNIST at $n \geq 30000$, a regularized ensemble of three MLPs is clearly better than v1, whose image members are nearest neighbours, axis-aligned trees and a multinomial logistic regression.

Part 9 adds two things and tests them on fresh held-out sets.

- *Sum features*, discrete functions of signed sums of inputs:

$$\phi(x) = g\Big(\textstyle\sum_{i \in S} s_i x_i\Big), \qquad S \subseteq [d],\ s_i \in \{-1, +1\},$$

  with $g$ the indicator of a set of values of the integer sum. They enter DFG as a new block type (DFG v1.2).
- *Discrete, pixel-permutation-invariant image members.* The comparator is an MLP, which is invariant to a fixed permutation of its inputs, so the image members may not use the pixel layout; a method that used it would have to be compared with a convolutional network. The QuickDraw sets are stored with their columns permuted.

**Pre-registration.** A protocol, `study/part9/PROTOCOL.md`, was committed before any Part 9 design work (commit `82ca257d`). It fixed:

- *Data access.* Development: the toy `designed` suite and a new symmetric dev suite `dlsymdev`; MNIST, Fashion-MNIST and a new QuickDraw set `qd_dev`; the tabular DEV sets. Held out, fixed before any design: the toy suite `dlheld4` and the QuickDraw sets `qd_a`, `qd_b` (Section 6.2).
- *Budgets* as in Part 8: one network training run per toy fit, three MLP training runs per real fit, everything counted, prediction reported separately.
- *References.* Toy: `nets`, `nets_wd`, `nets_cewd` (logistic loss and weight decay 0.1) and their per-target hindsight minimum `nets_best`, plus `lin` and v1. Images: `mlp`, `mlp_l2unit`,

| amendment | when | content |
|---|---|---|
| 1 | before any design (`f6ad9afe`) | the integration rule's dev image cells are MNIST, Fashion-MNIST and `qd_dev` at $n \in \{1000, 10000, 60000\}$, seed 0 |
| 2 | after builder S froze, before any held-out run of a Part 9 learner (`7449313e`) | S had seen the source of `dlheld4`'s target definitions; a fifth suite `dlheld5` is drawn from a fixed random family |
| 3 | after the protocol audit, before any run on the new suite (`0c4db784`) | `dlheld5`'s family was written after S's limits were read, inside S's representable class; a sixth suite `dlheld6` is written by an agent blind to the design, and Q1 is answered on it |
| 4 | after the four audits (`1a767058`, wording corrected in `bc22bb8c`) | stronger references (wider and CPU-matched MLPs; more toy weight decays) and post-hoc controls (an MLP on the image member's pooled features, a CNN on unpermuted pixels, member attribution); Q1's `nets_best` on `dlheld6` is taken over all five network variants, and Q2 is also answered against `mlp_best+`. Fixed before any v2 toy result or Q2 image cell was read; the orchestrator had seen v2's `qd_a`/`qd_b` test errors at $n \leq 1000$ in progress logs. Two compute cuts were recorded after v2's held-out image results had been read (`a58962d5`, 07:00 UTC): `mlp_xl` ran at $n = 60000$ only, at the weight decay `mlp_wide` chose on validation, and `cnn` on `qd_a` at $n = 1000$ and 10000 only |

**Table 14:** Amendments to the Part 9 protocol (`study/part9/PROTOCOL.md`).

`mlp_strong` and its single network `mlp_strong1`, the per-cell hindsight minimum `mlp_best`, `hgb` and v1. Seed 0 only at $n \geq 30000$ for every learner, 3 seeds below.

- *Design tasks.* Builder S: DFG v1.2 as a replacement for the portfolio's DFG member. Builders I1 and I2: two independent image members, each added to the portfolio. Builders work on development data only, freeze, and run one full dev evaluation.
- *An integration rule* (Section 6.5), applied before any held-out run of the integrated learner `hyb_portfolio2` (here *v2*).
- *Two questions*, with Part 8's verdict rules:
  - **Q1 (sum features).** On the held-out toy suite, is v2's mean $\log_2 n^*$ ratio against `nets_best` below 0, with more clearly-better than clearly-worse targets? Is it below v1's on the symmetric targets?
  - **Q2 (images at large $n$).** On `qd_a` and `qd_b` at $n \geq 10000$, is v2's mean $\log_2$(error/reference error) below 0 against `mlp_strong` and against `mlp_best`, and below v1's?

**Amendments.** Four amendments were made, each recorded with its commit (Table 14). The first was a compute restriction before any design. The other three answer exposures and audit findings that came up during the work (Section 6.9); each was fixed before the results it affects were read, except two disclosed parts of Amendment 4: the orchestrator had seen v2's small-$n$ image errors before it was fixed, and two of its runs were cut for compute after v2's held-out image results had been read.

## 6.2 New data

**QuickDraw.** The images are 28×28 bitmaps from the QuickDraw game [13]. Thirty of the 345 categories were drawn at random (seed 9191): the first 10 form `qd_dev`, the next 10 `qd_a`, the last 10 `qd_b`, so the three sets are disjoint.[1] Each category contributes 6000 training and 1000 test drawings, so each set has a 60000-point training pool and a 10000-point test set, with 10 balanced classes and $d = 784$. The pixel columns are permuted by one fixed permutation (seed 9192) and scaled to $[0, 1]$. QuickDraw is much harder than MNIST: on `qd_dev` the strong MLP ensemble ends at 13.2% error at $n = 60000$, against 1.4% on MNIST.

**Toy suites** on $\{\pm 1\}^{30}$, defined in `study/boolfuncs.py`.

- `dlsymdev` (development, 8 targets): sym7, exactly 3 of 8 inputs +1, $|x_1 + \cdots + x_{11}| \in \{3, 5\}$, $|x_1 + \cdots + x_9| \geq 5$, a signed-sum version of sym9, and three mixtures: $[|x_1 + \cdots + x_6| \leq 2] \oplus x_7 x_8$, an equivalence of symmetric functions of two disjoint blocks, and $\text{sign}(x_1 + \cdots + x_6 + 3x_7 x_8 + \frac{1}{2})$.
- `dlheld4` (held out, fixed before any design, 12 targets): 9 symmetric or signed-sum targets of new sizes and accept sets, with mixtures with parities and gates (for instance $[|x_1 + \cdots + x_{13}| \geq 5]$, $[|x_1 + \cdots + x_7| \leq 1] \oplus x_8$ and $\text{sign}(x_1 + \cdots + x_5 + 2x_6 x_7 + 2x_8 x_9 x_{10})$), and 3 non-symmetric controls (two random formulas and a 3-term DNF). Builder S saw its source code (Section 6.9).
- `dlheld5` (held out, 12 targets): 8 random signed sums with random accept sets, some combined with a gate, 2 threshold targets $\text{sign}(t + w\chi_T + \frac{1}{2})$ and 2 random-formula controls, drawn once (seed 9393) after builder S froze. The audit found its family inside S's representable class (Section 6.9).
- `dlheld6` (held out, 12 targets, Table 15): written by a fresh agent that had not seen the sum-feature design, the design notes, the other held-out suites or any result file. Its inputs were the pre-registered one-line description "9 symmetric / signed-sum targets of new sizes and accept-sets, mixtures with parities and gates, and 3 non-symmetric controls", the Part 9 question with sym9 (from `dlheld3`) as the motivating failure, three `dlsymdev` targets as a format example, and the constraints ($d = 30$, 5–14 relevant inputs, balance as the only rejection rule), plus the task context and a one-sentence gloss of the description that named random formulas, DNFs and decision lists as example controls `study/part9/PROTOCOL.md`; the three controls it drew are of these kinds. Its sums with a heavier-weighted inner term (part of the "composite sums" stratum) follow, by its own account, the style of the development target $\text{sign}(x_1 + \cdots + x_6 + 3x_7 x_8 + \frac{1}{2})$. It stated a stratified random family in writing and drew from it once (seed 52817), rejecting draws only for balance, $\Pr[f = +1] \in [0.25, 0.75]$ `study/part9/heldout6/FAMILY.md`. Each block is plain or randomly signed with probability $\frac{1}{2}$, and its accept set is a threshold, a centred band, an interval, a residue class mod 3 or 4, or an arbitrary non-constant set, each with probability $\frac{1}{5}$.

Q1's *sum-structured subset* is every target but the controls: 9 targets of `dlheld4`, 10 of `dlheld5` and 9 of `dlheld6`.

## 6.3 Sum features: DFG v1.2

A *block* is a subset $S$ of the inputs, signs $s_i \in \{\pm 1\}$, and a fold flag; its statistic is the integer $t(x) = \sum_{i \in S} s_i x_i$, or $|t(x)|$ when folded. The training values of $t$ are cut into contiguous groups of

[1] `qd_dev`: ocean, motorbike, matches, grapes, paper clip, campfire, washing machine, soccer ball, frying pan, foot. `qd_a`: broccoli, oven, leaf, strawberry, The Eiffel Tower, star, piano, camouflage, coffee cup, skateboard. `qd_b`: spoon, lantern, computer, marker, door, fireplace, bench, streetlight, stove, panda.

| target | stratum | definition | $k$ | $\Pr[f=+1]$ |
|---|---|---|---|---|
| h6_00 | one sum | $[\frac{1}{2}(s+13) \bmod 3 \neq 1]$, $s = x_1 + \cdots + x_8 - x_9 - \cdots - x_{12} + x_{13}$ | 13 | 0.667 |
| h6_01 | one sum | $[s \in \{-9,-7,-5,-1,1,7,9\}]$, $s = -x_1 + x_2 - x_3 + x_4 + x_5 + x_6 - x_7 - x_8 - x_9$ | 9 | 0.602 |
| h6_11 | one sum | $[x_1 + \cdots + x_7 = 1]$ | 7 | 0.273 |
| h6_02 | sum $\oplus$ parity | $[1 \le x_1 + \cdots + x_9 \le 5] \oplus x_{10}x_{11}$ | 11 | 0.500 |
| h6_06 | sum $\oplus$ parity | $[x_1 + \cdots + x_6 \in \{-6,-2,0,6\}] \oplus x_7x_8$ | 8 | 0.500 |
| h6_07 | sum with a gate | $\neg x_{11}\,?\,[x_1 - x_2 - \cdots - x_6 \in \{-2,4\}] : [x_7 + x_8 + x_9 - x_{10} \le 0]$ | 11 | 0.508 |
| h6_08 | sum with a gate | $x_{11}\,?\,[\lvert x_1 + \cdots + x_4\rvert \le 2] : [\frac{1}{2}(s_2+6) \bmod 3 = 0]$, $s_2 = -x_5 - x_6 + x_7 - x_8 + x_9 - x_{10}$ | 11 | 0.609 |
| h6_09 | composite sums | $[-4 \le x_1 + \cdots + x_6 \le 0]$ XOR $[3 \le -x_7 - x_8 + x_9 - x_{10} - x_{11} \le 5]$ | 11 | 0.588 |
| h6_10 | composite sums | $[\lvert x_1 + \cdots + x_{12} - 3x_{13}\rvert > 1]$ | 13 | 0.686 |
| h6_03 | control (DNF) | $(\neg x_4 \wedge x_5) \vee (\neg x_2 \wedge \neg x_6) \vee (x_1 \wedge \neg x_3 \wedge x_4)$ | 6 | 0.531 |
| h6_04 | control (read-once) | $\neg x_1 \vee (x_2 \wedge \neg x_3 \wedge (x_4 \vee x_5))$ | 5 | 0.594 |
| h6_05 | control (decision list) | 7 rules on $x_1, \ldots, x_8$ | 8 | 0.543 |

**Table 15:** The blind held-out suite `dlheld6`. $[\cdot]$ is $\pm1$-valued, $\oplus$ is the product of $\pm1$ values, XOR acts on truth values (true iff exactly one holds, that is $-ab$), $c\,?\,a : b$ is a multiplexer, and $k$ is the number of relevant inputs (the first $k$ columns; the discrete learners see the columns permuted). Both "sum with a gate" slots came out as multiplexers, so no AND/OR-gate or majority form was drawn.

at least $\max(3, 0.005n)$ points, and the block's features are the group indicators $[t(x) \in \text{group } v]$, with the most populous group dropped. Every function of $t$, and so every symmetric function of the signed inputs in $S$, is then a linear combination of the block's features, up to merged rare values. There are three variants: *additive* (the indicators themselves), *modulated* ($\chi_T(x)$ times the indicators, for a character $\chi_T$ already in the model, with $T \cap S = \emptyset$) and *product* (joint cells of two blocks on disjoint supports). An additive and a modulated block on the same $(S, s)$ together represent $g(t + w\chi_T)$ for any integer $w$.

**The test.** A candidate is accepted when the residual's projection on its group indicators passes a $\chi^2$ tail bound with a union bound over the whole class. With residual $r$, $\hat{\sigma}^2 = \overline{r^2}$ and $n_v$ points in group $v$, the statistic is $T = \sum_v n_v\, \overline{r}_v^2/\hat{\sigma}^2$, and the block passes if $T > m + 2\sqrt{mt} + 2t$ [18], with $m$ the number of occupied groups minus one and

$$t = \ln\!\big(4K\tbinom{d}{k}\, 2^{k-1} \cdot 2\,(1 + \#\text{modulators})\, k_{\max}/\delta\big)$$

for a block of size $k$ ($K = 4$, $\delta = 0.1$, $k_{\max} = 24$). The bound covers every $(S, s)$ of every size up to $k_{\max}$, both fold variants and every modulator, so the adaptive search below does not invalidate it. It is not free: at $d = 30$ the threshold is about 63 for $k = 5$ and 95 for $k = 11$, and since $T \le n$ no 7-input block can pass below $n \approx 75$.

**The search** uses only the training residual. Candidates come from prefixes of the inputs ordered by the top eigenvectors of the residual pair matrix $\overline{r\,xx^\top}$ (an even symmetric function on $(S, s)$ makes its restriction to $S$ rank one with sign pattern $s$), from prefixes ordered by degree-1 correlation, and from greedy growth started at the strongest pairs; the best are refined by single add, remove and sign-flip moves, and the winner is tested on all $n$. Each DFG round tries up to 2 additive blocks,

one product block, modulated blocks for the 4 heaviest characters, and products of earlier blocks. The readout, the stopping rule and the budget caps are v1.1's.

**What is discrete.** The hypothesis is a weighted sum of characters, group indicators of integer signed sums (optionally times a character), the inputs, and v1.1's spectral $(u \cdot x)^2$ features, which are unchanged and remain the one feature with a continuous direction. The eigenvectors only order inputs inside the search; removing them changed the test error by at most 0.002 on 10 dev fits (Section 6.9).

**Development.** Every change during development was driven by a failure on a dev target (local refinement for band11, degree-1 seeds for ex3of8, the product routes for twosym, and an acceptance test beyond the round's characters after a spurious block on sparsepoly). On the dev suites the S variant of the portfolio, `p9_sum`, has mean $\log_2 n^*$ ratio $-2.04$ against `nets_best` on `dlsymdev` (8 of 8 targets clearly better; v1: $+0.11$, 2 better and 5 worse, and v1 never learns band11) and $-2.39$ on `designed` (v1: $-2.32$; no target worse than v1 by more than 0.05). These numbers are in-sample: S built the design while looking at them.

### 6.4 Image members

Both members run only when every column is continuous and $d \geq 256$, so tabular and toy inputs follow v1's code path, random stream and predictions. Builder I1 and the resumed I2 session developed on validation rows of the training pool; the first I2 session, killed by a container restart, scored its development runs on the MNIST and Fashion-MNIST test sets and once on the `qd_dev` test set at $n = 10000$ (Section 6.9).

**`p9_gates` (builder I1).** Two objects are built from the training rows of each fit.

- *Pooling sets.* For each non-constant column $i$ and $k \in \{4, 9, 16\}$, $S_i(k)$ is the set of the $k$ columns most correlated with column $i$ on the first $\leq 20000$ training rows. The pooled feature is the subset sum $p_i(x) = \sum_{j \in S_i(k)} x_j$. The sets are computed from a data statistic, never from the pixel layout, and a column permutation permutes them exactly.
- *Prototype pairs.* 300 pairs $(a, b)$ of training points of different classes, with feature $(x_a - x_b) \cdot x$. A threshold on it compares the distances to the two prototypes.

The members are `gknn_<k>`, $k$-nearest-neighbour votes ($k \in \{1, 3, 9, 27\}$) on the pooled sums at scale 9, and `ggbt`, LightGBM on the bank $[x,\ p(x)$ at scales $4, 9, 16,\ (x_a - x_b) \cdot x]$ of about 3000–3200 columns (31 leaves, learning rate 0.1, 12% of the columns per tree, 63 bins, 100 rounds below 5000 points and otherwise early stopping on an inner validation split, 150–400 rounds). Each split of `ggbt` is a gate $[u \cdot x > t]$ with $u$ a pixel, a pooling set or a prototype pair and $t$ a bin boundary.

**`p9_bank` (builder I2)** is a closed-form ridge readout over a screened bank of prototype-pair threshold bits $[(x_a - x_b) \cdot x > t]$, with $b$ the nearest other-class point to $a$ in a reference sample, and the bank grown in 8 rounds by correlation with the current ridge residual. Its builder session was killed by a container restart and resumed by a new agent from the partial files.

**How discrete these are.** Nothing trains a neural network or fits a continuous feature direction. But the discreteness audit (Section 6.9) found that the two ingredients that matter are not discrete in any stronger sense. The pooling sets recover the pixel grid from correlations: on the permuted

| dataset | $n$ | p9_gates | p9_bank | v1 | mlp_strong | p9_gates vs mlp_best |
|---|---|---|---|---|---|---|
| MNIST | 1000 | **0.0722** | 0.0739 | 0.0789 | 0.0866 | better (−0.26) |
| | 10000 | 0.0299 | **0.0289** | 0.0318 | 0.0320 | tie (−0.10) |
| | 60000 | 0.0167 | 0.0194 | 0.0198 | **0.0143** | tie (+0.22) |
| Fashion-MNIST | 1000 | **0.1805** | 0.1844 | 0.1839 | 0.1860 | tie (−0.04) |
| | 10000 | **0.1247** | 0.1299 | 0.1300 | 0.1264 | tie (−0.02) |
| | 60000 | **0.0964** | 0.1077 | 0.1072 | 0.0966 | tie (−0.00) |
| qd_dev | 1000 | **0.2217** | 0.2402 | 0.2622 | 0.2704 | better (−0.29) |
| | 10000 | **0.1467** | 0.1612 | 0.1680 | 0.1749 | better (−0.25) |
| | 60000 | **0.1147** | 0.1302 | 0.1341 | 0.1324 | better (−0.21) |
| mean $\log_2$ vs mlp_best | | −0.105 | −0.007 | +0.046 | | |

**Table 16:** The 9 dev image cells of the integration rule: test error, seed 0 for the portfolio variants; mlp_strong (= mlp_best at every cell) is a 3-seed mean at $n \leq 10000$. Brackets: $\log_2$(error/mlp_best error). In-sample for the builders, who tuned on validation rows of the same datasets (the first I2 session also on their test sets, Section 6.9).

qd_dev data, 98% of the members of $S_i(4)$ lie within one grid step of pixel $i$, so $p_i$ is a $3 \times 3$ or $5 \times 5$ sum-pool on the true grid, recovered from data [6, 19]. The prototype-pair projections are continuous oblique directions, unfitted but drawn from the data, in the same category as nearest neighbours. And LightGBM chooses its splits by gradient gain, which is functional gradient descent (Section 5.9). ggbt is permutation invariant only in distribution: a column permutation changes its predictions about as much as a new seed.

## 6.5 Integration and freeze

The rule, fixed before any design: v2 is v1 with (i) DFG v1.2 in place of DFG v1.1 if, on designed and dlsymdev (inputs permuted, against nets_best), the S variant's mean $\log_2 n^*$ ratio is below v1's and no dev target gets worse by more than 0.5 in $\log_2$; and (ii) the image member whose portfolio variant has the lower mean $\log_2$(error/mlp_best error) over the 9 dev image cells, if that mean is below v1's.

- (i) is met: −2.30 (29 targets better, 0 worse) against v1's −1.71 (23 better, 5 worse); the largest increase on any dev target is +0.05. This part of the rule is a sanity gate rather than evidence: S had fixed, on the same cells, the two regressions that had crossed the 0.5 threshold (andxor and sparsepoly).
- (ii) is met by p9_gates: −0.105 against p9_bank's −0.007 and v1's +0.046 (Table 16).

v2 was assembled from p9_gates' file with two textual changes (the DFG module path and the name) and frozen at 02:11 UTC on 27 October (md5 6837fa52...). Its first held-out image cell crashed at $n = 100$: below 500 training points the gates booster lowered LightGBM's min_data_in_leaf under its construction value, which the dev grid ($n \geq 1000$) never exercised. A one-line fix that cannot act at $n \geq 1000$ was made before v2 had produced any held-out image result and is recorded (md5 e38af404...); it leaves the predictions at qd_dev, $n = 1000$ unchanged.

## 6.6 Held-out results: toy suites

Table 17 gives $n^*$ per target on the blind suite dlheld6, Table 18 the summaries on every Part 9 suite, and Figure 7 the targets of the three held-out suites against nets_best. Per-target tables for dlheld5 and dlheld4 are in Appendix B.

| target | v2 | v1 | lin | hgb | nets | nets_wd | nets_cewd | cewd 0.3 | cewd 1.0 | nets_best | $\log_2$ v2 / nets_best | $\log_2$ v2 / v1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| h6_00 | never | never | never | never | never | never | never | never | 12304 | 12304 | $+\infty$ | both never |
| h6_01 | 429 | 7045 | never | never | 3071 | 1946 | 1706 | 1684 | 1655 | 1655 | $-1.95$ | $-4.04$ |
| h6_02 | 428 | 464 | never | never | 1869 | 1401 | 1012 | 996 | 1250 | 996 | $-1.22$ | $-0.12$ |
| h6_06 | 342 | 878 | never | never | 3726 | 3319 | 2366 | 1872 | 1877 | 1872 | $-2.45$ | $-1.36$ |
| h6_07 | 678 | 2277 | never | 4492 | 3215 | 2619 | 2849 | 2170 | 1812 | 1812 | $-1.42$ | $-1.75$ |
| h6_08 | 446 | 1179 | never | 5940 | 3119 | 2437 | 2732 | 1720 | 1856 | 1720 | $-1.95$ | $-1.40$ |
| h6_09 | 392 | 2582 | never | 2706 | 1450 | 871 | 813 | 744 | 751 | 744 | $-0.93$ | $-2.72$ |
| h6_10 | 702 | 8775 | never | never | 1414 | 915 | 621 | 501 | 451 | 451 | $+0.64$ | $-3.64$ |
| h6_11 | 208 | 1009 | never | 2264 | 881 | 704 | 402 | 362 | 357 | 357 | $-0.78$ | $-2.28$ |
| h6_03 (control) | 98 | 98 | never | 103 | 323 | 232 | 271 | 235 | 231 | 231 | $-1.23$ | $+0.00$ |
| h6_04 (control) | 29 | 29 | 96 | 59 | 116 | 96 | 120 | 113 | 121 | 96 | $-1.71$ | $+0.00$ |
| h6_05 (control) | 33 | 33 | 48 | 59 | 78 | 56 | 76 | 62 | 75 | 56 | $-0.78$ | $+0.00$ |

**Table 17:** $n^*$ (samples to 10% test error, seed mean) on the blind held-out suite dlheld6 (Table 15); the discrete learners see the inputs permuted. "never": not reached by $n = 16384$. nets_best is the per-target minimum over all five network variants run on the suite, including Amendment 4's weight decays 0.3 and 1.0 (hindsight); the pre-registered three give the numbers of the Q1 paragraph. Below the rule: the three non-symmetric controls.

| suite | status | v2 vs nets_best all targets | v2 vs nets_best sum-structured | v1 vs nets_best sum-structured | v2 vs v1 sum-structured |
|---|---|---|---|---|---|
| dlheld6 | held out, blind (Q1) | $-1.25$ (10/2) | $-1.26$ (7/2) | $+0.91$ (3/6) | $-2.16$ (7/0) |
| dlheld5 | held out, inside S's class | $-2.40$ (10/0) | $-2.13$ (8/0) | $+0.03$ (3/6) | $-1.93$ (9/0) |
| dlheld4 | held out, definitions seen by S | $-1.68$ (12/0) | $-1.67$ (9/0) | $+0.19$ (4/3) | $-1.86$ (8/0) |
| dlsymdev | development | $-2.04$ (8/0) | $-2.04$ (8/0) | $+0.11$ (2/5) | $-2.28$ (8/0) |
| designed | development | $-2.39$ (21/0) | — | — | — |

**Table 18:** Mean $\log_2 n^*$ ratio (clearly better / clearly worse targets, $|\log_2| > 0.25$ or one side never) on the Part 9 suites. "Sum-structured" excludes the controls; designed has no designated controls and is mostly not sum-structured, so it has no sum-structured columns. On the dev suites v2 is p9_sum (identical on toy inputs). On dlheld6 nets_best includes Amendment 4's two extra weight decays; against the three pre-registered variants the first three columns read $-1.45$ (10/0), $-1.53$ (7/0) and $+0.63$ (3/4).

**Q1 on dlheld6.** Against nets_best over the three variants named in the pre-registration, v2's mean $\log_2 n^*$ ratio is $-1.45$, with 10 of 12 targets clearly better and none worse (about 2.7× less data on average); on the 9 sum-structured targets it is $-1.53$ (7 better, none worse), against v1's $+0.63$ (3 better, 4 worse). All three parts of Q1 are met. Two sum-structured targets are not wins: h6_10, a band of $x_1 + \cdots + x_{12} - 3x_{13}$, is a tie ($+0.18$), and h6_00, a residue class mod 3 of a 13-input count, is learned by none of these learners.

**With two more weight decays (Amendment 4, R3).** Logistic loss with weight decay 0.3 and 1.0 improves nets_best on most sum-structured targets, by 0.27 in $\log_2$ on average where both are finite, within the 0.2–0.5 the fairness audit predicted. The most regularized variant also learns h6_00, at $n^* = 12304$, the only learner to do so: the target is a function of one signed sum, inside v2's feature class, but v2's search does not find it. Against this nets_best over five variants (the one in Tables 17 and 18), v2's mean is $-1.25$ with 10 targets clearly better and 2 clearly worse (h6_00, and h6_10, where weight decay 1.0 needs 451 samples against v2's 702); on the sum-structured targets it is $-1.26$ (7 better, 2 worse) against v1's $+0.91$. This five-variant nets_best is the Q1 reference as amended (Amendment 4), and Q1's criteria hold against it too.

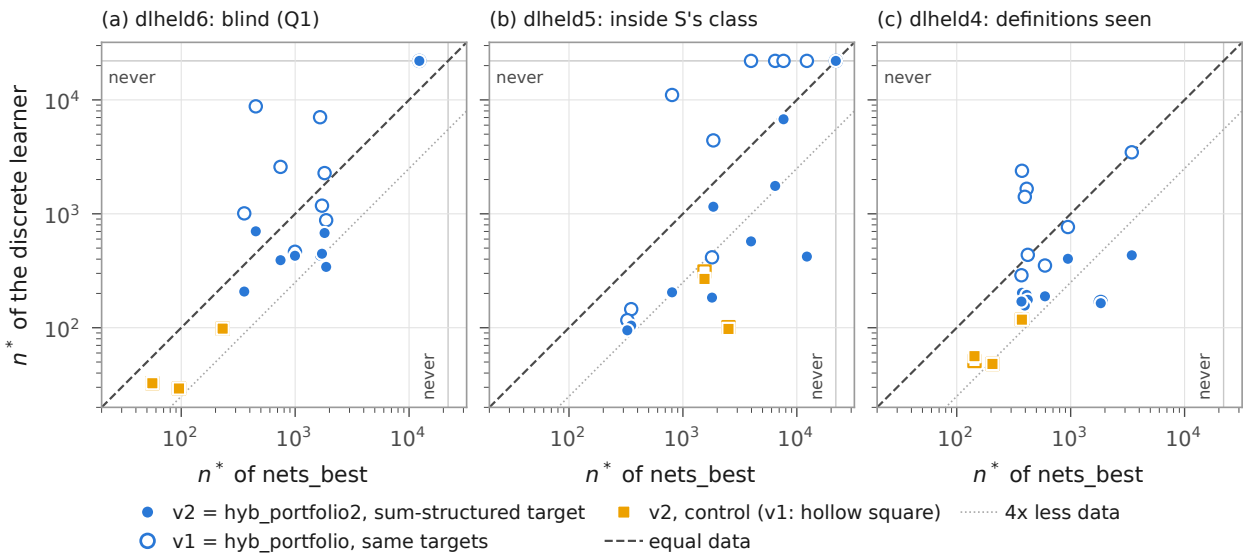



**Figure 7:** $n^*$ of v2 (filled) and v1 (hollow) against `nets_best` (on `dlheld6` over all five network variants) on the three held-out Part 9 suites; squares are controls; a target not learned by $n = 16384$ is drawn at $2.2 \times 10^4$ on that axis ("never"), and the point in the top-right corner of (b), h5s1, was learned by no learner. Points below the dashed line need less data than the best network.

**What the sum features add.** On the controls v2 and v1 have the same $n^*$, as they should, since no sum block passes there. On the sum-structured targets v2 needs $2^{2.16} \approx 4.5\times$ less data than v1 on average: it is clearly better on 7 of the 8 that both learn and level on the eighth. The largest gains are on h6_01, a randomly signed sum with an arbitrary accept set (429 against v1's 7045), and on the two composite-sum targets: h6_10, a band of $x_1 + \cdots + x_{12} - 3x_{13}$ (702 against 8775), and h6_09, an XOR of two sums (392 against 2582); the smallest are where v1's characters already capture the structure (h6_02, a sum times a parity: 428 against 464).

**The exposed suites agree.** On `dlheld5` v2 is clearly better than `nets_best` on 10 of 12 targets ($-2.40$; one target, h5s1, is learned by no one, and one is a tie) and on `dlheld4` on all 12 ($-1.68$). The margins are larger on `dlheld5`, as expected for a family inside S's class with oscillatory accept sets (h5s5: 421 against the networks' 12254), and v1 never learns 5 of its 8 signed-sum targets, v2 only h5s1. These suites are not evidence for Q1 (Section 6.9), but nothing in them contradicts the blind result.

**The networks.** Weight decay and the logistic loss help the networks on every held-out suite, by 0.7–1.0 in $\log_2$ on average over the sum-structured targets with the pre-registered variants (1.05 on `dlheld6` with the two extra decays), and `nets_cewd` is the best of the three pre-registered variants on most of them (on `dlheld6` the heavier decays 0.3 and 1.0 are best on every sum-structured target); nothing suggests a misconfigured reference. The fairness audit's reading (Section 7) is that hundreds of samples is what gradient descent should need for these targets at $d = 30$.

## 6.7 Held-out results: images

Table 19 gives the test error of v2, v1, `hgb`, `mlp_strong`, the strengthened MLPs and the two controls at every cell of the two fresh QuickDraw sets; the other pre-registered MLPs (`mlp`, `mlp_l2unit`,

| set | $n$ | v2 | v1 | hgb | mlp _strong | mlp _wide | mlp _wide1 | mlp _xl | mlp _best | mlp _best+ | mlp _pool | cnn |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| qd_a | 100 | **42.9** | 46.8 | 60.6 | 47.9 | 48.0 | 48.6 | — | 47.9 | 47.9 | — | — |
| | 300 | **29.9** | 33.6 | 43.1 | 36.3 | 36.0 | 36.6 | — | 36.3 | 36.0 | — | — |
| | 1000 | **22.6** | 26.3 | 31.3 | 28.3 | 28.4 | 29.4 | — | 28.3 | 28.3 | 25.6 | 21.0 |
| | 3000 | **18.6** | 21.3 | 24.3 | 23.0 | 22.7 | 23.4 | — | 23.0 | 22.7 | — | — |
| | 10000 | **15.0** | 17.2 | 19.4 | 18.4 | 17.8 | 18.7 | — | 18.4 | 17.8 | 16.4 | 12.0 |
| | 30000 | **13.4** | 15.1 | 16.6 | 14.8 | 14.3 | 15.8 | — | 14.8 | 14.3 | — | — |
| | 60000 | **12.0** | 14.2 | 15.2 | 13.3 | 12.9 | 14.2 | 12.5 | 13.3 | 12.5 | 12.3 | — |
| qd_b | 100 | **47.3** | 50.7 | 59.5 | 49.5 | 49.5 | 50.5 | — | 49.5 | 49.5 | — | — |
| | 300 | **35.7** | 39.9 | 46.0 | 39.9 | 39.5 | 41.1 | — | 39.9 | 39.5 | — | — |
| | 1000 | **27.2** | 30.7 | 35.3 | 31.7 | 31.9 | 33.1 | — | 31.7 | 31.7 | 28.9 | — |
| | 3000 | **22.3** | 25.1 | 28.0 | 26.4 | 26.1 | 26.9 | — | 26.4 | 26.1 | — | — |
| | 10000 | **17.9** | 20.3 | 22.7 | 21.2 | 20.6 | 21.4 | — | 21.2 | 20.6 | 19.6 | — |
| | 30000 | **15.6** | 17.8 | 19.4 | 17.1 | 16.6 | 17.2 | — | 17.1 | 16.6 | — | — |
| | 60000 | 14.9 | 16.4 | 17.8 | 16.2 | 15.2 | 15.5 | **14.8** | 16.2 | 14.8 | 14.7 | — |

**Table 19:** Test error (%) on the held-out QuickDraw sets (pixel-permuted; seed mean, seed 0 at $n \geq 30000$; `mlp_xl` and the controls seed 0 only). Bold: the lowest error among v2 and the references (every column but the per-cell minima `mlp_best`, `mlp_best+` and the two controls). The control `mlp_pool` is permutation-invariant like v2, and at `qd_b`, $n = 60000$ its 14.7 is below the bold 14.8; `cnn` uses the pixel layout. `mlp_best` is the pre-registered per-cell hindsight minimum over Part 8's MLP variants; `mlp_best+` adds `mlp_wide`, `mlp_wide1` and `mlp_xl`. `mlp_pool` is `mlp_strong` given v2's pooled sums as extra inputs (a control); `cnn` sees the unpermuted pixels (context, not a reference).

| reference | v2, $n \geq 10000$: cells better / tie / worse | v2, $n \geq 10000$ mean $\log_2$ | v2, $n \leq 3000$ mean $\log_2$ | v1, $n \geq 10000$ mean $\log_2$ |
|---|---|---|---|---|
| `mlp_strong` (pre-registered) | 6: 3 / 3 / 0 | −0.18 | −0.22 | +0.01 |
| `mlp_best` (pre-registered) | 6: 3 / 3 / 0 | −0.18 | −0.22 | +0.01 |
| `mlp_wide` | 6: 2 / 4 / 0 | −0.13 | −0.21 | +0.06 |
| `mlp_wide1` | 6: 4 / 2 / 0 | −0.21 | −0.25 | −0.02 |
| `mlp_xl` (CPU-matched) | 2: 0 / 2 / 0 | −0.02 | — | +0.17 |
| `mlp_best+` | 6: 2 / 4 / 0 | −0.11 | −0.21 | +0.07 |
| `mlp_pool` (control) | 4: 0 / 4 / 0 | −0.07 | −0.14 | +0.12 |
| `cnn` (context; uses the layout) | 1: 0 / 0 / 1 | +0.32 | +0.11 | +0.52 |

**Table 20:** Q2: v2 (and v1) against each image reference on `qd_a` and `qd_b`. Cells are (set, $n$) pairs; verdicts by Part 8's rule (relative difference above 10% and above 2 s.e.). Mean $\log_2$(error/reference error), errors floored at 0.002.

`mlp_strong1`) are never better than `mlp_strong`, enter through `mlp_best`, and are compared with v2 in `study/part9/RESULTS.md`. Table 20 gives the summaries and Figure 8 the learning curves.

**Q2 as pre-registered.** At $n \geq 10000$ v2's mean $\log_2$ error ratio is $-0.18$ against `mlp_strong` and against `mlp_best` (6 cells: 3 better, 3 tied, none worse), and v1's is $+0.01$ on the same cells. All three parts of Q2 are met. v2 also wins at small $n$ ($-0.22$ at $n \leq 3000$, 7 of 8 cells better), where Part 8's v1 was only level, and it has the lowest error of every pre-registered and strengthened reference at 13 of the 14 cells; at the 14th (`qd_b`, $n = 60000$) `mlp_xl` is 0.1 points lower, a tie.

**Against the strengthened references.** The 512-wide ensemble `mlp_wide` is the strongest MLP reference at every $n$ from 3000 to 30000 on both sets, and it narrows the large-$n$ margin to $-0.13$ (2 better, 4 tied); at `qd_b`, $n = 60000$ it is level with v2 (15.2% against 14.9%). The CPU-matched ensemble `mlp_xl` (three 1024-1024 networks at the weight decay `mlp_wide` chose on validation,

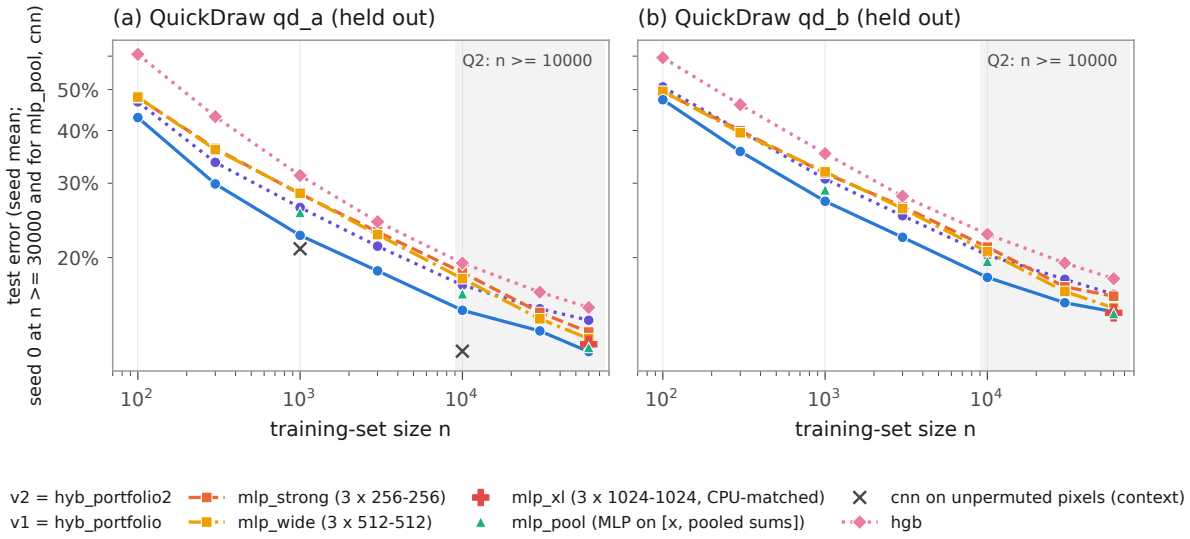



**Figure 8:** Test error against $n$ on the two held-out QuickDraw sets (log–log; seed mean, seed 0 at $n \geq 30000$; `mlp_pool` and `cnn` seed 0 at every $n$, `mlp_xl` at $n = 60000$ only). The shaded band is Q2's range $n \geq 10000$.

$n = 60000$ only; 1207–1359 seconds of wall time per fit on the shared 4-core machine, against v2's 1178–1757 CPU seconds) is the best MLP reference at $n = 60000$ on both sets: 12.5% on `qd_a` (v2 12.0%) and 14.8% on `qd_b` (v2 14.9%), two ties. So at the largest $n$ a wide enough MLP ensemble closes the gap. Against `mlp_best+`, the per-cell minimum over every MLP reference, v2's mean at $n \geq 10000$ is $-0.11$ (2 better, 4 tied, none worse).

**The control: an MLP on v2's pooled sums.** `mlp_pool` is `mlp_strong` with v2's pooled subset sums (scales 4, 9, 16, pooling sets computed from each fit's training rows) appended to the pixels. It is the strongest MLP at every $n$ it was run at, and at $n = 60000$ it matches v2: 12.2% against 12.0% on `qd_a` and 14.7% against 14.9% on `qd_b`. v2 keeps a margin at $n = 1000$ and $n = 10000$ (for example 15.0% against 16.4% on `qd_a` at $n = 10000$), but by the cell rule 5 of the 6 cells are ties and one (`qd_a`, $n = 1000$) favours v2; with a single-seed control the rule is a weak test. So most of v2's large-$n$ advantage over the MLPs is carried by the *representation*, the locality recovered from pixel correlations, which a gradient-trained network can use as well as the discrete members can. A small CNN that sees the *unpermuted* pixels (one network, fixed weight decay; context, run on `qd_a` at two cells for compute) has lower error than every permutation-invariant learner at both cells: 21.0% at $n = 1000$ and 12.0% at $n = 10000$, against v2's 22.6% and 15.0% (by the cell rule a tie at $n = 1000$, clearly better at $n = 10000$); v2 reaches the CNN's $n = 10000$ error only at $n = 60000$ (12.0%).

**What carries v2's image error.** From 03:06 UTC the evaluator recorded the stack weights and every member's out-of-fold error on the training rows (all cells with $n \geq 3000$, except `qd_a` at $n = 3000$, seed 0). `ggbt` is the best single member by out-of-fold error at every recorded cell (for example 12.7% against `gknn`'s 15.6% and `gbt`'s 15.9% at `qd_a`, $n = 60000$), and its stacking weight grows with $n$ to 0.50–0.55 at $n = 60000$; `gknn` carries the larger weight at $n = 3000$ in 4 of the 5 recorded fits. Part 8's members (`gbt`, `et`, raw `knn`, `lin`) together keep weights of 0.12–0.31. A post-hoc ablation (`study/part9/posthoc_ggbt.py`; run after the results were read, so explanatory) refitted `ggbt` alone at $n = 60000$ on both sets (Table 21). Replacing the bank by raw pixels at the

| `ggbt` alone, $n = 60000$ | bank | rounds | `qd_a` | `qd_b` |
|---|---|---|---|---|
| frozen | $x$, pooled sums, 300 pairs | early stopping (378–400) | 11.9% | 14.4% |
| no pairs | $x$, pooled sums | early stopping | 12.1% | 14.6% |
| raw pixels | $x$ | early stopping | 14.2% | 16.8% |
| 100 rounds | $x$, pooled sums, 300 pairs | 100 | 13.6% | 16.2% |
| v2 (the whole portfolio) | | | 12.0% | 14.9% |

**Table 21:** Post-hoc ablation of v2's `ggbt` member: test error of the member refitted alone on all $n = 60000$ training rows (seed 0), with the frozen `fit_ggbt`.

same early-stopped schedule raises its error by 0.22–0.25 in $\log_2$; cutting the schedule to v1's 100 rounds raises it by 0.17–0.19; dropping the prototype pairs costs only 0.02. So the gain over Part 8's tree member splits roughly evenly between the pooled subset sums and longer boosting, and the prototype pairs carry almost nothing. `ggbt` alone on all $n$ rows is as good as the whole of v2 (11.9% against 12.0% on `qd_a`, 14.4% against 14.9% on `qd_b`): at this size v2's final booster is the one trained on its stacking split (54000 rows, about 49000 after its inner validation split of 5000), while the refit trains on 55000 (60000 less the same 5000-point inner split).

### 6.8 Compute

**Toy.** v2's largest fit uses 0.33, 0.82 and 0.65 of one network run's FLOPs on `dlheld4`, `dlheld5` and `dlheld6` (v1: 0.73, 0.76, 0.70), and at most 16–33 CPU seconds. The compute audit found the sum-block search charged to within about ±40%; it is not scaled by DFG's budget share, so the by-construction worst case rises from Part 8's $\approx 1.1$ to $\approx 1.2$ network runs, and the budget holds by measurement, not by construction.

**Images.** By its own count v2 uses 0.21–0.62 of the budget of three MLP runs on the held-out cells, the most at $n = 3000$; the compute audit's correction of the library-tree formulas puts that cell at $\approx 0.72$ (0.69–0.81). Equal FLOPs are far from equal CPU time. A v2 fit takes 1178–1757 CPU seconds at $n = 60000$, against 130–205 seconds of wall time for one `mlp_strong` fit and 631–812 seconds for one `mlp_wide` fit (the MLP script first recorded one seed's wall time divided by three at its single-seed cells; the figures here are corrected); the gates member's LightGBM spends most of it on per-tree column copies, which the FLOP convention does not charge. Prediction, not charged for any learner, costs v2 about $2.8 \times 10^8$ FLOPs per query at $n = 60000$, about 500× an MLP's, because up to three nearest-neighbour scans over the training set are kept.

### 6.9 Audits, exposures and deviations

Four independent auditors, who opened no held-out result file, examined code and process (02:13–03:01 UTC on 27 October, before any v2 toy result or Q2 image cell was read; during and just after that window, at 02:40 and 03:03 UTC, the orchestrator saw v2's `qd_a`/`qd_b` test errors at $n \leq 1000$ in progress logs, Table 14): protocol and leakage, compute accounting, discreteness and permutation invariance, and the fairness of the references (`study/part9/audit/`). None reported a blocker. Their major findings changed the protocol (Table 14) and the wording of the claims; Table 22 lists them with what was done.

**Exposures of the toy test suites.** Three things weakened the held-out toy evidence before `dlheld6` existed.

- At the start of its work, builder S ran an over-long text search that printed the source of `dlheld4`'s target definitions. S never loaded or ran the suite. But its design note later used held-out `sumpar`'s form as its example of an unsupported form; the orchestrator's own task prompt had quoted a truncation of the same target; and the block-size cap (24) and the number of modulators (4) were set minutes after the exposure. `dlheld4` is therefore not a test of the sum-feature design.
- `dlheld5` was drawn after S froze and is blind to target identities. But its family was written 35 seconds after the orchestrator read S's design summary, including its list of forms it cannot represent, and every family member lies inside S's representable class: one signed sum with an arbitrary accept set, optionally with one modulating character or one gate literal. Its accept sets are also more oscillatory than any `dlsymdev` or `dlheld4` target's (3–5 runs of accepted values, against 1–2), which costs a character search and a network far more than it costs one sum block.
- The orchestrator saw one `p9_sum` result line on `dlheld5` before v2 was frozen (h5t0: $n^* = 104$ against the networks' 770). Both parts of the integration rule were mechanical and reproduce exactly from dev files, so no integration decision depended on it.

**Other deviations.**

- *A dev test look by a killed builder.* The first I2 session scored its development runs on the MNIST and Fashion-MNIST test sets and once on the `qd_dev` test set at $n = 10000$, an integration-rule cell. `p9_bank`'s dev numbers are therefore slightly optimistic; it lost the rule anyway.
- *Two container restarts* killed running builders and jobs. v2's held-out image runs and the `hgb` image references therefore used a resumable per-cell evaluator with the harness's splits and random streams. From 03:06 UTC on 27 October it also stored the learner file's md5 and v2's member attribution with each cell.
- *An unapplied speed fix.* Builder I1 found that LightGBM's row-wise layout copies the sampled columns for every tree on the 3000-column bank, and recommended `force_col_wise`, which leaves predictions bit-identical and cuts `ggbt`'s time by about 2.2–2.7×. v2 was assembled with only the two textual changes, so the CPU figures in Section 6.8 are with the slow layout.

### 6.10 Answers to Q1 and Q2

- **Q1 (sum features): yes.** On the blind suite `dlheld6`, v2's mean $\log_2 n^*$ ratio against the pre-registered `nets_best` is $-1.45$ with 10 targets clearly better and none worse, and on the sum-structured targets it is $-1.53$ against v1's $+0.63$. With two more network weight decays it is $-1.25$ (10 better, 2 worse), and the criteria still hold. The exposed suites `dlheld5` and `dlheld4` agree. Part 8's loss on symmetric functions is removed: what v1 lacked was a feature class that contains functions of one signed sum, and the union-bound test pays for searching it with a few dozen nats.
- **Q2 (images at large $n$): yes, as pre-registered.** On `qd_a` and `qd_b` at $n \geq 10000$, v2's mean $\log_2$ error ratio is $-0.18$ against `mlp_strong` and against `mlp_best`, and v1's is $+0.01$. Against the strengthened references it is $-0.11$ (`mlp_best+`, no cell worse). But at $n = 60000$ the CPU-matched MLP ensemble ties v2 on both sets, so the large-$n$ advantage that remains is within what more network capacity buys.

| audit | major finding | action |
|---|---|---|
| protocol | `dlheld5`'s family was written after S's limits were read and lies inside S's class | Amendment 3: `dlheld6`, written blind; Q1 answered on it |
| protocol | `dlheld4`'s exposure went beyond the text search (prompt, parameters) | `dlheld4` secondary; design note corrected |
| compute | v2's image fits take 22–29 CPU-min at $n = 60000$, over I1's own 15-min target | CPU reported next to FLOPs |
| compute | a Fashion-MNIST $n = 3000$ fit would project to 0.94 (0.77–1.12) of the budget, invisible to the guard | that grid was not run; disclosed |
| discreteness | the pooling sets recover the pixel grid | wording; controls `mlp_pool` and `cnn` |
| discreteness | prototype pairs are continuous unfitted projections | wording |
| discreteness | what carries the image gain was not recorded; `ggbt`'s gain is confounded with longer boosting | attribution recorded from 03:06 UTC; confound disclosed |
| fairness | the image references are not at the FLOP frontier: a 512-wide MLP beats the 3-net ensemble at $n = 60000$ on dev | `mlp_wide`, `mlp_wide1`; Q2 also against `mlp_best+` |
| fairness | equal FLOPs is not equal CPU: v2 uses 4–12× the CPU of the budget's MLP runs | `mlp_xl`, CPU-matched |
| fairness | an MLP is the comparator only formally, since the member recovers locality | wording; `cnn` as context; information-matched GD methods cited, not run |

**Table 22:** Major findings of the four Part 9 audits and what was done. Minor findings (commit times, per-cell md5s, a dropped reference in the tables, a prediction-cost table, the strong MLP's quantile encoding on QuickDraw, toy weight decay) were all acted on; see `study/part9/PART9.md`, §8.

- **What the image answer means.** It is not evidence that discrete learning beats gradient-based learning on images. The member that wins is gradient-boosted trees over subset sums on pixel sets that recover the image grid from correlations; an MLP given those sums matches v2 at $n = 60000$; about half of the trees' gain over Part 8's tree member comes from boosting longer; a v2 fit at $n = 60000$ takes about nine times as long as the strong MLP ensemble on the same set (20–30 CPU minutes against 2–3.5 minutes of wall time); and a CNN given the layout is better (clearly so at $n = 10000$, where v2 needs $n = 60000$ to match it). What Part 9 shows is that a permutation-invariant learner can recover enough spatial structure from pixel statistics alone to beat the pre-registered permutation-invariant MLP references on fresh image data, and that this structure, not discreteness, carries the large-$n$ result.

# 7 Discussion

**What discrete search buys** [Heuristic]. On sparse Boolean targets, enumerating a dictionary that contains the target's features and admitting them by union-bound tests needs about $\ln N$ samples per feature. It pays for search in FLOPs. At $d = 30$ that fits inside the FLOPs of one gradient-descent training run, up to degree 4 on all the data. The networks pay for their search in samples: about $d^{k-1}$ for a $k$-parity in Part 3's law, and $d^{1.9}$ and $d^{2.4}$ measured for $k = 3, 4$. This is why DFG's advantage is largest on parity-like targets and grows with $d$, while on dense targets it is a constant factor that a regularized linear readout already achieves (Section 5.3). That much agrees with what SQ and leap theory lead one to expect [2, 3, 8]. The experiments measure the factors at equal FLOPs against concrete networks, on held-out targets, and they show where the toy-side advantage comes from: DFG's ablations and the portfolio's attribution both locate it in exhaustive

character search.

Part 9 extends this to symmetric functions. A symmetric function of a block of $k$ signed inputs has no degree-1 Fourier weight when it is even, and its weight is spread over many characters of degrees $2, 4, \ldots$, so a character search pays for each one; a gradient-trained network has to find the block's direction from the degree-2 statistic $\mathbb{E}[f\, xx^\top]$, whose spike along the block's sign vector (the signs $s_i$ on $S$, 0 off it) competes with sampling noise of order $\sqrt{d/n}$. This is the information-exponent picture of single-index learning [4, 7]: hundreds of samples at $d = 30$ are what gradient descent should need. A dictionary that contains "subset sum with $\pm 1$ signs" can instead round a coarse spectral estimate to the exact block and test it with a union bound. On the blind suite that gives a factor of about $2^{1.45} \approx 2.7$ in samples at $d = 30$ against the best pre-registered network variant, and $2^{1.25} \approx 2.4$ against five variants; whether the factor grows with $d$, as it does for parities, was not tested.

**What it does not buy.** The advantage does not transfer to standard tasks through DFG's recipe.

- On tabular data DFG's dictionaries are about as good as a plain MLP and worse than a regularized one. What beats the MLPs on fresh tabular data, on average and not significantly, is a classical stack of tree ensembles, nearest neighbours and logistic regression, a known result about tabular data [12, 25].
- On images the useful features are not in a small dictionary. Part 9's portfolio v2 has lower error than every permutation-invariant MLP reference on fresh QuickDraw sets at 13 of the 14 cells, and ties the CPU-matched ensemble at the 14th, where that ensemble is slightly lower; but the ablations show what its winning member is made of: subset sums over pixel sets that recover the pixel grid from correlations, boosted for longer than Part 8's tree member, by gradient-boosted trees. An MLP handed the same pooled sums matches it at $n = 60000$. The image result is therefore mostly about a representation, locality recovered from data [6, 19], which gradient-trained networks can use as well, and a network that is given the layout (a small CNN, run on one set at two sizes) has lower error than both, clearly so at $n = 10000$.

**The honest claim.** One learner, v2, beats the study's gradient-descent-trained networks on both kinds of task in the pre-registered sense: on the blind toy suite (Q1), on fresh tabular data on average (Part 8; v2 is v1 there), and on two fresh image sets against the pre-registered MLP references at $n \geq 10000$ (Q2; mean $\log_2$ error ratio $-0.18$, 3 cells better and 3 tied). The qualifications are as large as the claim.

- It is a portfolio that dispatches by input type: the toy wins come from DFG's character and sum-block search, the tabular wins from the classical stack, the image wins from boosted trees over correlation-pooled sums.
- It is not discrete in any strong sense on images. Boosting is functional gradient descent, the prototype-pair features are continuous projections drawn from the data, and the pooling recovers spatial structure that the protocol meant to exclude.
- Equal FLOPs are not equal compute. A v2 image fit takes 20–30 CPU minutes at $n = 60000$, against 2–3.5 minutes of wall time for the strong MLP ensemble, and its prediction costs about 500× an MLP's per query. An ensemble of three 1024-wide MLPs whose wall time per fit (20–23 minutes) is about v2's CPU time ties it at $n = 60000$ on both image sets.
- The references are concrete networks: $\mu$P MLPs of one width on the toy suites, MLPs of width 256–1024 on images, all with standard recipes; the hindsight minima `nets_best` and `mlp_best`

favour them. No information-matched gradient method (a locally connected network on the recovered neighbourhoods, or a CNN on a recovered layout) was run.

**Limitations.**

- **The references.** *Toy networks:* one width-128 architecture at one learning rate and a fixed step count, in up to five loss and weight-decay settings. Wider networks, longer training and better checkpoint choice would plausibly lower their $n^*$ further (an audit estimated 0.1–0.3 in $\log_2$ from longer training, Appendix A). *MLPs:* no augmentation or tuned architectures; the CNN was run only as context, at two cells.
- **Power.** The fresh sets are 10 and 12 toy targets, 10 tabular datasets (sign test $p = 0.11$) and two image sets with one seed at $n \geq 30000$.
- **Exposure.** Two of Part 9's three held-out toy suites turned out to be exposed to the sum-feature design (Section 6.9); only `dlheld6` is blind to it, and its family was written by an agent from the pre-registered one-line description of the suite, with three `dlsymdev` development targets as a format example and sym9 as the motivating failure, not from an independent catalogue of signed-sum structure. Several held-out toy suites in Parts 6–8 were written within the study, mostly from target families already in the development suites.
- **Compute accounting.** FLOPs of library trees are analytic estimates, and the audits found them inexact: Part 8's LightGBM charge is off by 0.05–1.7×, and on QuickDraw the tree counts are off by about 0.6–1.5×. The largest image budget fractions are reported with the audits' corrections: about 0.74 in Part 8 (0.58–0.90 depending on the per-bin cost) and about 0.72 for v2 (0.69–0.81). CPU time and prediction cost are reported, not charged.
- **Not run.** Several cells, references and secondary-design evaluations were cut for compute (Appendix A, Section 6.9).

**Open problems.**

1. *An information-matched comparison on images.* Does a gradient-trained network that uses the same correlation-derived neighbourhoods (a locally connected or graph-convolutional layer over the pooling sets [22]) beat v2's boosted gates at equal compute? The `mlp_pool` control suggests it would come close.
2. *Scaling of the sum-feature advantage.* At $d = 30$ it is a constant factor. Does it grow with $d$, as the character search's advantage does for parities, or stay constant, as the information-exponent picture suggests for single sums?
3. *Beyond one signed sum.* One blind target, a residue class mod 3 of a 13-input count, is a function of one signed sum, but v2's search does not find it within 16384 samples; only the most regularized network learns it, at 12304. Sums with unequal integer weights and several modulating characters are outside DFG v1.2's search.
4. *One mechanism.* Is there one discrete learner, rather than a portfolio, that wins on both kinds of task? Parts 8 and 9 attribute each win to a different component.

## References


[1] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. The merged-staircase property: a necessary and nearly sufficient condition for SGD learning of sparse functions on two-layer neural networks. In *Conference on Learning Theory (COLT)*, 2022. arXiv:2202.08658.

[2] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. In *Conference on Learning Theory (COLT)*, 2023. arXiv:2302.11055.

[3] Boaz Barak, Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning: SGD learns parities near the computational limit. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Gérard Ben Arous, Reza Gheissari, and Aukosh Jagannath. Online stochastic gradient descent on non-convex losses from high-dimensional inference. *Journal of Machine Learning Research*, 22(106):1–51, 2021.

[5] Claude (Anthropic). Gradient descent on Boolean targets as noise-limited greedy feature growth: experiments, discrete models, and generalization bounds. Prompted by Kaarel Hänni. https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper, 2026. Parts 1–5 of the study this paper continues.

[6] Adam Coates and Andrew Y. Ng. Selecting receptive fields in deep networks. In *Advances in Neural Information Processing Systems 24*, 2011.

[7] Alex Damian, Loucas Pillaud-Vivien, Jason D. Lee, and Joan Bruna. Computational-statistical gaps in Gaussian single-index models. In *Proceedings of the Thirty-Seventh Conference on Learning Theory (COLT)*, 2024.

[8] Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Pareto frontiers in neural feature learning: data, compute, width, and luck. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2309.03800.

[9] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. AutoGluon-Tabular: robust and accurate AutoML for structured data, 2020. arXiv:2003.06505.

[10] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[11] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[12] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[13] David Ha and Douglas Eck. A neural representation of sketch drawings, 2017. arXiv:1704.03477; ICLR 2018.

[14] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[15] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[16] Michael Kearns. Efficient noise-tolerant learning from statistical queries. *Journal of the ACM*, 45(6):983–1006, 1998.

[17] Diederik P. Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[18] Béatrice Laurent and Pascal Massart. Adaptive estimation of a quadratic functional by model selection. *Annals of Statistics*, 28(5):1302–1338, 2000.

[19] Nicolas Le Roux, Yoshua Bengio, Pascal Lamblin, Marc Joliveau, and Balázs Kégl. Learning the 2-D topology of images. In *Advances in Neural Information Processing Systems 20*, 2007.

[20] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[22] Behnam Neyshabur. Towards learning convolutions from scratch. In *Advances in Neural Information Processing Systems 33*, 2020.

[23] Randal S. Olson, William La Cava, Patryk Orzechowski, Ryan J. Urbanowicz, and Jason H. Moore. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10:36, 2017.

[24] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[25] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: deep learning is not all you need. *Information Fusion*, 81:84–90, 2022.

[26] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[27] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[28] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. arXiv:1708.07747.

[29] Greg Yang and Edward J. Hu. Tensor programs IV: feature learning in infinite-width neural networks. In *International Conference on Machine Learning (ICML)*, 2021.

# A Protocols and audits

Every part was reviewed by independent audit agents: separate AI sessions with their own instructions, which reported to the orchestrating session. This appendix lists what each audit of Parts 6–8 found and what was done about it; the Part 9 audits are listed in Section 6.9, with their reports in `study/part9/audit/`. The Part 8 audit reports are in `study/hybrid/audit/`. The Part 6 and 7 audit scripts and logs are in the session scratch directory, not in the repository; their findings are recorded in the write-ups `study/discrete_learner/DISCRETE-LEARNER.md` and `study/standard_tasks/STANDARD-TASKS.md`.

## A.1 Part 6

Five independent audits checked the comparison:

- data and leakage;
- compute accounting;
- reproduction;
- two reviews of the write-up, one of numbers and one of framing.

**Found sound.**

- There was no test-data leakage. The training sets match the networks' row for row, for every $n$ and seed, across suites and $d$, and the networks' saved weights reproduce their recorded training error on these sets.
- DFG sees only $(X, y)$ and a random generator.
- An independent ufunc-level FLOP count agrees with DFG's counter to within 0.98–1.00.

**Found wrong, and what was done.** Each item was fixed or disclosed in the write-up.

- *The degree-5/6 search* was added knowing par5 was in `dlheld`. The blind row now uses v1 without it.
- *The development suites.* `random` was used during development, and the small-$n$ rules were tuned on the designed suite's test error. Both are labelled as not out of sample.
- *A compute-budget bug at $d = 120$.* The cross-validation folds each had the whole budget, costing up to 2.7 runs of a 20000-step network per fit. v1.1 fixes it.
- *A tie-break bias toward low-index inputs.* The input-permutation wrapper was added before any run on `dlheld2`, and the size of the bias was measured (Section 3.4).
- *A seed-lucky $n^*$ for mux.* The fresh-seed value (about 87 against 65) is reported.
- *Several overclaims in drafts.* They were corrected after the numbers and framing reviews (commit `c12c3b5b`).

## A.2 Part 7

- **Protocol audit.** No builder touched a held-out dataset. The plan to designate a primary design did not hold: `dfg_dirs` froze its second version after the other designs' held-out runs had started. All three are therefore reported on an equal footing.

| audit finding | what was done |
|---|---|
| *protocol and leakage* | |
| F1: toy runs had no input permutation; all designs break ties toward low column indices (dev effect up to 0.25 in $\log_2$ per target with $n^* \lesssim 64$) | every held-out toy suite re-run with inputs permuted per fit; that run is the headline; natural-order runs kept (suite means differ by 0.01–0.04) |
| F2: the Part 7 HELDOUT set is not blind at the reference level (the primary's builder read Part 7's per-dataset held-out tables) | headline tabular evidence = HELDOUT2; Part 7 tabular set and images labelled "references known to the designers"; no fresh image set |
| F3: `random`, `dlheld`, `dlheld2` are development, post-hoc or once-used data for DFG | toy headline = `dlheld3`; other suites labelled |
| F4: `hyb_union` stops on a 480 s wall clock | disclosed; cells with $\geq$ 300 CPU s flagged (Appendix C) |
| F5: large-input settings never run on real data before the held-out runs | disclosed |
| F6: `hyb_stage` chooses its kNN metric before the CV split | disclosed; slightly optimistic validation, no effect on test sets |
| F7: `hyb_union` froze once, aborted its dev run after 12 s, kept developing on dev data, and froze again | disclosed; only dev data involved |
| F8: designation scores are in-sample to development | disclosed |
| *fairness of the references* | |
| F1: the toy networks are one untuned, unregularized square-loss configuration; in the audit's probe, AdamW with weight decay 0.1 cut $n^*$ by 1.7–1.9× (thr10, mux, par3) and logistic loss plus weight decay by at least 1.9–2.0× (thr10, mux) | added `nets_wd` (all five suites), `nets_cewd` (`dlheld3` only; the audit preferred it on all suites) and `nets_best`; in the full runs the per-target cut is 1.0–3.7× |
| F2–F4: final-checkpoint reading, truncated training, width | disclosed, not re-run: a test-set oracle checkpoint would lower the networks' suite-mean $\log_2 n^*$ by 0.00–0.20 (`dlheld3`: 0.09), a realizable validation checkpoint recovered little, and longer training would plausibly lower it by another 0.1–0.3 |
| F5: literature framing: sparse-parity wins are structural, dense-target wins are constant factors | reflected in Sections 5.3 and 5.6 |
| F6, F7: the MLP references are lightly tuned single networks, and the baseline's per-pixel standardization handicaps it on images | added `mlp_strong` and `mlp_strong1` on HELDOUT2, the images and the Part 7 tabular set at $n \leq 10000$; `mlp_l2unit` used on images at every $n$ |
| F8: `mlp_reg` is not uniformly stronger than `mlp` (fixed learning rate, final checkpoint, $\lambda$ chosen on 10–30 points) | disclosed; `mlp_best` covers those cells with hindsight, and `mlp_strong` selects decoupled weight decay on validation |
| F9: `mlp_best` covers different variants per split | variant lists given (Table 25) |
| F10: baseline checkpoint selection is near random at $n \leq 300$; the step cap and constant learning rate bind at $n > 25600$ | disclosed; `mlp_strong` uses cosine decay to 0 |
| F11: Fashion-MNIST `mlp_l2unit` was missing at $n \geq 30000$, and missing reference rows were silently dropped | both rows run; `compare_hybrid.py` lists every missing row |

**Table 23:** Part 8 audit findings and actions (1 of 2): the protocol and fairness audits. Source: `study/hybrid/HYBRID.md`, §8, and the reports in `study/hybrid/audit/`.

| audit finding | what was done |
|---|---|
| *compute accounting* | |
| F1: `hyb_portfolio`'s LightGBM FLOP formula is off by 0.05–1.7× | corrected image maximum ≈ 0.74 of the budget reported |
| F2: kNN prediction costs up to 350× an MLP's per query | disclosed (Appendix C) |
| F3: equal FLOPs are not equal CPU time | CPU seconds reported for image cells |
| F4: `hyb_stage`'s tree count is uncapped and partly undercounted | disclosed; 2 cells over budget |
| F5: toy compute is bounded by measurement, not by construction (worst case ≈ 1.1) | disclosed; largest measured toy fit 0.75 |
| F6: `hyb_union`'s wall-clock stop makes large fits load-dependent | disclosed; cells flagged |
| F7, F8: ExtraTrees count plausible within about 2× and $\leq 0.3\%$ of the budget; small items approximated or uncounted, $< 2\%$ of the budget | disclosed; no budget verdict changes |
| *discreteness* | |
| F1: on real data the primary is a classical stack; DFG runs on 1 of 23 held-out datasets | wording (Section 5.9); post-hoc ablations (Section 5.6); the suggested no-DFG run on mushroom was not done |
| F2: logistic regression alone carries the halfspace-like toy targets | `lin` column added to every toy table |
| F3: the spectral $(u \cdot z)^2$ features have continuous directions | disclosed; not ablated |
| F4: attribution was not recorded | post-hoc member ablations |
| F5–F7: boosting is functional gradient descent; kNN is Euclidean memory; logistic regression on continuous inputs is a linear background | wording |
| F8: `hyb_union`'s parity family is idle on real data; `hyb_stage`'s final stage choice is not recorded | disclosed |
| F9: toy runs not input-permuted (as protocol F1) | as protocol F1 |

**Table 24:** Part 8 audit findings and actions (2 of 2): the compute and discreteness audits.

| split | variants in `mlp_best` |
|---|---|
| HELDOUT2 | `mlp`, `mlp_l2`, `mlp_strong`, `mlp_strong1` |
| MNIST, Fashion-MNIST | `mlp`, `mlp_unit` ($n \leq 10000$), `mlp_l2unit`, `mlp_strong`, `mlp_strong1` |
| QuickDraw (Part 9) | `mlp`, `mlp_l2unit`, `mlp_strong`, `mlp_strong1` |
| Part 7 tabular | `mlp`, `mlp_plus` ($n \leq 3000$), `mlp_l2`, `mlp_strong` and `mlp_strong1` ($n \leq 10000$) |
| dev | `mlp`, `mlp_l2` |

**Table 25:** The MLP variants over which `mlp_best` takes its per-cell hindsight minimum. `mlp_plus` is effectively a learning-rate-grid variant, since its weight decay is negligible. On QuickDraw, `mlp_best+` adds `mlp_wide`, `mlp_wide1` and `mlp_xl` (Section 6.7).

- **MLP-fairness audit.**
  - *Finding.* The baseline MLP was under-regularized at small $n$: its only regularizer was early stopping, chosen on as few as 10 validation points.
  - *Action.* `mlp_l2` was added, and 18 of DFG's 23 cell wins over the baseline became ties or losses against it.
  - *Calibration.* The audit also checked the baseline's large-$n$ errors against ranges it recalled for 2-layer MLPs. They are within those ranges except MNIST (2.5% against about 1.6–2%). The ranges were recalled, not checked against sources.
- **Compute audit.** The FLOP counters were checked against an instrumented count. True over reported is 0.98–1.00 for `dfg_bits`, 1.00–1.05 for `dfg_gates` and 1.01–1.05 for `dfg_dirs`. `dfg_gates`'s final readout refit is not reserved in its budget check.
- **Claims check** (commit `9fd619b8`). It corrected the following: which wins survive, the image error ranges, the pendigits range, the gradient-boosting comparison, the framing of the test stall, and protocol details.
- **Incidents.**
  - *A crash and a post-freeze fallback.* `dfg_gates` crashed on MNIST when a Cholesky factorization failed. A jitter fallback was added that runs only when the factorization fails, so fits that never crashed are bit-identical.
  - *The one fit where the fallback is known to have fired* (MNIST $n = 10000$, seed 1) is an outlier: 0.233 against 0.075–0.076 for the other seeds. It raises that learner's image mean $\log_2$ ratio from +0.36 to +0.41, and its retry FLOPs are not counted.
  - *Memory kills.* Several image runs were killed by the memory limit and rerun with resume support, with no effect on the results.

## A.3 Part 8

Four code-and-process audits ran while the primary's held-out jobs were running: protocol and leakage, compute accounting, discreteness, and fairness of the references. Each audit reports that it read no held-out result of a Part 8 learner.

- None found test-data use inside a learner, or held-out access by a builder.
- The freezes hold: the md5 sums match, and a re-run of the frozen primary reproduced its dev errors exactly.

- HELDOUT2 and `dlheld3` were fixed (commit `04a215b2`, 06:05) before the first builder started (06:22).

Tables 23 and 24 list the findings that changed something or needed disclosure. No learner was changed.

**Deviations from the Part 8 protocol.**

- *Headline sets.* The protocol listed all held-out sets as plain held-out data. After the audits, the headline was restricted to the fresh `dlheld3` and HELDOUT2.
- *Toy headline.* It uses permuted inputs, which the protocol did not specify.
- *References.* The protocol named `mlp_best` as "best of five" variants; in fact 2–3 existed per split at the time. This report adds `nets_wd`, `nets_cewd`, `nets_best`, `mlp_strong` and `mlp_strong1`, all of which make the references stronger. The calibration references `lin` and `hgb` enter no verdict.
- *Secondary designs.* The protocol evaluates all three frozen designs on all held-out sets.
  - `hyb_stage` and `hyb_union` were run on the held-out toy suites only on `dlheld3`, with inputs permuted.
  - `hyb_union` was not run on the Part 7 tabular set.
- *Ablations.* They are post hoc.

**Scope cuts for compute.**

- `mlp_strong` was not run at five large Part 7 cells (adult $n = 30000$ and 38842, letter 15000, connect_4 30000 and 57557), or on the dev sets.
- `nets_cewd` was run on `dlheld3` only.
- The real ablations cover $n \leq 3000$ with seeds 0–1, on HELDOUT2 and the images only.
- `hgb` was not run on the `designed` toy suite.

# B Per-suite and per-dataset tables

The tables in this appendix were generated by `make_tables.py` (in this paper's folder) from the study's result files, with the study's own functions, and checked against its markdown tables. Toy entries are $n^*$ at $d = 30$; "never" means test error 0.1 was not reached on the grid $n \leq 16384$. "portfolio" is `hyb_portfolio` with inputs permuted per fit. The DFG columns are the Part 6 runs, with inputs in natural order except on `dlheld2`. The last column is $\log_2$ of the portfolio's $n^*$ over `nets_best`'s, which equals `nets_wd` on these four suites.

| target | definition | DFG v1 | portfolio | `lin` | `nets` | `nets_wd` | $\log_2$ |
|---|---|---|---|---|---|---|---|
| and3 | AND$(x_1, x_2, x_3)$ | 101 | 25 | 68 | 94 | 70 | −1.51 |
| andxor | AND$(x_1x_2, x_3x_4)$ | 43 | 40 | never | 376 | 204 | −2.34 |
| dict | $x_1$ | 22 | 16 | 27 | 47 | 40 | −1.33 |
| dlist5 | decision list on 5 literals | 61 | 32 | 85 | 110 | 85 | −1.41 |
| dnf3 | 3-term DNF, 3 literals per term | 220 | 147 | never | 505 | 353 | −1.27 |
| maj3 | MAJ$(x_1, x_2, x_3)$ | 57 | 31 | 56 | 112 | 91 | −1.53 |
| maj9 | MAJ$(x_1..x_9)$ | 107 | 88 | 80 | 263 | 149 | −0.75 |
| majpar | MAJ$(x_1x_2, x_3x_4, x_5x_6)$ | 62 | 60 | never | 1380 | 1026 | −4.08 |
| mux | $x_4 \,?\, x_1x_2x_3 : \text{MAJ}(x_1, x_2, x_3)$ | 65 | 77 | never | 1156 | 729 | −3.25 |
| par2 | $x_1x_2$ | 16 | 16 | never | 200 | 109 | −2.77 |
| par3 | $x_1x_2x_3$ | 16 | 55 | never | 843 | 443 | −3.00 |
| par4 | $x_1x_2x_3x_4$ | 21 | 56 | never | 2728 | 1726 | −4.95 |
| parmaj | $\text{MAJ}_3 \oplus \text{MAJ}_3$ on $x_{1..6}$ | 221 | 214 | never | 743 | 434 | −1.02 |
| rand0 | random 6-gate formula | 23 | 16 | never | 222 | 125 | −2.97 |
| rand1 | random 6-gate formula | 104 | 94 | never | 463 | 316 | −1.75 |
| rand2 | random 6-gate formula | 28 | 16 | 50 | 91 | 59 | −1.89 |
| rand3 | random 6-gate formula | 189 | 181 | never | 1016 | 692 | −1.93 |
| rmaj9 | MAJ of three $\text{MAJ}_3$ | 167 | 163 | never | 2027 | 1273 | −2.97 |
| sparsepoly | sign$(2x_1 + x_2x_3 + x_4x_5x_6)$ | 73 | 95 | never | 2205 | 1238 | −3.71 |
| stair5 | sign$(x_1 + x_1x_2 + \cdots + x_1 \cdots x_5)$ | 112 | 109 | never | 1514 | 871 | −2.99 |
| thr10 | sign$(w \cdot x_{1..10})$, Gaussian $w$ | 164 | 121 | 113 | 563 | 307 | −1.34 |

**Table 26:** Toy suite `designed` (development; in-sample for DFG and for the Part 8 designs).

| target | DFG v1 | portfolio | lin | hgb | nets | nets_wd | $\log_2$ |
|---|---|---|---|---|---|---|---|
| rf00 | 28 | 40 | 83 | 60 | 98 | 62 | −0.62 |
| rf01 | 48 | 48 | never | 120 | 467 | 256 | −2.42 |
| rf02 | 51 | 49 | never | 228 | 266 | 191 | −1.97 |
| rf03 | 87 | 71 | never | 139 | 425 | 276 | −1.97 |
| rf04 | 54 | 48 | never | 111 | 342 | 203 | −2.07 |
| rf05 | 74 | 76 | never | 250 | 873 | 493 | −2.70 |
| rf06 | 27 | 16 | 37 | 58 | 61 | 50 | −1.64 |
| rf07 | 105 | 97 | never | 168 | 407 | 281 | −1.53 |
| rf08 | 52 | 21 | 69 | 53 | 85 | 62 | −1.54 |
| rf09 | 57 | 45 | never | 292 | 320 | 182 | −2.01 |
| rf10 | 56 | 36 | 66 | 53 | 103 | 79 | −1.15 |
| rf11 | 16 | 56 | never | 416 | 864 | 445 | −3.00 |
| rf12 | 93 | 90 | never | 524 | 963 | 610 | −2.76 |
| rf13 | 53 | 48 | never | 213 | 401 | 245 | −2.33 |
| rf14 | 56 | 53 | never | 378 | 714 | 404 | −2.92 |
| rf15 | 51 | 22 | 67 | 53 | 108 | 75 | −1.78 |
| rf16 | 54 | 36 | never | 214 | 354 | 200 | −2.46 |
| rf17 | 28 | 56 | never | never | 3049 | 1715 | −4.94 |
| rf18 | 51 | 46 | never | 93 | 174 | 112 | −1.29 |
| rf19 | 56 | 52 | never | 366 | 902 | 654 | −3.65 |

**Table 27:** Toy suite `random`: 20 random formulas (development data for DFG). Networks: 2 seeds.

| target | definition | DFG v1$^-$ | DFG v1.1 | portfolio | lin | hgb | nets | nets_wd | $\log_2$ |
|---|---|---|---|---|---|---|---|---|---|
| hrf0 | random formula, 6 inputs | 68 | 86 | 46 | 191 | 63 | 192 | 133 | −1.54 |
| hrf1 | random formula, 6 inputs | 117 | 118 | 117 | never | 461 | 1203 | 789 | −2.75 |
| hrf2 | random formula, 6 inputs | 44 | 46 | 54 | 93 | 60 | 107 | 90 | −0.74 |
| inter2 | AND of two halfspaces on $x_{1..10}$ | 704 | 583 | 394 | never | 416 | 859 | 454 | −0.20 |
| maj15 | MAJ($x_1..x_{15}$) | 132 | 130 | 118 | 110 | 1278 | 355 | 210 | −0.83 |
| majmaj5 | MAJ of three MAJ$_5$ | 1252 | 816 | 1161 | never | 1638 | 2847 | 2355 | −1.02 |
| majpar5 | MAJ($x_1x_2, \ldots, x_9x_{10}$) | 141 | 123 | 127 | never | 4127 | 5229 | 4648 | −5.20 |
| majxor | MAJ$_5 \oplus$MAJ$_5$ on $x_{1..10}$ | 782 | 766 | 771 | never | 4076 | 1313 | 792 | −0.04 |
| mux6 | 6-multiplexer | 197 | 197 | 123 | never | 188 | 481 | 358 | −1.54 |
| par5 | $x_1 \cdots x_5$ | never | 56 | 839 | never | never | 7245 | 7096 | −3.08 |
| stair6 | sign($x_1 + \cdots + x_1 \cdots x_6 + \frac{1}{2}$) | 212 | 207 | 180 | never | 695 | 1967 | 1495 | −3.06 |
| thr30 | sign($w \cdot x$), Gaussian $w$ on all 30 | 191 | 158 | 149 | 148 | 1645 | 528 | 311 | −1.06 |

**Table 28:** Toy suite `dlheld` (held-out 1). DFG v1$^-$ is frozen v1 without its degree-5/6 search, the blind run; DFG v1.1 is post hoc.

| target | definition | DFG v1.1 | portfolio | lin | hgb | nets | nets_wd | $\log_2$ |
|---|---|---|---|---|---|---|---|---|
| dnf4x2 | $(x_1 \wedge x_2) \vee \cdots \vee (x_7 \wedge x_8)$ | 235 | 112 | never | 98 | 440 | 306 | −1.45 |
| h2rf0 | random formula, 7 inputs | 117 | 108 | never | 220 | 527 | 389 | −1.85 |
| h2rf1 | random formula, 7 inputs | 85 | 31 | 103 | 58 | 127 | 103 | −1.73 |
| maj21 | MAJ($x_1..x_{21}$) | 134 | 122 | 116 | 1956 | 413 | 216 | −0.83 |
| sym8 | $[\lvert x_1 + \cdots + x_8 \rvert \geq 4]$ | 395 | 401 | never | 2689 | 504 | 377 | +0.09 |
| thr20 | sign($w \cdot x_{1..20}$), Gaussian $w$ | 136 | 130 | 117 | 1262 | 593 | 304 | −1.23 |
| xmaj | $x_1x_2$ MAJ($x_3, x_4, x_5$) | 61 | 63 | never | 1538 | 1552 | 871 | −3.80 |

**Table 29:** Toy suite `dlheld2` (held-out 2; blind for DFG v1.1, inputs permuted).

| dataset | $n$ | MLP | MLP-L2 | dfg_bits | dfg_gates | dfg_dirs | linear | hgb |
|---|---|---|---|---|---|---|---|---|
| mnist | 300 | 0.186 | – | 0.203 | 0.177 | 0.177 | 0.186 | 0.198 |
| | 3000 | 0.075 | – | 0.101 | 0.095 | 0.098 | 0.113 | 0.063 |
| | 60000 | 0.025 | – | 0.072 | 0.071 | 0.055 | 0.078 | 0.022 |
| fashion_mnist | 300 | 0.252 | – | 0.241 | 0.318 | 0.249 | 0.257 | 0.272 |
| | 3000 | 0.169 | – | 0.181 | 0.184 | 0.182 | 0.210 | 0.154 |
| | 60000 | 0.110 | – | 0.142 | 0.151 | 0.156 | 0.165 | 0.105 |
| adult | 300 | 0.170 | 0.181 | 0.177 | 0.177 | 0.174 | 0.168 | 0.180 |
| | 3000 | 0.155 | 0.156 | 0.153 | 0.156 | 0.152 | 0.151 | 0.147 |
| | 38842 | 0.143 | 0.140 | 0.141 | 0.141 | 0.141 | 0.146 | 0.126 |
| letter | 300 | 0.379 | 0.306 | 0.375 | 0.355 | 0.356 | 0.380 | 0.348 |
| | 3000 | 0.098 | 0.094 | 0.161 | 0.138 | 0.148 | 0.235 | 0.104 |
| | 15000 | 0.033 | 0.036 | 0.069 | 0.109 | 0.114 | 0.223 | 0.041 |
| pendigits | 300 | 0.054 | 0.041 | 0.092 | 0.079 | 0.075 | 0.107 | 0.100 |
| | 3000 | 0.010 | 0.010 | 0.023 | 0.018 | 0.014 | 0.059 | 0.018 |
| | 8244 | 0.006 | 0.005 | 0.013 | 0.012 | 0.010 | 0.052 | 0.011 |
| connect_4 | 300 | 0.305 | 0.306 | 0.294 | 0.300 | 0.286 | 0.294 | 0.310 |
| | 3000 | 0.229 | 0.210 | 0.243 | 0.223 | 0.241 | 0.245 | 0.215 |
| | 57557 | 0.143 | 0.141 | 0.197 | 0.179 | 0.189 | 0.236 | 0.158 |
| dna | 300 | 0.144 | 0.119 | 0.060 | 0.093 | 0.101 | 0.104 | 0.057 |
| | 2390 | 0.058 | 0.048 | 0.046 | 0.052 | 0.049 | 0.058 | 0.034 |
| nursery | 300 | 0.093 | 0.097 | 0.090 | 0.105 | 0.102 | 0.104 | 0.079 |
| | 3000 | 0.006 | 0.002 | 0.022 | 0.009 | 0.025 | 0.077 | 0.005 |
| | 9718 | 0.000 | 0.000 | 0.003 | 0.000 | 0.001 | 0.076 | 0.000 |
| ring | 300 | 0.254 | 0.097 | 0.078 | 0.128 | 0.150 | 0.273 | 0.082 |
| | 3000 | 0.048 | 0.029 | 0.042 | 0.052 | 0.032 | 0.249 | 0.031 |
| | 5550 | 0.033 | 0.027 | 0.037 | 0.039 | 0.029 | 0.251 | 0.025 |
| twonorm | 300 | 0.034 | 0.033 | 0.040 | 0.031 | 0.035 | 0.030 | 0.062 |
| | 3000 | 0.028 | 0.030 | 0.030 | 0.026 | 0.032 | 0.024 | 0.037 |
| | 5550 | 0.027 | 0.027 | 0.026 | 0.026 | 0.032 | 0.024 | 0.033 |
| phoneme | 300 | 0.192 | 0.184 | 0.209 | 0.206 | 0.235 | 0.261 | 0.176 |
| | 3000 | 0.137 | 0.132 | 0.158 | 0.146 | 0.147 | 0.243 | 0.115 |
| | 4053 | 0.128 | 0.133 | 0.154 | 0.141 | 0.142 | 0.243 | 0.110 |
| waveform_40 | 300 | 0.196 | 0.180 | 0.173 | 0.157 | 0.190 | 0.181 | 0.186 |
| | 3000 | 0.149 | 0.136 | 0.136 | 0.127 | 0.148 | 0.122 | 0.141 |
| | 3750 | 0.143 | 0.127 | 0.118 | 0.120 | 0.146 | 0.119 | 0.133 |
| mushroom | 300 | 0.013 | 0.012 | 0.010 | 0.006 | 0.007 | 0.022 | 0.006 |
| | 3000 | 0.002 | 0.000 | 0.000 | 0.001 | 0.001 | 0.000 | 0.000 |
| | 6093 | 0.001 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |

**Table 30:** Part 7: seed-mean test error on the 13 held-out datasets at $n = 300$, 3000 and the full pool. MLP-L2 was run on tabular data at $n \leq 3000$. On images at $n \geq 30000$ the DFG adaptations are seed 0 only. hgb on Fashion-MNIST was run in Part 8. Source: study/standard_tasks/results/.

| dataset | cells | vs `mlp_best` (b/w) | vs `mlp_strong` (b/w) | `hgb` vs `mlp_best` | full $n$ | `mlp_best` error | variant | portfolio error |
|---|---|---|---|---|---|---|---|---|
| *HELDOUT2 (fresh)* | | | | | | | | |
| texture | 5 | +0.61 (0/0) | +0.36 (0/0) | +1.80 | 4125 | 0.0027 | strong | 0.0061 |
| churn | 5 | −0.35 (2/0) | −0.36 (2/0) | −0.11 | 3750 | 0.0469 | strong | 0.0363 |
| wine_quality_white | 5 | −0.15 (2/0) | −0.19 (2/0) | −0.10 | 3674 | 0.3881 | l2 | 0.3216 |
| page_blocks | 5 | −0.02 (0/0) | −0.03 (0/0) | +0.40 | 4105 | 0.0222 | strong | 0.0222 |
| clean2 | 5 | −1.12 (1/0) | −1.92 (3/0) | −1.14 | 4948 | 0.0000 | l2 | 0.0000 |
| led24 | 4 | −0.14 (1/0) | −0.15 (1/0) | −0.00 | 2400 | 0.2925 | l2 | 0.2933 |
| ann_thyroid | 5 | −0.77 (4/0) | −0.78 (4/0) | −0.68 | 5400 | 0.0176 | l2 | 0.0074 |
| banana | 5 | −0.00 (0/0) | −0.04 (0/0) | +0.26 | 3975 | 0.0958 | strong1 | 0.0974 |
| krkopt | 6 | +0.19 (0/2) | +0.19 (0/2) | +0.31 | 21042 | 0.1333 | strong | 0.2037 |
| sleep | 8 | −0.05 (0/0) | −0.10 (3/0) | +0.04 | 95908 | 0.2301 | strong | 0.2243 |
| *Part 7 tabular set (references known to the designers)* | | | | | | | | |
| adult | 7 | −0.11 (2/0) | −0.20 (4/0)$^\dagger$ | −0.05 | 38842 | 0.1401 | l2 | 0.1280 |
| letter | 6 | −0.13 (1/0) | −0.15 (2/0)$^\dagger$ | +0.21 | 15000 | 0.0327 | mlp | 0.0259 |
| pendigits | 5 | +0.19 (0/0) | −1.28 (5/0) | +0.99 | 8244 | 0.0053 | l2 | 0.0072 |
| connect_4 | 7 | +0.03 (0/1) | −0.00 (0/0)$^\dagger$ | +0.10 | 57557 | 0.1412 | l2 | 0.1566 |
| dna | 4 | −0.77 (3/0) | −0.81 (3/0) | −0.60 | 2390 | 0.0402 | strong | 0.0281 |
| nursery | 5 | +0.70 (0/2) | +0.64 (0/2) | +0.34 | 9718 | 0.0000 | l2 | 0.0007 |
| ring | 5 | −0.50 (2/0) | −0.94 (3/0) | −0.05 | 5550 | 0.0268 | l2 | 0.0254 |
| twonorm | 5 | −0.08 (0/0) | −0.15 (0/0) | +0.73 | 5550 | 0.0270 | mlp | 0.0254 |
| phoneme | 5 | −0.24 (2/0) | −0.27 (2/0) | −0.09 | 4053 | 0.1256 | strong | 0.0834 |
| waveform_40 | 5 | −0.21 (2/0) | −0.31 (3/0) | +0.09 | 3750 | 0.1269 | l2 | 0.1149 |
| mushroom | 5 | −0.31 (0/0) | −0.36 (0/0) | −0.14 | 6093 | 0.0000 | l2 | 0.0000 |
| *images (references known to the designers)* | | | | | | | | |
| mnist | 7 | +0.08 (1/2) | +0.08 (1/2) | +0.40 | 60000 | 0.0143 | strong | 0.0198 |
| fashion_mnist | 7 | +0.03 (0/2) | +0.03 (0/2) | +0.11 | 60000 | 0.0966 | strong | 0.1072 |

**Table 31:** Part 8: `hyb_portfolio` per held-out real dataset. "vs `mlp_best`" is the mean over cells of $\log_2$(error/reference error), followed by (better/worse) resolved cells; likewise "vs `mlp_strong`". $^\dagger$`mlp_strong` covers only the cells with $n \leq 10000$. The last four columns give the full training pool, the seed-mean error there of `mlp_best`, the variant that attains it (without the `mlp_` prefix), and the portfolio's error.

**Part 9 held-out toy suites.** Columns as in Table 17: v2 is `hyb_portfolio2` and v1 `hyb_portfolio`, both with inputs permuted per fit; `nets_best` is the per-target minimum over the network variants run on the suite. These two suites are exposed to the sum-feature design (Section 6.9) and are secondary evidence. Definitions: Section 6.2 and `study/part9/PART9.md`, §2.2.

| target | v2 | v1 | lin | hgb | nets | nets_wd | nets_cewd | nets_best | $\log_2$ v2 / nets_best | $\log_2$ v2 / v1 |
|---|---|---|---|---|---|---|---|---|---|---|
| h5s0 | 205 | 11042 | never | never | 1454 | 803 | 867 | 803 | −1.97 | −5.75 |
| h5s1 | never | never | never | never | never | never | never | never | both never | both never |
| h5s2 | 1154 | 4409 | never | 10860 | 3322 | 1865 | 1850 | 1850 | −0.68 | −1.93 |
| h5s3 | 1757 | never | never | never | 8160 | 6779 | 6454 | 6454 | −1.88 | $-\infty$ |
| h5s4 | 184 | 415 | never | 1465 | 2744 | 1801 | 1891 | 1801 | −3.29 | −1.18 |
| h5s5 | 421 | never | never | never | 13832 | 13610 | 12254 | 12254 | −4.86 | $-\infty$ |
| h5s6 | 573 | never | never | never | 6227 | 4771 | 3963 | 3963 | −2.79 | $-\infty$ |
| h5s7 | 6760 | never | never | never | 13863 | 12878 | 7641 | 7641 | −0.18 | $-\infty$ |
| h5t0 | 104 | 146 | never | 2886 | 770 | 459 | 352 | 352 | −1.75 | −0.48 |
| h5t1 | 95 | 116 | never | 993 | 683 | 393 | 325 | 325 | −1.78 | −0.30 |
| h5rf0 (control) | 268 | 311 | never | 2658 | 2673 | 2056 | 1547 | 1547 | −2.53 | −0.22 |
| h5rf1 (control) | 98 | 101 | never | 9281 | 3352 | 2859 | 2512 | 2512 | −4.69 | −0.05 |

**Table 32:** Toy suite `dlheld5` (held out; drawn after the sum-feature design froze, from a family inside its representable class).

| target | v2 | v1 | lin | hgb | nets | nets_wd | nets_cewd | nets_best | $\log_2$ v2 / nets_best | $\log_2$ v2 / v1 |
|---|---|---|---|---|---|---|---|---|---|---|
| band10 | 432 | 3468 | never | never | 5920 | 4722 | 3440 | 3440 | −2.99 | −3.00 |
| ex4of9 | 194 | 1658 | never | never | 1073 | 741 | 411 | 411 | −1.08 | −3.09 |
| majsym | 189 | 352 | never | 2840 | 1542 | 1062 | 595 | 595 | −1.66 | −0.90 |
| ssym8 | 157 | 1410 | never | 8072 | 1160 | 715 | 396 | 396 | −1.34 | −3.17 |
| sumpar | 164 | 168 | never | never | 3487 | 2619 | 1837 | 1837 | −3.48 | −0.04 |
| sym11 | 203 | 2386 | never | never | 934 | 656 | 374 | 374 | −0.88 | −3.56 |
| symand | 170 | 289 | never | 1307 | 498 | 382 | 370 | 370 | −1.12 | −0.77 |
| symxor | 403 | 766 | never | 6909 | 1706 | 946 | 969 | 946 | −1.23 | −0.93 |
| tail13 | 176 | 436 | never | never | 612 | 458 | 420 | 420 | −1.26 | −1.31 |
| dnf3x3 (control) | 118 | 118 | never | 148 | 493 | 373 | 392 | 373 | −1.66 | +0.00 |
| h4rf0 (control) | 48 | 48 | never | 160 | 349 | 206 | 240 | 206 | −2.10 | +0.00 |
| h4rf1 (control) | 56 | 51 | 158 | 61 | 211 | 143 | 181 | 143 | −1.35 | +0.14 |

**Table 33:** Toy suite `dlheld4` (held out, fixed before any Part 9 design; its definitions were seen by the sum-feature builder).

# C Compute accounting

**Units.**

- *FLOPs.* A multiply-add counts as 2. Every fit-time operation is counted, including model selection, cross-validation and out-of-fold fits; prediction is not counted.
- *Toy unit:* one network run, $6P \cdot 64 \cdot T + 10P \cdot T$ FLOPs ($1.62 \times 10^{11}$ at $d = 30$).
- *Real unit:* one MLP training run at size $n$, $6P \cdot 128 \cdot S$ FLOPs. The baseline MLP's model selection costs 3 runs, and `mlp_l2`'s costs 5.
- *Budgets per fit:* DFG (Part 6), one toy unit; the DFG adaptations (Part 7), one real unit;

the Part 8 designs and Part 9's `hyb_portfolio2`, one toy unit or three real units (Part 9's compute: Section 6.8). `mlp_strong` costs 6 real units.

Library learners such as the tree boosters must charge an explicit analytic FLOP estimate.

**Part 6.**

- *Fit compute.* DFG's largest per-fit compute is 0.08–0.46 of a network run, depending on the suite (Table 3). It occurs at $n = 16384$, in the degree-4 search, the extensions, least squares and the logistic refinement. At $n \leq 512$ DFG uses at most 7.5% of a run at $d = 30$ (under 0.3% at $n \leq 32$), and up to 14–15% at $d = 60$ and 120.
- *The counter.* An independent ufunc-level count agrees with DFG's counter to within 0.98–1.00.
- *CPU time.* DFG runs at about 8–12 GFLOP/s at $d = 30$ and 15–18 GFLOP/s at $d = 60$ and 120 (float64 numpy driven from Python); the networks run at 32–50 GFLOP/s (float32 under XLA). So at $n = 16384$ a DFG fit takes 0.25–1.7× a network's CPU time at $d = 30$, and 0.26–0.52× at $d = 120$.

**Part 7.**

- *Counter accuracy.* Against an instrumented count, true over reported FLOPs is 0.98–1.00 (`dfg_bits`), 1.00–1.05 (`dfg_gates`) and 1.01–1.05 (`dfg_dirs`).
- *Within budget:* `dfg_dirs` in every cell by its own count (maximum 0.99, MNIST $n = 60000$); given the measured undercount, that cell may be marginally over.
- *Over budget:*
  - `dfg_gates` in 7 cells, at most 1.07×: letter, pendigits and connect_4 at large $n$, and two image cells.
  - `dfg_bits` on images: up to 3.45× on Fashion-MNIST at $n \geq 10000$, and 1.2× on MNIST at $n = 60000$. These cells count against it.

**Part 8: fit FLOPs.**

| design | toy (per network run) | Part 7 tabular | HELDOUT2 | images | dev |
|---|---|---|---|---|---|
| `hyb_portfolio` | 0.20–0.75 by suite | 0.24 | 0.09 | 0.54 | 0.14 |
| `hyb_stage` | 0.74 (`dlheld3`) | 1.17 (2 of 59 cells over) | 0.77 | 0.23 | 0.36 |
| `hyb_union` | 0.34 (`dlheld3`) | not run | 0.38 | 0.17 | 0.09 |

These are the largest per-fit fractions of the budget, by the learners' own counts. The own counts are uncorrected for the problems below.

- *LightGBM charge.* `hyb_portfolio` charges $10\,n\,d + 8 \cdot 31 \cdot d \cdot 255$ FLOPs per tree. Against an algorithmic count that tracks CPU time, this is off by 0.05–1.7×. It overcounts on toy and low-cardinality inputs and undercounts on Fashion-MNIST at about 800–27000 training rows.
- *Corrected image maxima:* about 0.74 at Fashion-MNIST $n = 3000$ (0.58–0.90 depending on the per-bin cost, against the own count of 0.54), about 0.41 at MNIST $n = 3000$, and about 0.31 for both datasets at $n = 60000$.
- *Tabular and toy.* Tabular cells other than mushroom stay at most 0.15 after correction; mushroom, where DFG is counted exactly, is at most 0.24. On the toy suites the LightGBM

term is overcounted and DFG is counted exactly, so the largest fit is 0.75. Toy compute is bounded by measurement, not by construction: the audit's worst-case bound is about 1.1.

- ***hyb_stage*** ran over on letter at $n = 10000$ and 15000, at 1.17 of the budget. Its trees and readouts have no budget guard.
- ***hyb_union*** stops on a 480 s wall clock, so its large fits depend on machine load. The cells with at least 300 CPU s per fit are MNIST $n = 30000$ and 60000, Fashion-MNIST 60000 and sleep 95908.

**Part 8: prediction cost.** Prediction is not counted for any learner. `hyb_portfolio`'s kNN members scan all $n$ training points under up to two metrics:

| cell | FLOPs per query | vs an MLP's $2P = 5.4 \times 10^5$ | whole test set (10000 points) |
|---|---|---|---|
| images, $n = 60000$ | $1.9 \times 10^8$ | $\approx 350\times$ | $1.9 \times 10^{12}$: 0.30 of the fit budget, 0.91 of one MLP training run |
| images, $n = 10000$ | $3.1 \times 10^7$ | $58\times$ | – |

- With prediction charged, the $n = 60000$ fits still stay within budget (fit $\approx 0.31$ plus test set $\approx 0.30$).
- The portfolio stores all $n$ training rows twice, about 0.38 GB in float32 at $n = 60000$, against the MLP's 0.27 M parameters.
- `hyb_stage` and `hyb_union` cost at most $12\times$ and $36\times$ an MLP per query.

**Part 8: CPU time.** Equal FLOPs are not equal time.

- *Rates, by the audit's count:* tree work 0.4–1.9 GFLOP/s, ExtraTrees 0.11–0.15, BLAS 57–111, the networks 32–50.
- *Image fits.* Process CPU seconds per `hyb_portfolio` fit (seed mean) at $n = 100$, 300, 1000, 3000, 10000, 30000, 60000:
  - MNIST: 14, 29, 58, 162, 158, 116, 237;
  - Fashion-MNIST: 37, 91, 214, 488, 335, 200, 314.

  The drop above $n = 3000$ is consistent with the switch to holdout stacking, and the drop above $n \cdot d = 10^7$ with the subsampled booster.
- *Against the MLPs.* At Fashion-MNIST $n = 3000$ the portfolio uses about 0.74 of the FLOP budget and about 8 CPU-minutes; the audit estimates 25–40 s for the three MLP runs that budget corresponds to.
- *Tabular.* On dev, every design's per-fit CPU time is at most 30 s, comparable to the MLPs.

# D How this work was done

This paper and the work it reports were produced by an AI system, Claude (Anthropic), on 25–27 October 2026. Kaarel Hänni prompted and steered it by posing the questions of Parts 6–9. No human has reviewed the code, the results or this text.

**Organisation.** One orchestrating session took each part's question from the user, wrote protocols, fixed held-out sets, ran evaluations and wrote the study's write-ups. It delegated six kinds of work to separate agents, which reported back to it:

- design work (*builders*);
- writing the blind held-out toy suite `dlheld6` (Part 9);
- reviews of code and process (*audits*);
- checks of the write-ups against the result files (*claims checks* or *numbers and framing reviews*);
- the first draft of this paper (Parts 6–8);
- pre-drafting `PART9.md` and applying the Part 9 claims-check corrections (Part 9).

Builders, auditors and the author of `dlheld6` had their own instructions and data restrictions; the restrictions on data access are recorded in the protocol files; the builders' were checked by the protocol audits, and the `dlheld6` author's from its transcript (its tool calls touched only its own folder and the scratchpad).

**Part 6** (25 October; commits on the working branch).

- `805bff0e`, 19:59: DFG's harness, a first DFG, and the held-out suite `dlheld`.
- `359a6b61`, 20:29: DFG v1 frozen for the held-out evaluation.
- `0220cf04`, 20:48: v1.1, made post hoc after v1's held-out run, with frozen v1 kept.
- `5716239b`, 21:03: the input-permutation wrapper, added after the reproduction audit found the tie-break bias, and before any run on `dlheld2`. `dlheld2` itself was committed after v1.1 (`70c604eb`).
- `43c842b5`, 21:30: the write-up and all results.
- `c12c3b5b`, 21:47: corrections from the numbers and framing reviews.

The networks on `dlheld` were trained, and their learning curves inspected, before v1 was frozen; DFG was run on that suite only after the freeze. `dlheld2` was written by DFG's designer.

**Part 7** (25–26 October).

- `1ae3b424`, 22:43: datasets, harness, MLP baseline and references.
- `ba1f1322`, 23:51: the three frozen adaptations with their dev results, and the post-freeze Cholesky fallback of `dfg_gates`.
- The MLP variants followed, with `mlp_l2` added after the fairness audit (`196a36d1`, `8c11bf6b`).
- `11e7eec7`, 02:01: the write-up.
- `9fd619b8`, 02:14: corrections from the claims check.

The three builders worked from the dev datasets only, and had about 70 minutes each.

**Part 8** (26 October).

- `04a215b2`, 06:05: `dlheld3` and HELDOUT2 fixed.
- 06:21–06:27: the shared harness, the protocol and the comparison script were committed. The first builder started at 06:22; the protocol was written before any design result was seen.

- `b3489586`, 09:11:20: the three frozen designs with full dev results, and the primary designated. The held-out jobs started at 09:11:27.
- `82d7877d` and `49807a59`, 10:26: the audit-driven additions (permuted toy runs, `lin`, network variants with weight decay and logistic loss, `mlp_strong`, coverage checks) and the four audit reports.
- 09:47–16:21: held-out results (the first was `hyb_portfolio` on MNIST, `e5a8cc30`, before the audit commits), references, secondary designs and post-hoc ablations.
- `dc51998a`, 16:59: the write-up, after a claims check.

**What the orchestrator had seen when it changed the evaluation (Part 8).** The four audits ran while the primary's held-out jobs were running. By the time the audit-driven changes were decided, the orchestrating session had seen parts of the primary's held-out results in the job logs:

- the per-dataset error curves of 7 tabular datasets (phoneme, ring, twonorm and waveform_40 from the Part 7 set; texture, wine_quality_white and sleep from HELDOUT2);
- the full MNIST learning curve, next to the MLP references then available;
- a few single-seed cells (connect_4 $n = 30000$, sleep $n = 10000$ and 30000);
- the summaries of the first two `random` toy shards, 4 of the 20 targets;
- the held-out toy results of the `hgb` reference.

Every change made on the audits' findings strengthens a reference, removes a bias in the learner's favour, or narrows the claim:

- the network variants;
- `mlp_strong`, with settings from the auditor's recommendation, smoke-tested on dev data only;
- the input permutation;
- the restriction of the headline to the fresh sets.

No learner was changed after its freeze.

**Job scheduling (Part 8).**

- To give the reference runs priority, the running jobs of `hyb_stage` on Fashion-MNIST, led24 and ann_thyroid were paused with SIGSTOP for about 50 minutes. `hyb_stage` has no wall-clock rule, and a stopped process accrues no CPU time, so neither its results nor its recorded CPU time are affected. No `hyb_union` job was paused.
- Two duplicate reference runs, `nets_wd` on `dlheld` and `mlp_strong` on pendigits, were started by mistake and stopped; one copy of each was kept.

**Part 9** (26–27 October). The protocol, the new development sets and the original held-out sets (`dlheld4`, `qd_a`, `qd_b`) were committed before any Part 9 design work (`82ca257d`, 20:14 UTC on 26 October), and an amendment fixing the dev image cells followed before design (`f6ad9afe`, 20:15). The later held-out toy suites came with amendments: `dlheld5` (Amendment 2, `7449313e`, 23:08, after builder S froze) and the Q1 suite `dlheld6` (Amendment 3, `0c4db784`, 02:54 UTC on 27 October, after v2 was frozen).

- *Builders.* Three independent agents built the sum-feature component (S) and the two image members (I1, I2) on development data and froze before their full dev evaluations. A container restart killed the first I2 session; a new agent resumed from its files (Section 6.9).
- *Integration and freeze.* The orchestrating session applied the pre-registered rule mechanically from dev result files and froze v2 at 02:11 UTC on 27 October; a one-line crash fix followed before any held-out result existed.
- *Audits.* Four auditors (protocol, compute, discreteness, fairness) examined code and process (02:13–03:01 UTC on 27 October) without opening any held-out result file (the protocol auditor read the one `p9_sum dlheld5` line the orchestrator had quoted in its chat); the orchestrator's looks at v2's $n \leq 1000$ progress lines, at 02:40 and 03:03, fell during and just after that window (see below). Their findings led to Amendments 3 and 4 (Table 14).
- *The blind suite.* `dlheld6` was written by a fresh agent that was told not to read the design notes, the other suites or any result; its tool calls touched only its own folder and the scratchpad.
- *What the orchestrator saw.* Before the freeze, three held-out log lines (v1 on `qd_a` and `qd_b`, and one `p9_sum` line on `dlheld5`); before Amendment 4, v2's `qd_a`/`qd_b` test errors at $n \leq 1000$ in progress logs. Both are recorded in the protocol.
- *Compute incidents.* Two container restarts; out-of-memory kills of `p9_gates`' $n = 60000$ dev fits (three times; rerun with the frozen file) and, in the held-out phase, of the resumed baseline `mlp` (twice), the `mlp_pool` control and one `ggbt` ablation, all on `qd_a` at $n = 60000$, when too many large jobs ran at once (all rerun from saved state, one at a time); and three compute cuts: the CNN context reference was reduced to one network at a fixed weight decay (recorded before it ran on `qd_a`/`qd_b`) and to two cells on `qd_a` (recorded once its $n = 1000$ cell had finished, before it ran past that cell), and the CPU-matched MLP ran at $n = 60000$ only, at the weight decay `mlp_wide` chose (recorded before those runs).

**This paper.** Its sections on Parts 6–8 were first drafted by a separate agent from the study's write-ups (`DISCRETE-LEARNER.md`, `STANDARD-TASKS.md`, `HYBRID.md` and their result tables), the protocols and the audit reports. The Part 9 section, and the Part 9 revisions of the other sections, were written by the orchestrating session from `PART9.md`, `study/part9/RESULTS.md`, the Part 9 protocol and audit reports, and the result files. `PART9.md`'s methods and process sections were pre-drafted by a separate agent; the corrections from the Part 9 claims check were applied by separate agents, one per group of files, and an independent agent checked each group's edits.

- The figures (`make_figures.py`) and the generated tables (`make_tables.py`) are computed directly from the result files with the study's own comparison functions.
- Their values were compared with the study's markdown tables.
- An independent claims check of the paper, the README and `PART9.md` (one checker per group of sections, then an adversarial re-check of each reported discrepancy) compared every claim with the result files, the protocol and the audit reports. Its 69 confirmed findings were applied, the edits were checked by independent verifiers, and their remaining findings were applied by the orchestrating session.

**Reproducibility.** All code is in study/: `discrete_learner/` (Part 6), `standard_tasks/` (Part 7), `hybrid/` (Part 8) and `part9/` (Part 9, with `PROTOCOL.md`, `PART9.md`, `RESULTS.md` and the audit reports). Each write-up ends with the commands that regenerate its results.

- The networks' raw outputs and the cached datasets are in a session scratch directory, not in the repository. The scripts regenerate them, and `study/hybrid/results/nets_final_err.json` holds the networks' final-step test errors used for the toy comparisons.
- The orchestration scripts of Part 8 (`study/hybrid/orchestration/`) are kept on the working branch only.
- Part 9's job drivers and queue scripts (among them `run_v2.sh`, `jax_refs.sh`, `cpu_refs*.sh`, `refs5.sh`, `nets5.sh`, `after_restart2.sh`, `refs_a4.sh`, `run_h6.sh`, `nets_r3.sh`, `final_q*.sh`), the builders' development scripts and the equivalence checks (`equiv.py`, `equiv2.py`) stayed in the session scratch directory and are in neither branch. The drivers only wrapped the committed scripts, and `PART9.md` gives the commands that reproduce the runs. The Part 6 and 7 audit scripts are likewise in the scratch directory (Appendix A).